

PROGRAMACIÓN I-2022

Teoría – Ma. Virginia Ainçhil

**TEMAS
de la
CLASE**

- ① Presentación de la materia
- ② Concepto Informática
- ③ Etapas de resolución de un problema por computadora
- ④ Concepto de Algoritmo
- ⑤ Concepto de Programa
- ⑥ Concepto de Datos y Tipos de Datos
- ⑦ Clasificación de Tipos de Datos Simples definidos por el lenguaje
- ⑧ Declaración de Datos en Pascal
- ⑨ Estructura general de un programa en Pascal
- ⑩ Ejercitación

Docentes

Teoría:

Miércoles y viernes de 8 a 10 hs. (aula 11)

Sábados 12/03, 26/03, 23/04, 7/05 y 28/05 de 9 a 13 hs.
(aula 11)

Profesores: Ma. Virginia Ainchil - Federico Cristina

Práctica: Miércoles y viernes de 10 a 12 hs. (varias aulas,
distribuidos en comisiones)

JTP: César Estrebou

Información de la Cátedra

En este curso trabajaremos con el entorno virtual Ideas.

URL: <http://ideas.info.unlp.edu.ar>

Deben solicitar inscripción al curso **“Programación I”**.

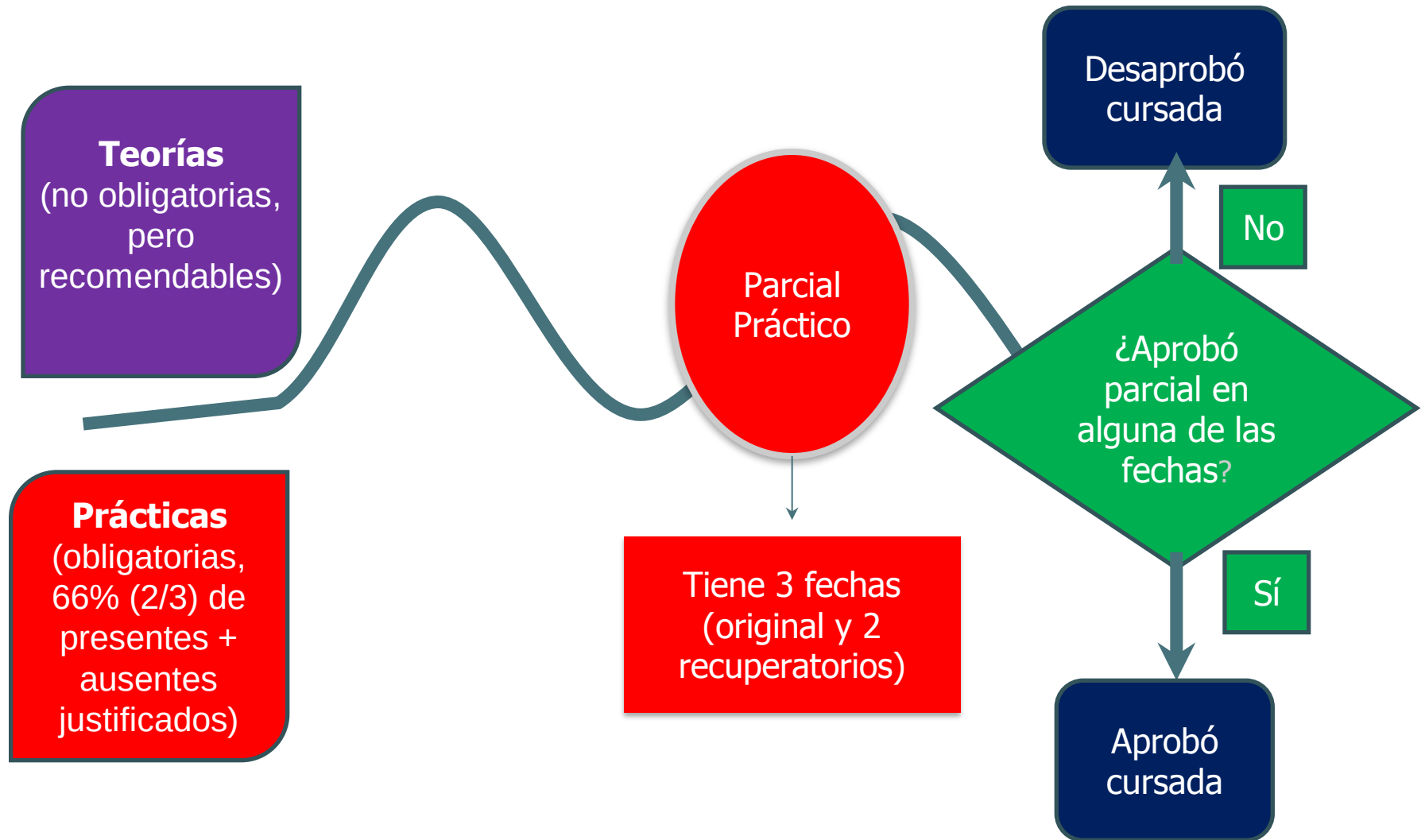
Dicho curso servirá para:

- Comunicarnos vía una mensajería interna
- Descargar las prácticas y materiales de estudio
- Obtener información de interés de la cursada
- Descargar el reglamento y programa

Una vez finalizado el curso, podrán encontrar información en:

Blog de la cátedra: <http://blogs.unlp.edu.ar/progra1/>

Aprobación de la cursada

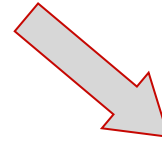


Aprobación de la materia



Examen final

- Se toma todos los meses y Uds. deben anotarse según el calendario académico.
- Programación I toma los días martes de la semana de finales (generalmente 8:30 hs.).
- La información se publica en el blog de la Cátedra y en la Cartelera Virtual.



Régimen de promoción

- Realizar la autoevaluación propuesta en el entorno Ideas y obtener un 6 o más.
- Aprobar el parcial en primera o segunda fecha.
- Aprobar una evaluación breve en el mes de julio con nota 6 o más.

Régimen de promoción de final

- ✓ Los alumnos que, **habiendo aprobado el parcial práctico (en primera o segunda fecha)**, no obtuvieran el mínimo de 6 en la evaluación breve, podrán rendir un examen recuperatorio, que abarcará los temas de la materia que el Profesor indique.

Los alumnos que aprueben la promoción y **se encuentren inscriptos en el curso bajo esta modalidad**, tendrán registrada su nota al final del curso.

Examen final

- ✓ **Plazo de validez de la habilitación:** La habilitación para rendir Examen Final, tendrá una validez de cuatro (4) semestres, inmediatos siguientes al del curso realizado.
- ✓ Para rendir los exámenes finales existirán mesas examinadoras integradas por los Profesores de las Áreas/ Asignaturas. Los alumnos **deberán inscribirse en las fechas determinadas**, según el calendario académico.

Concepto de Informática

La Informática es la ciencia que estudia el análisis y resolución de problemas utilizando computadoras.

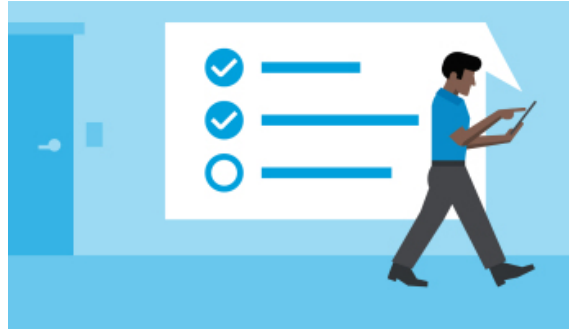
Ciencia se relaciona con una metodología fundamentada y racional para el estudio y resolución de los problemas.

La **resolución de problemas** aplicaciones en áreas muy diferentes tales como biología, comercio, control industrial, administración, robótica, educación, arquitectura, etc.

Computadora máquina digital y sincrónica, con capacidad de cálculo numérico y lógico y de comunicación con el mundo exterior. Ayuda al hombre a realizar tareas repetitivas en menor tiempo y con mayor exactitud.

Etapas de resolución de un problema por computadora

Del Problema a la Solución



Modelo

1er etapa: se sintetizan los requerimientos del problema y se simplifica el contexto y los datos a utilizar por el programa en la computadora.

Etapas de resolución de un problema por computadora

Del Problema a la Solución



2da etapa: la descomposición funcional nos ayudará a reducir la complejidad, a distribuir el trabajo y en el futuro a re-utilizar los módulos. Algoritmos.

Problema del Mundo Real

Análisis

Diseño

Algoritmos



**Solución
Modularizada**

Etapas de resolución de un problema por computadora

Del Problema a la Solución

Problema
Solución

Problema del
Mundo Real

Análisis

Diseño

Implementación

Programa

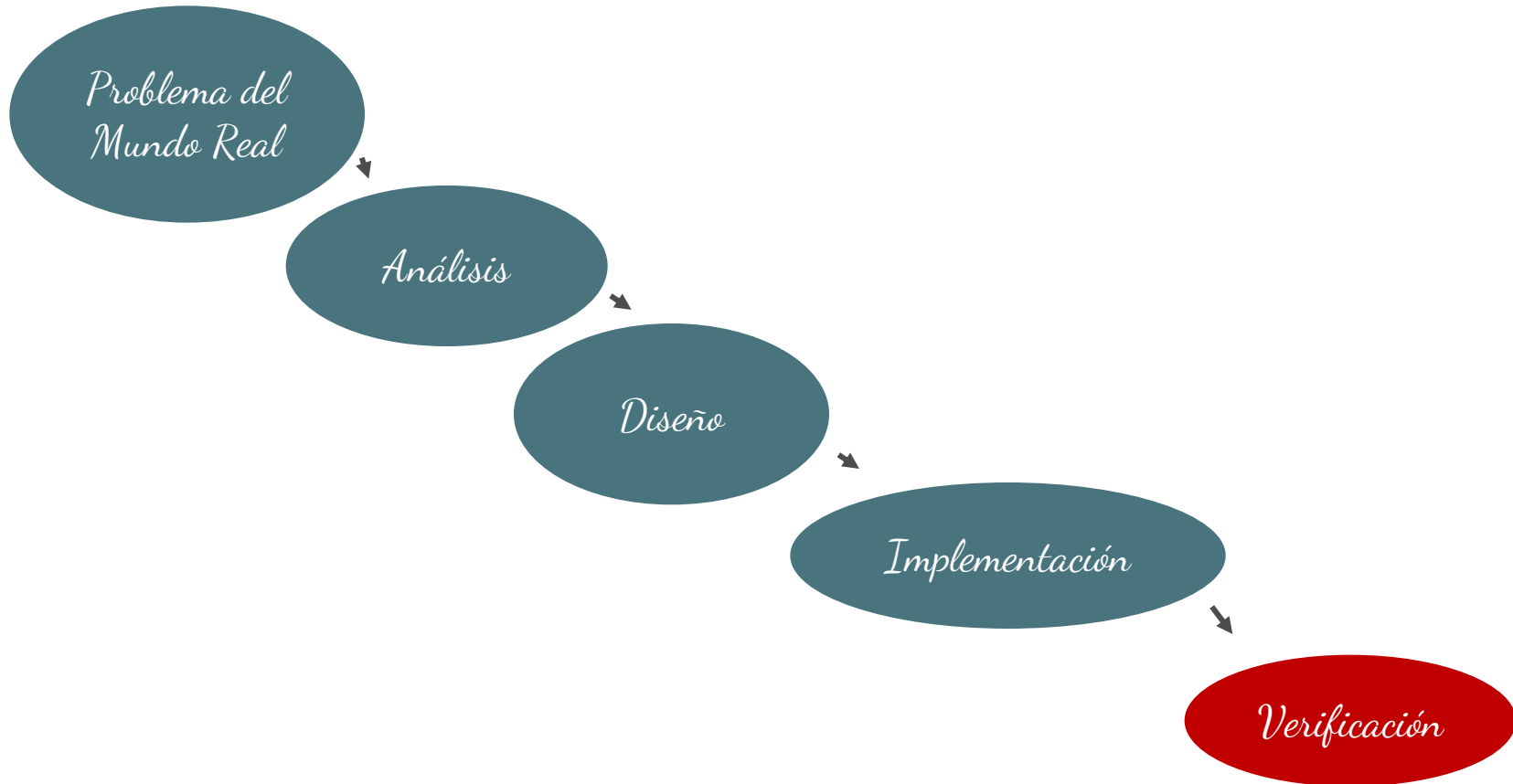
```
<?php·}·?>  
<?php·if(·$this->item->typ  
$this->item->linkparts[·op  
$this->item->linkparts[·va  
else·if(·document·getEleme  
> ·> alert(·"<?php·echo·?Te  
}·<?php·}·?>  
<?php·if(·$this->item->typ
```

3er etapa: escribir algoritmos en un lenguaje de programación y elegir la representación de los datos.

Etapas de resolución de un problema por computadora

Del Problema a la Solución

Problema
Solución



Etapa de Análisis del Problema

- En una primera etapa, se analiza el problema en su contexto del mundo real. Deben obtenerse los **requerimientos** del usuario. El resultado del análisis del problema es un modelo preciso del ambiente del problema y del objetivo a resolver.

Etapa de Diseño de la solución

- Suponiendo que el problema es computable, a partir del modelo se debe diseñar una solución. En el paradigma procedural, esta etapa involucra entre otras tareas la **modularización del problema**, considerando la descomposición del mismo y los datos necesarios para cumplir su objetivo.

Esta etapa involucra la **especificación de los algoritmos**:

- Cada uno de los módulos del sistema diseñado tiene una función que podemos traducir en un algoritmo (que puede no ser único). La elección del algoritmo adecuado para la función del módulo es muy importante para la eficiencia posterior del sistema de software.

Etapa de Implementación de la solución

- Esta etapa involucra la **escritura de los programas**. Los algoritmos definidos en la etapa de diseño se convierten en programas escritos en un lenguaje de programación concreto.

Etapa de Verificación de la solución

- Una vez que se tienen los programas escritos y depurados de errores de sintaxis, se debe **verificar que su ejecución conduce al resultado deseado**, utilizando datos representativos del problema real.

En resumen:

Problema
Solución

Análisis y Diseño

(NO depende del lenguaje)

- Entender el problema
- Modelizarlo.
- Modularizar.
- Escribir los algoritmos.

Implementación

(depende del lenguaje)

- Codificación de los algoritmos en un lenguaje de programación.
- Prueba en ejecución de cada módulo.
- Prueba general del programa y verificación de que cumple los requisitos del problema.

*Problema del
Mundo Real*

Análisis

Diseño

Implementación

Verificación

Modelo

Algoritmos

Programa

Concepto de Algoritmo

Definiremos **algoritmo** como la especificación rigurosa de la secuencia de pasos (instrucciones) a realizar sobre un **autómata** para alcanzar un resultado deseado en un tiempo finito.

✓ *Especificación rigurosa* significa que debemos expresar un algoritmo en forma clara y unívoca.

✓ Alcanzar el resultado en *tiempo finito* significa que suponemos que un algoritmo *comienza y termina*. Por lo tanto, el número de instrucciones debe ser también finito.

✓ Si el autómata es una computadora, tendremos que escribir el algoritmo en un lenguaje "entendible" y ejecutable por la máquina.



PROGRAMA

Concepto de Programa

Definiremos **programa** como el conjunto de instrucciones u órdenes **ejecutables** sobre una **computadora**, que permite cumplir con una función específica (dichas órdenes están expresadas en un lenguaje de programación concreto).

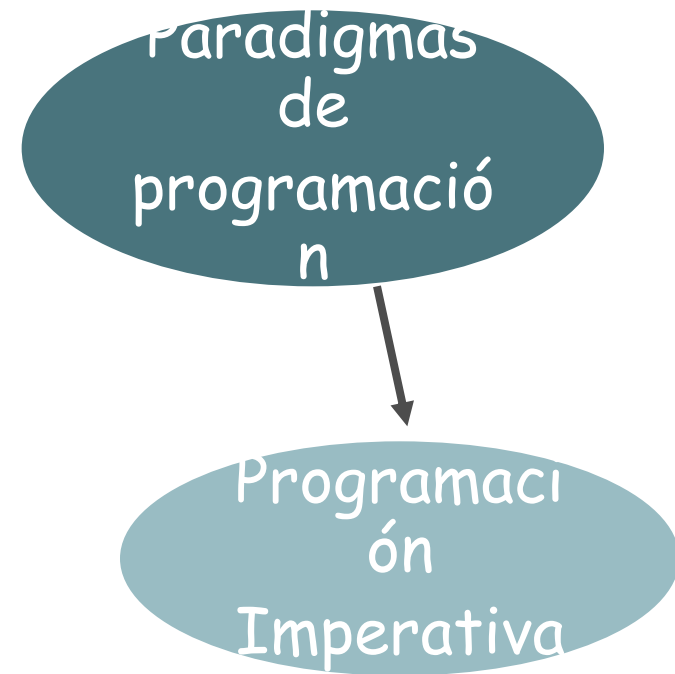
- ✓ Normalmente los programas alcanzan su función objetivo en un **tiempo finito**.
- ✓ Los **programas de aplicación** constituyen el verdadero valor que da utilidad a las computadoras (programas WEB, de administración, cálculo, comunicaciones, control industrial, sistemas expertos etc.).
- ✓ Los **programas** se escriben en un **lenguaje de programación** siguiendo un conjunto de reglas sintácticas y semánticas.

Programación Imperativa

El modelo que siguen los lenguajes de programación para **DEFINIR** y **OPERAR** la información, permite asociarlos a un paradigma de programación particular.

En esta primera parte del curso trabajaremos bajo el paradigma imperativo/procedural.

Lenguaje de programación: PASCAL



Escribir un programa exige:

- ✓ *Elegir la representación* adecuada de los datos del problema.
- ✓ *Elegir el lenguaje de programación* a utilizar, según el problema y la máquina a emplear.
- ✓ *Definir el conjunto de instrucciones* (en el lenguaje elegido) cuya ejecución ordenada conduce a la solución.

Programa = Instrucciones + Datos

Programa = Instrucciones + Datos

```
graph TD; A[Programa = Instrucciones + Datos] --> B[Las instrucciones (acciones) representan las operaciones que ejecutará la computadora al interpretar el programa.]; A --> C[Los datos son los valores de información de los que se necesita disponer y en ocasiones transformar para ejecutar la función del programa.]; B --> D[Se escriben en un lenguaje de programación determinado]; C --> E[Cada lenguaje de programación tiene los propios]; D --> F[PASCAL];
```

Las *instrucciones* (acciones) representan las operaciones que ejecutará la computadora al interpretar el programa.

Se escriben en un lenguaje de programación determinado

PASCAL

Los *datos* son los valores de información de los que se necesita disponer y en ocasiones transformar para ejecutar la función del programa.

Cada lenguaje de programación tiene los propios

Concepto de Dato

Un **Dato** es una representación de un objeto del mundo real. Los datos permiten modelizar los aspectos del problema que se quieren resolver mediante un programa ejecutable en una computadora.

Datos constantes

datos que no cambian durante la ejecución del programa.

Datos variables

datos que durante la ejecución del programa pueden cambiar.

En PASCAL cada dato debe tener asociado un tipo de dato

Definición de Tipo de Dato

Un *tipo de dato* es una clase de objetos de datos ligados a un conjunto de operaciones para crearlos y manipularlos.

Los tipos de datos se caracterizan por:

- ✓ Un rango de valores posibles
- ✓ Un conjunto de operaciones realizables sobre ese tipo
- ✓ Una representación interna

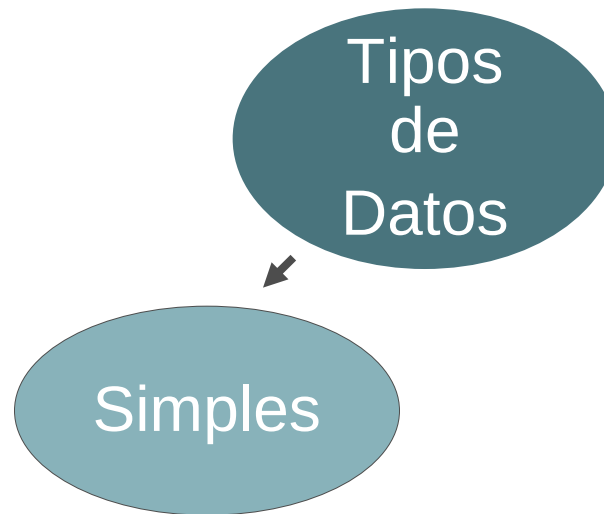
¿Qué valores puedo usar?

¿Qué operaciones puedo aplicar?

¿Cuánta memoria utiliza?

Clasificación de Tipos de Datos

Una primera clasificación

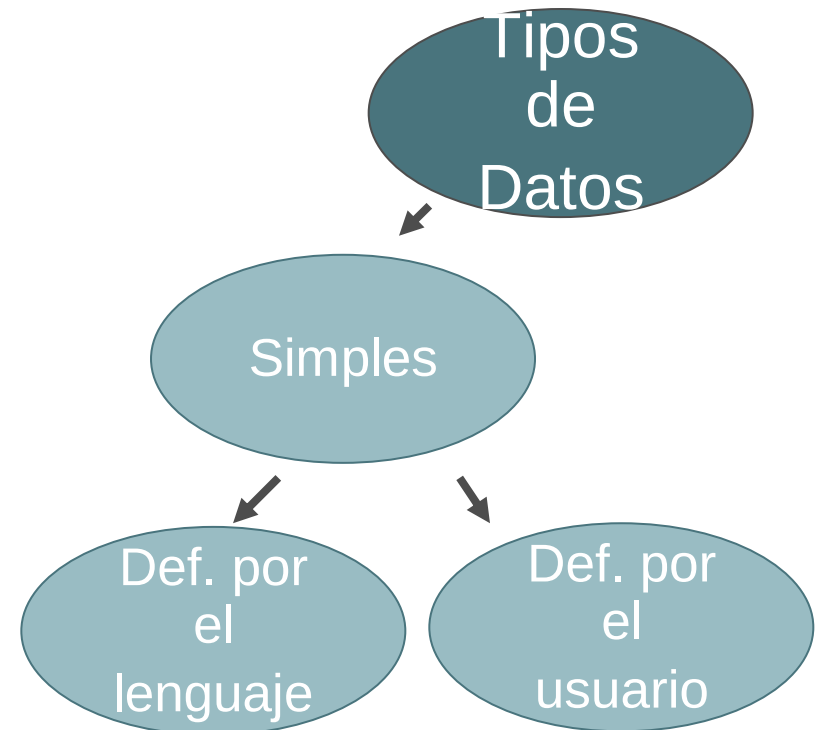


Los tipos de datos simples son aquellos que toman un único valor, en un momento determinado, de todos los permitidos para ese tipo.

Clasificación de Tipos de Datos Simples

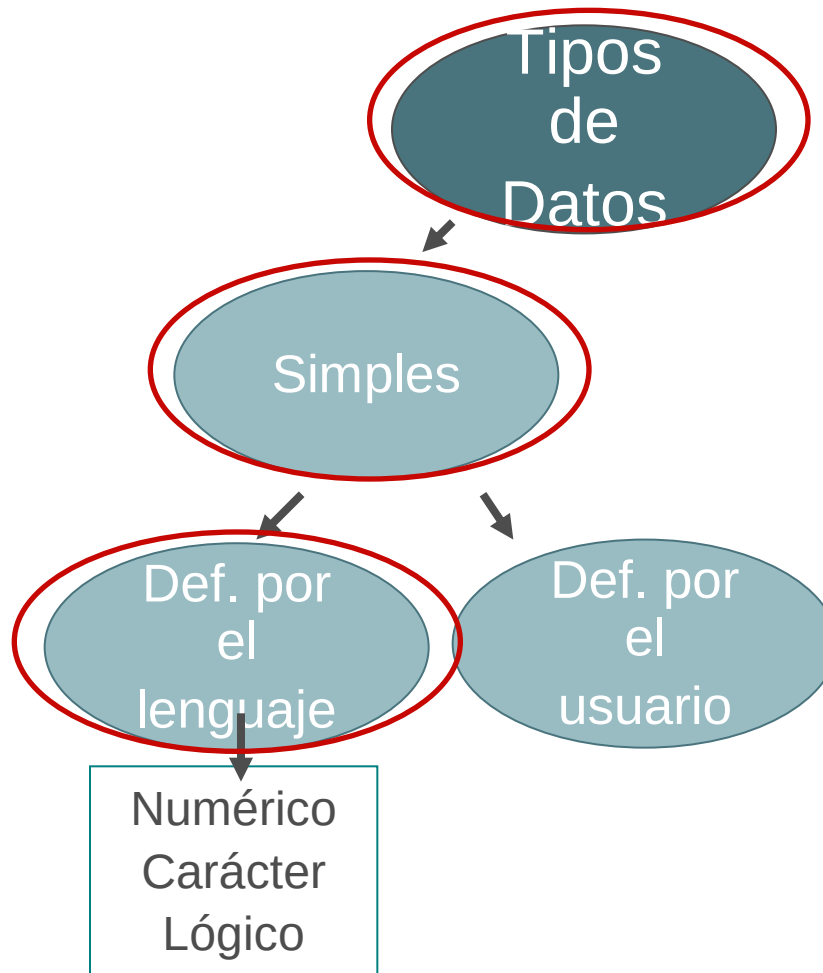
A su vez, los tipos simples, pueden clasificarse en:

- Los tipos de datos definidos por el lenguaje (primitivos o estándar) son provistos por el lenguaje y tanto la representación como sus operaciones y valores son reservadas al mismo.
- Los tipos definidos por el usuario, permiten definir nuevos tipos de datos a partir de los tipos simples.



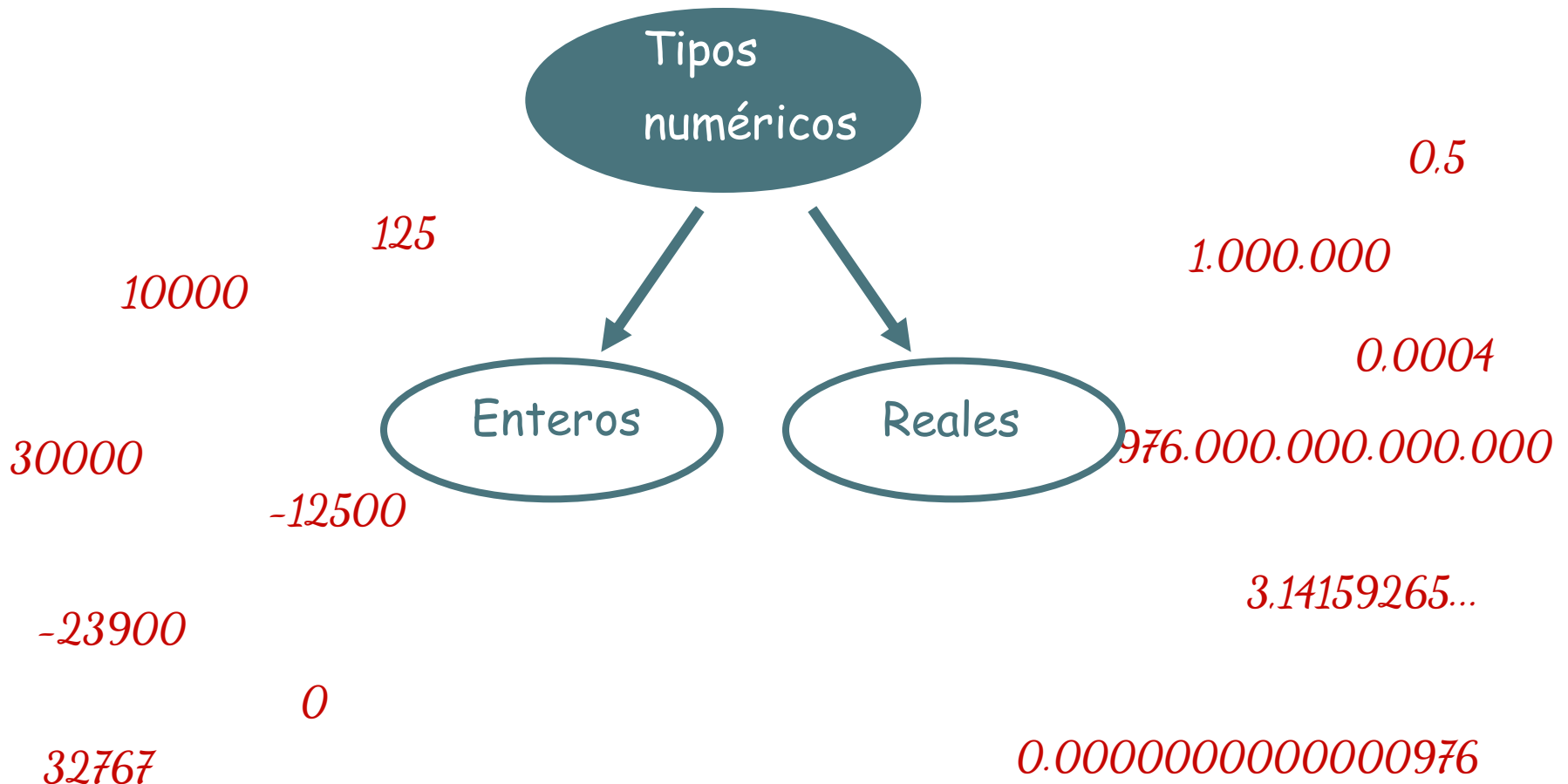
Clasificación de Tipos de Datos Simples Definidos por el lenguaje

Comenzaremos presentando los tipos simples y definidos por el lenguaje



Tipo de dato numérico

El **tipo de dato numérico** permite representar el conjunto de los valores numéricos que puede referirse a los números enteros o a los números reales:



Tipo de Dato numérico: ENTERO

El tipo de dato entero es un tipo de dato simple y ordinal.

Los elementos son del tipo: ..., -3, -2, -1, 0, 1, 2, 3, ...

Dado que una computadora tiene memoria finita, la cantidad de valores enteros que se pueden representar sobre ella son finitos, por esto se deduce que existe un número entero máximo y otro mínimo.

Hay sistemas de representación numérica que utilizan 16 dígitos binarios (bits) para almacenar en memoria cada número entero, permitiendo un rango de valores enteros entre -2^{15} y $+2^{15}$. Entonces podrán representarse los enteros entre -32768 y +32767

Trabajaremos con la idea que la ocupación en memoria es de 2 bytes

Diagram illustrating the range of integers that can be represented in memory (2 bytes):

Values shown: 32767, ~~-50000~~, 0, -23900, -12500, ~~100000~~, 30000.

The values ~~-50000~~ and ~~100000~~ are crossed out, indicating they are outside the representable range.

Tipo de Dato numérico: REAL

El tipo de dato real es una clase de dato numérico que permite representar números decimales. Es un tipo de dato simple.

- ✓ El tipo de dato real tiene una representación finita de los números reales.
- ✓ Se utiliza la notación exponencial o científica, utilizada por cualquier tipo de calculadora, y consiste en definir cada número como una *mantisa* (parte decimal) y un *exponente* (posición de la coma).

La ocupación en memoria es de 6 bytes

$976.000.000.000.000 \rightarrow 9,76 \times 10^{14}$

$0.00000000000000976 \rightarrow 9,76 \times 10^{-14}$

Operaciones del tipo de dato numérico

■ Operaciones aritméticas

Las operaciones válidas para el tipo de dato numérico son:

- ✓ suma (+),
- ✓ resta (-),
- ✓ multiplicación (*),
- ✓ división (/),
- ✓ división entera (div)
- ✓ módulo (mod).

Operador	Tipo de Operando	Tipo de resultado
Suma (+)	Entero ó Real	Entero ó Real
Resta (-)	Entero ó Real	Entero ó Real
Multiplicación (*)	Entero ó Real	Entero ó Real
División (/)	Real	Real
Div	Entero	Entero
Mod	Entero	Entero

3+4

2.5/3

10 div 4

10 mod 4

Orden de precedencia de las operaciones con números

Las expresiones que tienen dos o más operandos requieren reglas matemáticas que permitan determinar el orden de las operaciones.

El orden de precedencia para la resolución, ya conocido, es:

1ro: Operadores $*$, $/$

2do: Operadores $+$, $-$

3ero: Operadores **div y mod.**

Se pueden utilizar paréntesis para alterar el orden de precedencia

$$\cdot 6 + 8 * 5 = 6 + 40 = 46$$

$$\cdot (6 + 8) * 5 = 14 * 5 = 70$$

$$\cdot 5 * 2 + 7 + 4 * 3 = 10 + 7 + 12 = 17 + 12 = 29$$

$$\cdot 5 * (2 + 7 + 4) * 3 = 5 * 13 * 3 = 195$$

Operaciones del tipo de dato numérico

■ Operación de asignación ($:=$)

■ Operaciones de comparación

Además de los operadores matemáticos mencionados, el tipo de dato numérico posee operadores relacionales que permiten comparar valores.

✓ igualdad ($=$),

✓ desigualdad ($\neq$)

✓ orden ($<$, $\leq$, $>$, $\geq$)

Su **resultado** es Verdadero o Falso

▪ $0 < 7$	Resultado: verdadero
▪ $5 \leq 7$	Resultado: verdadero
▪ $5 > 7$	Resultado: falso
▪ $5 \neq 7$	Resultado: verdadero
▪ $5 \geq 2$	Resultado: verdadero

¿Cómo se identifican los datos del programa?

IDENTIFICADORES

Para identificar los datos de un programa (constante / variable) se utilizan nombres descriptivos. Esa identificación permite conocer su dirección real en la memoria y el valor que contiene.

En Pascal, los identificadores están formados por letras, dígitos en cualquier orden y algunos símbolos especiales (excepto el primer carácter que **debe ser una letra**).

flores1

papeles

Total_cuadras

pasos

cantidad

Es obligatorio declarar cada uno de los datos que utiliza el programa

Declaración de constantes y variables en Pascal

En Pascal, las constantes deben ser declaradas antes de ser usadas.

Siguiendo su notación, la declaración de constantes en Pascal se realiza mediante la palabra clave **const**, de la forma:

```
const nombre = valor
```

donde nombre es el identificador que representa el nombre de la constante. El tipo de dato de la constante queda definido implícitamente por el valor que se le asigna.

```
const
```

```
  N = 25      {N se asume entero }
```

```
  pi = 3.1416 {pi se asume real }
```

Declaración de variables

Las variables en Pascal se declaran utilizando la palabra clave **var**, de la forma:

```
var variable : tipo
```

Si hay varias variables del mismo tipo, se pueden declarar separadas por coma y se le asocia el tipo de dato, una sola vez.

Por ejemplo:

```
var  
    variable1: tipo A;  
    variable2, variable3: tipoB;
```

Declaración de variables numéricas en Pascal

En Pascal, para representar los datos numéricos se cuenta con:

- **integer** → dato numérico entero
- **real** → dato numérico real

Var

```
cantidad: integer; {cantidad puede contener un nro. entero}  
total_cobrado: real; {total_cobrado puede contener número real }
```

Recordar que cuando se asigna el tipo de dato a una variable queda determinado: Valores posibles , Operaciones permitidas y Representación en memoria

Estructura general de un Programa en Pascal

Program nombre;

Const

.....

Var

....

begin {Cuerpo del programa}

.....

.... {Instrucciones ejecutables}

End.

Zona de
Declaración de
constantes

Zona de
Declaración de
variables

Zona de
Instrucciones
ejecutables

Ejemplo de un Programa en Pascal

Program nombre;

Const

num = 5;

Zona de
Declaración de
constantes

Var

cantidad: integer;

totalCobrado: real;

Zona de
Declaración de
variables

Begin { *Cuerpo del programa*}

cantidad := 4;

totalCobrado := 10,50;

End.

Zona de
Instrucciones
ejecutables

Ejercicio 1



Escribir un programa que calcule la superficie de un terreno que tiene 15,5 mts. de frente y 60 mts. de largo e informe el resultado.

Algoritmo:

- Calcular superficie
- Mostrar resultado

¿Cómo se calcula la superficie?

¿Cómo se muestra el resultado?

Operación de salida en Pascal: WRITE

■ Write

se usa para mostrar el contenido de una variable, por defecto en pantalla. Pueden ser de tipo entero, real, char. Los datos a mostrar si son más de uno deben ir separados por coma.

```
Write (a, b);
```

```
Write (a);
```

```
Write ('El resultado es: ', b);
```

■ También puede utilizarse la instrucción Writeln

Es similar a la instrucción Write, pero hace que la siguiente sentencia Write muestre el contenido de la variable en la siguiente línea.

Resolución del ejercicio 1



Escribir un programa que calcule la superficie de un terreno que tiene 15,5 mts. de frente y 60 mts. de largo e informe el resultado.

Algoritmo:

- Calcular superficie
- Mostrar resultado

Program superficie;

var sup: real;

Begin

sup := 15,5 * 60;

write ('La superficie es: ', sup);

End.

Ejercicio 2



Escribir un programa que calcule la superficie de un terreno cuyas dimensiones se leen de teclado e informe el resultado.

Algoritmo:

- Leer dimensiones
- Calcular superficie
- Mostrar resultado

*¿Como se leen las
dimensiones?*

Operación de entrada en Pascal: READ

■ **Read**

se usa para leer datos (por defecto desde teclado) y asignarlos a las variables correspondientes.

```
Read (a, b, c);
```

```
Read (a);
```

```
Read (b);
```

```
Read (c);
```

■ **También puede utilizarse la instrucción Readln**

Es similar a la instrucción Read pero hace que la siguiente sentencia Read comience leyendo en la siguiente línea.

Resolución del ejercicio 2



Escribir un programa que calcule la superficie de un terreno cuyas dimensiones (frente y fondo) se leen de teclado e informe el resultado.

Algoritmo:

- Leer dimensiones
- Calcular superficie
- Mostrar resultado

Program superficie;

var frente, fondo: real;

 sup: real;

Begin

readln (frente);

readln (fondo);

 sup := frente * fondo;

write ('La superficie es: ', sup);

End.

Ejercicio 3



Si se quiere alambrar ese terreno ¿Cuántos metros lineales de alambrado se necesitan?

Modificar el programa anterior, para que también resuelva e informe la cantidad de metros necesarios para alambrarlo

Tipo de dato lógico

El **tipo de dato lógico** permite representar datos que pueden tomar solamente uno de dos valores. Este tipo de dato se conoce también como tipo de dato boolean. Es un tipo de dato simple y ordinal.

Valores posibles

- **verdadero** (*true*)
- **falso** (*false*)

Se utiliza en situaciones donde se representan dos alternativas de una condición.

¿Juan es mayor que María?

¿Edad de Juan?

¿Edad de María?

Una variable de tipo lógico ocupa 1 byte de memoria

Operaciones con el dato lógico

Las operaciones permitidas son:

- **asignación** ($:=$)
- **negación** (*not*)
- **conjunción** (*and*)
- **disyunción** (*or*)

Existe un orden de precedencia para los operadores lógicos:

1. operador *or*
2. operador *and*
3. operador *not*

El resultado de estas operaciones es el correspondiente a las conocidas tablas de verdad.

Ejemplos

- $(18 < 14)$ *falso*
- $(25 / 2.5 = 10)$ *verdadero*
- $(28 > 13)$ *verdadero*
- *not* $(8 > 3)$ *falso*
- $(17 > 2)$ *and* $(19 = 33)$ *falso*
- $(17 > 4)$ *or* $(19 = 33)$ *verdadero*

Tipo de dato lógico en Pascal

Program nombre;

Const

max = 10;

Var

exito: boolean;

resultado: boolean;

Begin { *Cuerpo del programa*}

exito := 8 < max;

resultado := false;

.....

End.

Tipo de Dato Carácter

El **tipo de dato carácter** es un tipo de dato simple y ordinal.

Un dato de tipo carácter contiene solo un carácter.

Los caracteres que reconocen las computadoras no son estándar. Sin embargo, este conjunto de valores se normalizó, entre otros, por un estándar llamado ASCII, el cual permite establecer un **orden de precedencia** entre los mismos.

Valores permitidos

- ✓ **Caracteres especiales:** '!', '#', '\$', '%', ...
- ✓ **Dígitos:** '0', '1', '2', ..., '8', '9'
- ✓ **Letras mayúsculas:** 'A', 'B', 'C', ..., 'Y', 'Z'
- ✓ **Letras minúsculas:** 'a', 'b', 'c', ..., 'y', 'z'

Una variable de tipo carácter ocupa 1 byte de memoria

Tipo de Dato Caracter

TABLA DE CARACTERES DEL CÓDIGO ASCII

1	␣	25	↓	49	1	73	I	97	a	121	y	145	æ	169	—	193	±	217	⌈	241	±
2	⦿	26		50	2	74	J	98	b	122	z	146	Æ	170	—	194	⌋	218	⌋	242	±
3	♥	27		51	3	75	K	99	c	123	{	147	Ô	171	—	195	⌋	219	⌋	243	±
4	♦	28	—	52	4	76	L	100	d	124		148	Ö	172	—	196	⌋	220	⌋	244	±
5	♣	29	↔	53	5	77	M	101	e	125	}	149	ò	173	—	197	⌋	221	⌋	245	±
6	♠	30	▲	54	6	78	N	102	f	126	~	150	û	174	«	198	⌋	222	⌋	246	±
7		31	▼	55	7	79	O	103	g	127	⌘	151	ù	175	»	199	⌋	223	⌋	247	±
8		32		56	8	80	P	104	h	128	Ç	152	ÿ	176		200	⌋	224	α	248	±
9		33	!	57	9	81	Q	105	i	129	ü	153	Ö	177		201	⌋	225	β	249	±
10		34	"	58	:	82	R	106	j	130	é	154	Ü	178		202	⌋	226	Γ	250	±
11		35	#	59	;	83	S	107	k	131	â	155	Ç	179		203	⌋	227	π	251	±
12		36	\$	60	<	84	T	108	l	132	ä	156	£	180		204	⌋	228	Σ	252	±
13		37	%	61	=	85	U	109	m	133	à	157	¥	181		205	⌋	229	σ	253	±
14		38	&	62	>	86	V	110	n	134	â	158	℞	182		206	⌋	230	μ	254	±
15		39	'	63	?	87	W	111	o	135	ç	159	f	183		207	⌋	231	τ	255	±
16	▶	40	(	64	@	88	X	112	p	136	ê	160	á	184		208	⌋	232	Φ	PRESIONA LA TECLA	
17		41	)	65	A	89	Y	113	q	137	ë	161	í	185		209	⌋	233	Θ	Alt	
18	‡	42	*	66	B	90	Z	114	r	138	è	162	ó	186		210	⌋	234	Ω	MÁS EL NUMERO	
19	‡‡	43	+	67	C	91	[	115	s	139	ï	163	ú	187		211	⌋	235	δ	CORTESÍA DE:	
20	¶	44	,	68	D	92	\	116	t	140	î	164	ñ	188		212	⌋	236	∞		
21	§	45	-	69	E	93	]	117	u	141	ì	165	Ñ	189		213	⌋	237	φ		
22	—	46	.	70	F	94	^	118	v	142	Ë	166	°	190		214	⌋	238	€		
23	‡	47	/	71	G	95	_	119	w	143	Ä	167	º	191		215	⌋	239	∩		
24	†	48	0	72	H	96	`	120	x	144	É	168	¿	192		216	⌋	240	≡		

Operaciones con caracteres

■ Asignación (:=)

■ Comparación (>, <, =...)

dos valores de tipo carácter se pueden comparar por =, <>, >, <, >=, <=. El resultado es un valor lógico (verdadero/falso) que depende del orden establecido en el código ASCII

Ejemplos:

- ('b' = 'B') *falso*
- ('c' < 'Z') *falso*
- ('c' < 'z') *verdadero*
- ('X' > '5') *verdadero*
- (' ' < 'H') *verdadero*
- ('4' = 4) *no puede evaluarse*
- ('@' > '\$') *verdadero (ya que la ubicación de \$ es 36 y la del @ es 64)*

Tipo de dato carácter en Pascal

```
Program nombre;
```

```
Const
```

```
    fin = '*';
```

```
Var
```

```
    resultado: boolean;
```

```
    letra1, letra2 : char;
```

```
Begin  { Cuerpo del programa}
```

```
    letra1:= 'A';
```

```
    letra2:= 'm';
```

```
    resultado := letra 1 < letra2;
```

```
    .....
```

```
End.
```

Tipos de lenguajes

- ✓ Algunos lenguajes, como Pascal, exigen que se especifique a qué tipo pertenece cada una de las variables.
- ✓ Verifican que el tipo de los datos asignados a esa variable se correspondan con su definición.
- ✓ Esta clase de lenguajes se denomina ***fuertemente tipados*** (strongly typed).

Tipos de lenguajes

- ✓ Otra clase de lenguajes, determina el tipo acorde al primer valor que se le asigne, se denomina *auto tipados* (**self typed**).
- ✓ Existe una tercera clase de lenguajes que permiten que una variable tome valores de distinto tipo durante la ejecución de un programa. Esta se denomina *dinámicamente tipados* (**dynamically typed**).

Programación I 2022

Concepto de Estructura de control

Todos los lenguajes de programación tienen un conjunto mínimo de instrucciones que permiten especificar el **control** del algoritmo que se quiere implementar.

Dicho conjunto **debe contener al menos:**

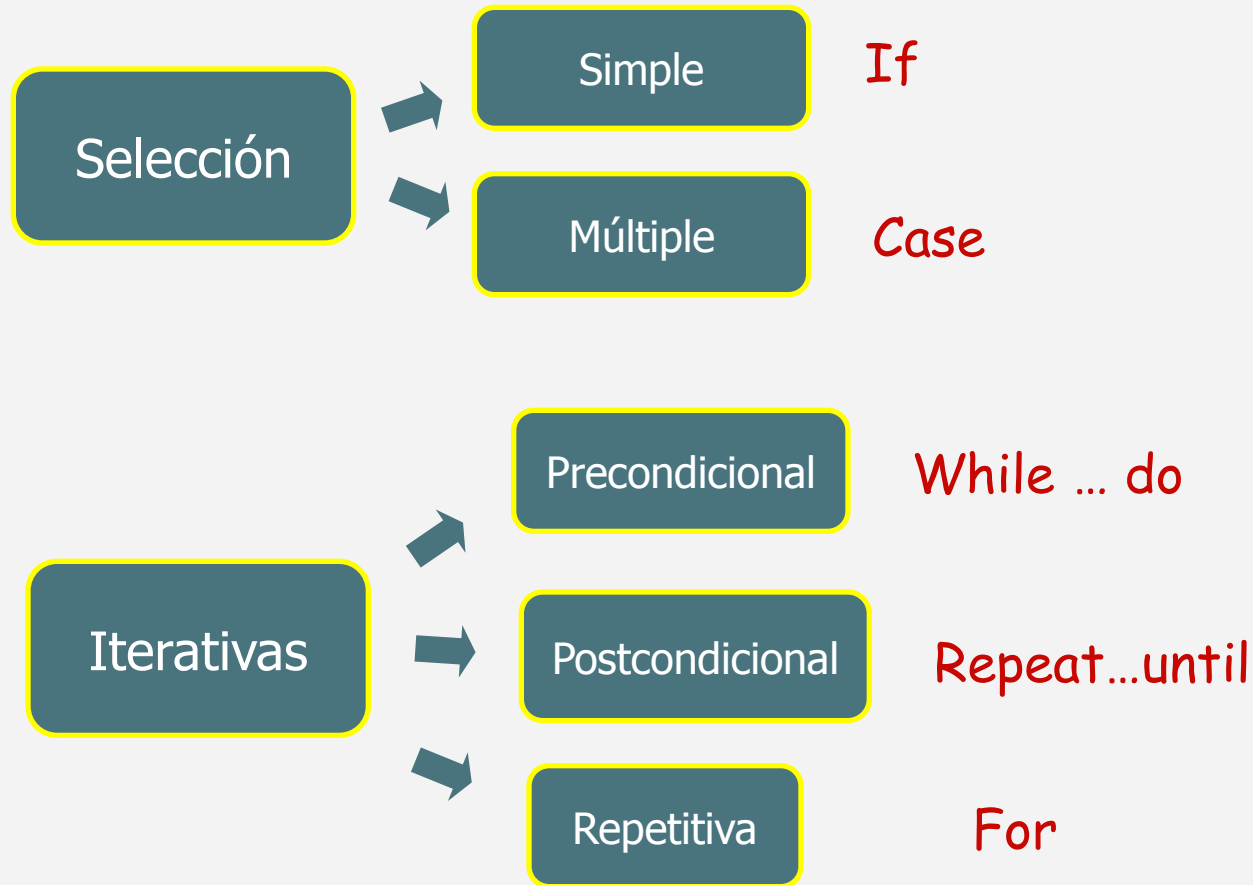
✓ **Selección**

✓ **Iteración**

¿Para qué nos sirven las estructuras de control?

Las estructuras de control permiten modificar el flujo de ejecución de las instrucciones de un programa.

Clasificación de las Estructuras de control

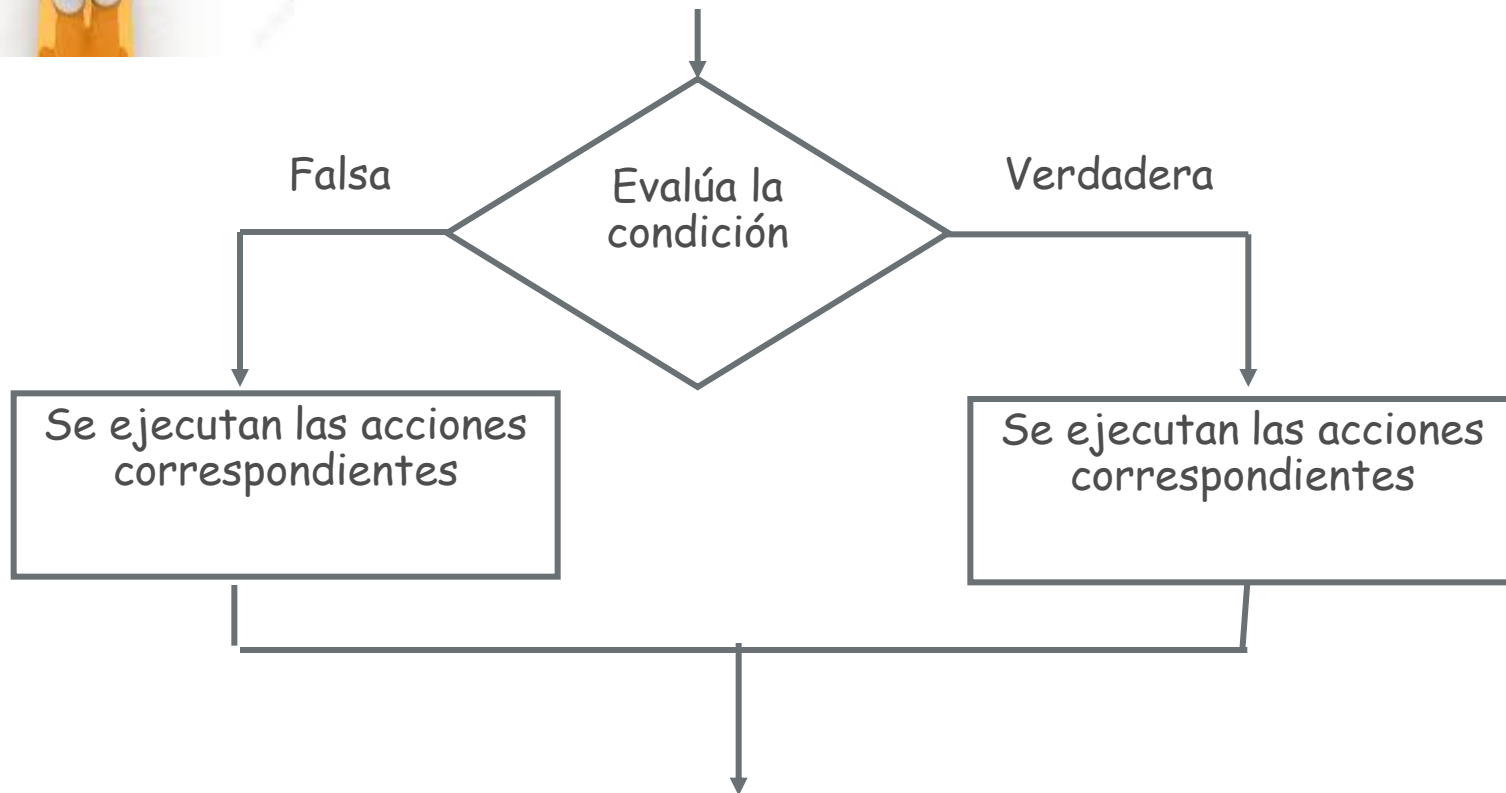


Estructura de control: Selección simple (en Pascal)



Puede ocurrir que en un problema real sea necesario elegir una alternativa entre 2 posibles.

La estructura de selección simple se representa simbólicamente:



Estructura de control: Selección simple (en Pascal)

```
If  (condición)  then begin
    Acciones_por_Condición_Verdadera;
end
    else begin
    Acciones_por_Condición_Falsa;
end;
```

Caso Especial (sin else)

```
If  (condición)  then begin
    Acciones_por_Condición_Verdadera;
end;
```

Estructura de control: Selección simple (en Pascal)



Ejercicio 1: Se quiere comprar un terreno solo si su superficie es mayor a 900 mts².

Implementar un programa que calcule la superficie del terreno cuyas dimensiones se leen e informe si lo compra o no.

Algoritmo:

Leer dimensiones

Calcular superficie

Si (superficie > 900) entonces Mostrar Comprar

sino Mostrar No comprar

```
program ejer1Clase2;
var frente, fondo, superficie: real;
begin
  write (' Ingrese frente: ');
  readln(frente);
  write (' Ingrese fondo: ');
  readln(fondo);
  superficie:= frente * fondo;
  if (superficie > 900) then write('Comprar')
    else write('No comprar');
End.
```

Estructura de control: Selección simple (en Pascal)



Ejercicio 2: Un centro de cobro de impuestos necesita un programa que permita que se ingrese un tipo de impuesto y en función del tipo de impuesto se muestre a qué caja debe dirigirse para su cobro. Los impuestos de tipo "X" se cobran en la caja 1 y los tipo "Z" en la caja 2. Para otros impuestos no hay cajas disponibles.

Algoritmo:

Leer impuesto

Si es X -> Mostrar ir a Caja 1

sino Si es Z -> Mostrar ir a Caja 2

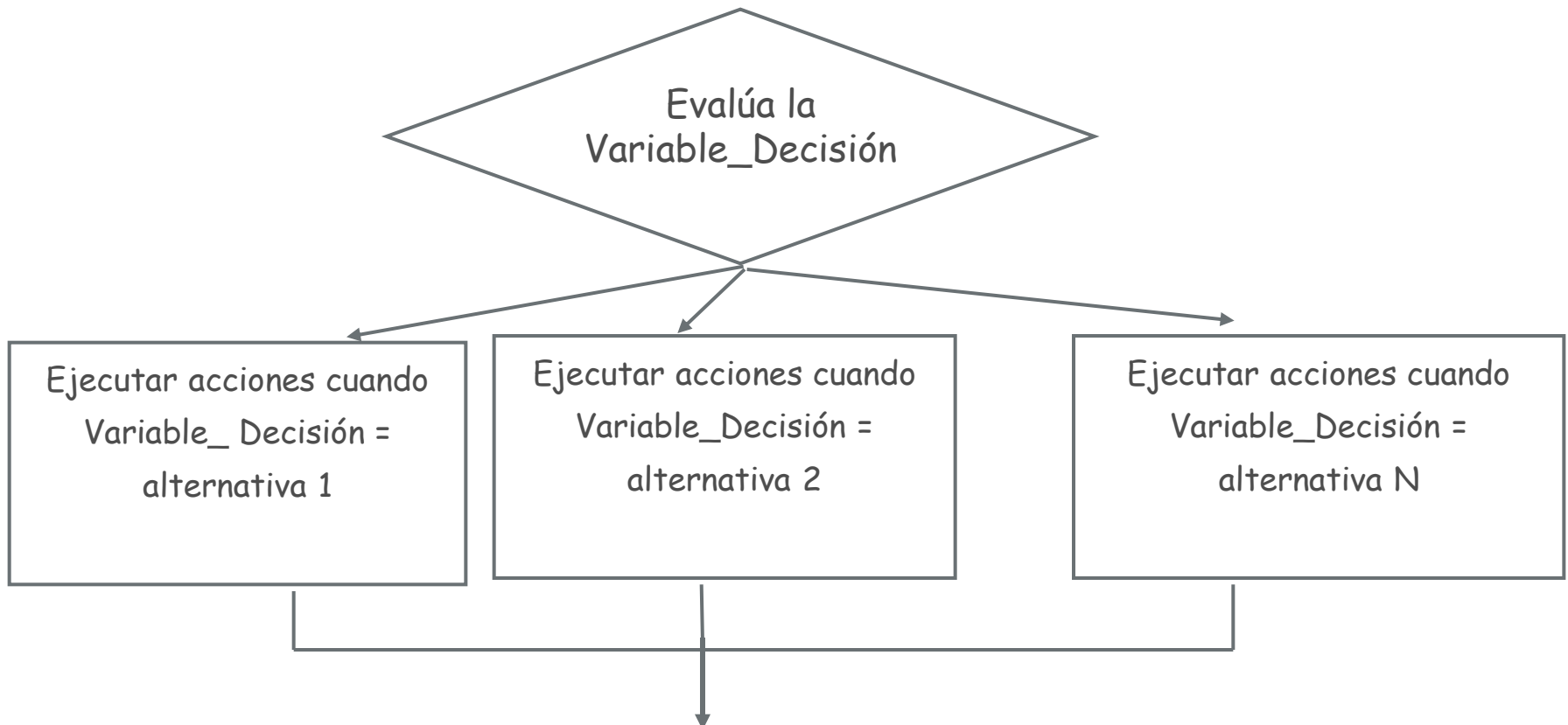
sino Mostrar No se cobra

Estructura de control: Selección múltiple



Puede ocurrir que en un problema real sea necesario elegir una alternativa entre varias posibles en función del problema a resolver.

La estructura de selección múltiple se representa simbólicamente:



Estructura de control: Selección múltiple (en Pascal)

Puede ocurrir que en un problema real sea necesario elegir una alternativa entre varias posibles en función del problema a resolver.

```
Case variable decision of
  alternativa 1 : Acciones;
  alternativa 2 : Acciones;
  .....
else otras acciones
End;
```

Estructura de control: Selección múltiple (en Pascal)



Ejercicio 3: Un centro de cobro de impuestos necesita un programa que permita que se ingrese un tipo de impuesto y en función del tipo de impuesto se muestre a qué caja debe dirigirse para su cobro. Se cobran impuestos de los tipos "X", "Z", "P" y "S", en las cajas 1 a 4 respectivamente. Para otros impuestos no hay cajas disponibles.

Algoritmo:

Leer impuesto

Verificar tipo

Si es X -> Caja 1

sino si es Z -> Caja 2

sino si es P ...

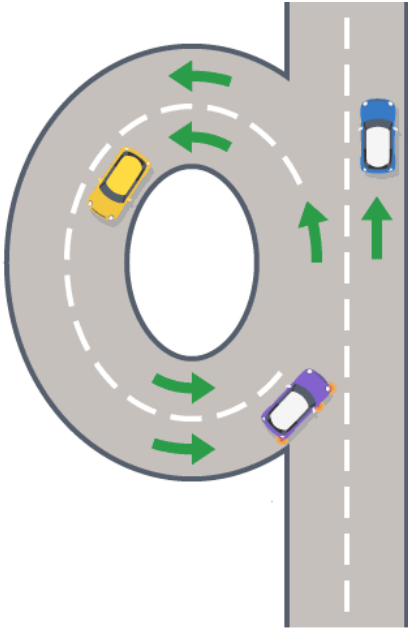
```
program ejer3Clase2;
var tipoImpuesto: char;

BEGIN
  write('Ingrese su tipo de impuesto: ');
  read(tipoImpuesto);
  case tipoImpuesto of
    'Z', 'z' : write ('Se cobra en caja 2');
    'X', 'x' : write ('Se cobra en caja 1');
    'P', 'p' : write ('Se cobra en caja 3');
    'S', 's' : write ('Se cobra en caja 4');
  else writeln('No hay cajas disponibles para ese tipo de impuesto');
  end;
END.
```

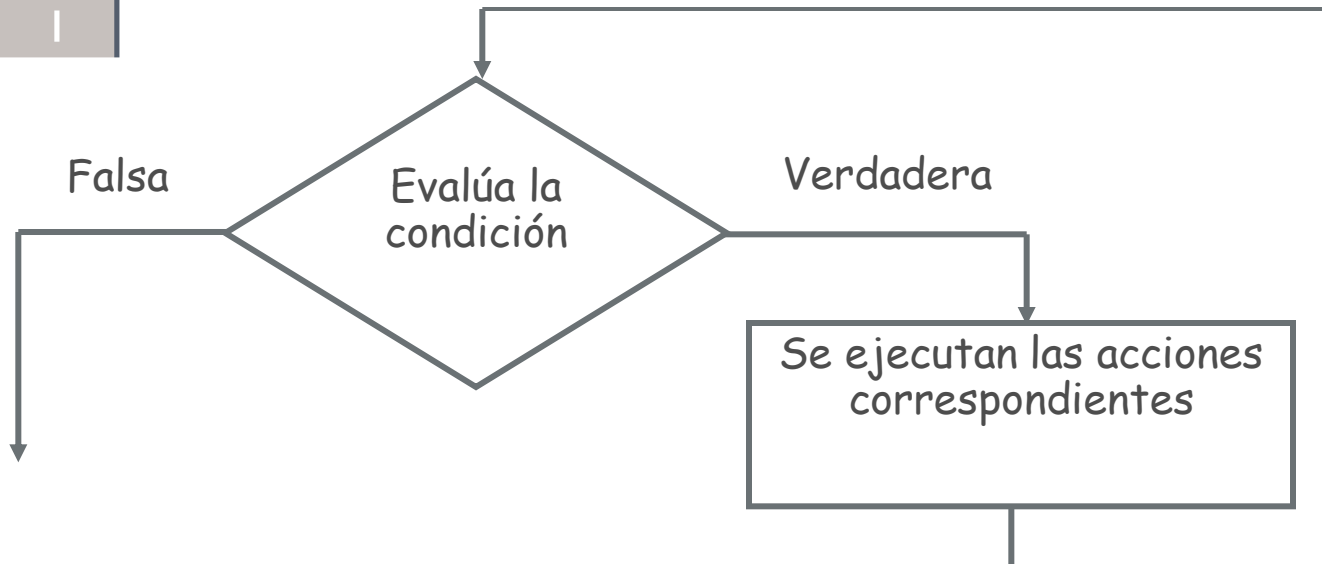
Estructuras de Control Iterativas

- Puede ocurrir que se desee ejecutar un bloque de instrucciones desconociendo el número exacto de veces que se ejecutan.
- Para estos casos existen en la mayoría de los lenguajes de programación estructurada las **estructuras de control iterativas condicionales**.
- Como su nombre lo indica las acciones se ejecutan dependiendo de la evaluación de la condición.
- Estas estructuras se clasifican en **pre-condicionales** y **post-condicionales**.

Estructura de Control: Iterativa precondicional



- Las **estructuras de control iterativas precondicionales** primero evalúan la condición y si es verdadera se ejecuta el bloque de acciones. Dicho bloque se puede ejecutar 0, 1 ó más veces.
- Importante: el valor inicial de la condición debe ser conocido o evaluable antes de la evaluación de la condición.



Estructura de Control: Iterativa precondicional (en Pascal)

■ Las **estructuras de control iterativas precondicionales** primero evalúan la condición y si es verdadera se ejecuta el bloque de acciones. Dicho bloque se puede ejecutar 0, 1 ó más veces.

■ **Importante**: el valor inicial de la condición debe ser conocido o evaluable antes de la evaluación de la condición.

```
While (condición) do  
  begin  
    Acciones a realizar  
  end;
```

Estructura de Control: Iterativa precondicional (en Pascal)



Ejercicio 4: Un centro de cobro de impuestos necesita un programa que permita que se ingresen tipos de impuesto y en función del tipo de impuesto se muestre a qué caja debe dirigirse para su cobro. El ingreso de impuestos finaliza cuando se ingresa el impuesto '@'. Se cobran impuestos de los tipos "X", "Z", "P" y "S", en las cajas 1 a 4 respectivamente. Para otros impuestos no hay cajas disponibles.

Algoritmo:

Leer impuesto

Mientras el impuesto sea distinto de '@'

Verificar tipo

Si es I -> Caja 1

sino si es M -> Caja 2

sino si es P ...

Leer impuesto

Fin mientras

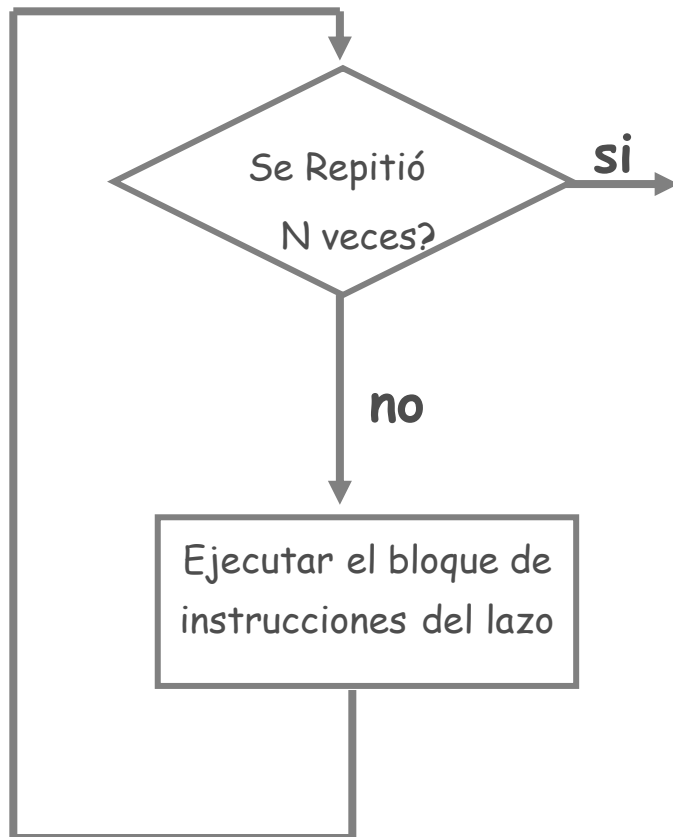
Estructura de control: Iterativa repetitiva



- Puede ocurrir que se desee ejecutar un bloque de instrucciones conociendo el número exacto de veces que se ejecutan. Es decir ***repetir N veces un bloque de acciones.***

- Este número de veces que se deben ejecutar las acciones es fijo y conocido de antemano.

Se muestra el diagrama esquemático de la ***repetición***



Estructura de control: Iterativa repetitiva

```
For indice := valor_inicial to valor_final do  
    begin  
        Acciones a realizar  
    End;
```


Estructura de control: Iterativa repetitiva



Ejercicio 5: Escribir un programa que informe la superficie total de una parcela de terrenos cuyas dimensiones se leen de teclado. La parcela se compone de 15 terrenos.

Algoritmo:

Inicializar total de sup.

Repetir 15 veces

Leer frente y fondo

Calcular superficie

Sumar al total

Mostrar Resultado

Estructura de control: Iterativa repetitiva



Ejercicio 6: Reescribir el programa anterior para que además informe la superficie máxima calculada.

Algoritmo:

Inicializar total de sup.

Inicializar valor máximo

Repetir 15 veces

Leer frente y fondo

Calcular superficie

Sumar al total

Si superficie > valor máximo

actualizar valor máximo

Mostrar Resultados

Estructuras de Control



Ejercicio 7: Escribir un programa que informe la suma de 20 números enteros que se leen de teclado y además que informe la cantidad de números pares entre los números leídos.

Algoritmo:

Inicializar la suma

Inicializar contador de pares

Repetir 20 veces

Leer un número

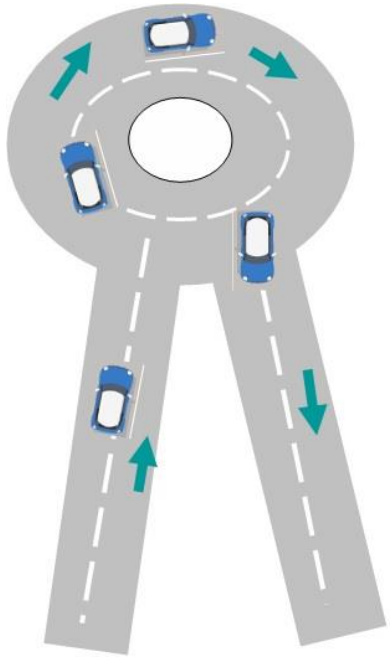
Sumar el número leído

Verificar si es par

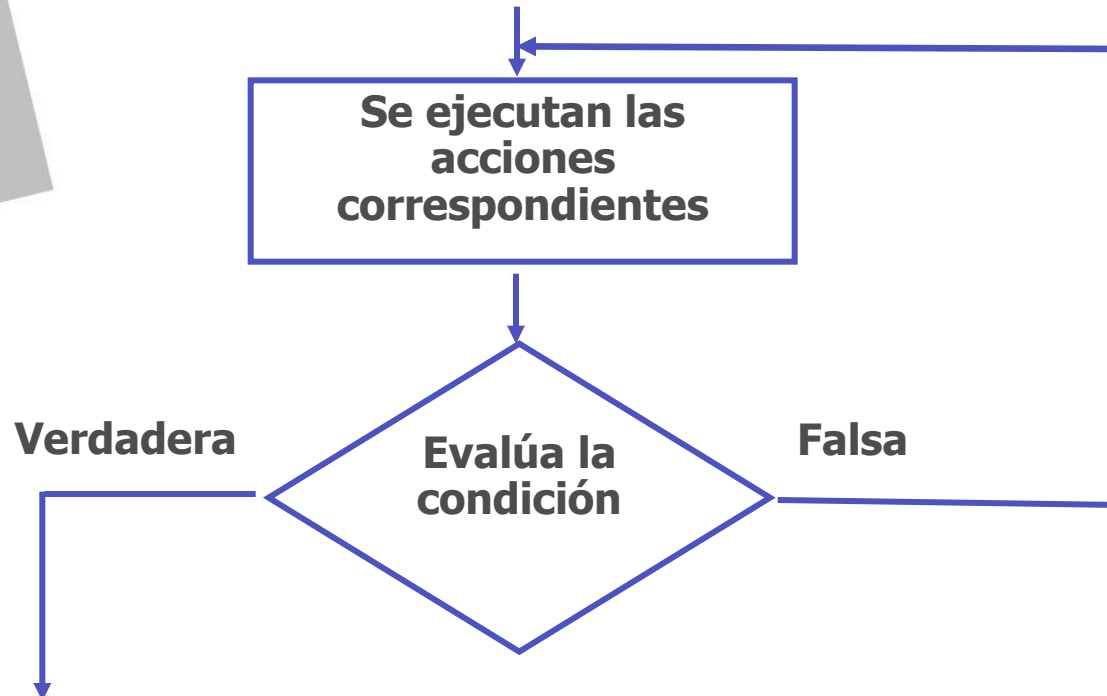
incrementar contador

Mostrar Resultados

Estructura de Control: Iterativa postcondicional



- Las **estructuras de control iterativas postcondicionales** primero ejecutan el bloque de acciones y luego evalúan la condición. A diferencia de la estructura iterativa precondicional, el bloque de acciones se ejecuta 1 ó más veces.
- Importante: el valor inicial de la condición debe ser conocido o evaluable antes de la evaluación de la condición.



Estructura de Control: Iterativa postcondicional

- Las **estructuras de control iterativas postcondicionales** primero ejecutan el bloque de acciones y luego evalúan la condición. A diferencia de la estructura iterativa pre-condicional, el bloque de acciones se ejecuta 1 ó más veces.

repeat

Acciones a realizar

until (condicion);

Estructura de Control: Iterativa postcondicional (en Pascal)



Ejercicio 8: Escribir un programa que informe las superficies de terrenos cuyas dimensiones se leen de teclado. El ingreso de datos finaliza cuando se ingresa un terreno cuya superficie supera los 1000 mt².

Algoritmo

Repetir

Leer frente

Leer fondo

Calcular Superficie

Mostrar Superficie

Hasta que Superficie > 1000

Fin

Programación I

- 1 Concepto de Modularización
- 2 Procedimientos y Funciones
- 3 Alcance de una variable

1 Concepto de Modularización

Etapas de resolución de un problema por computadora

Problema
Solución

*Problema del
Mundo Real*

Análisis

Complejos
Extensos
Modificables

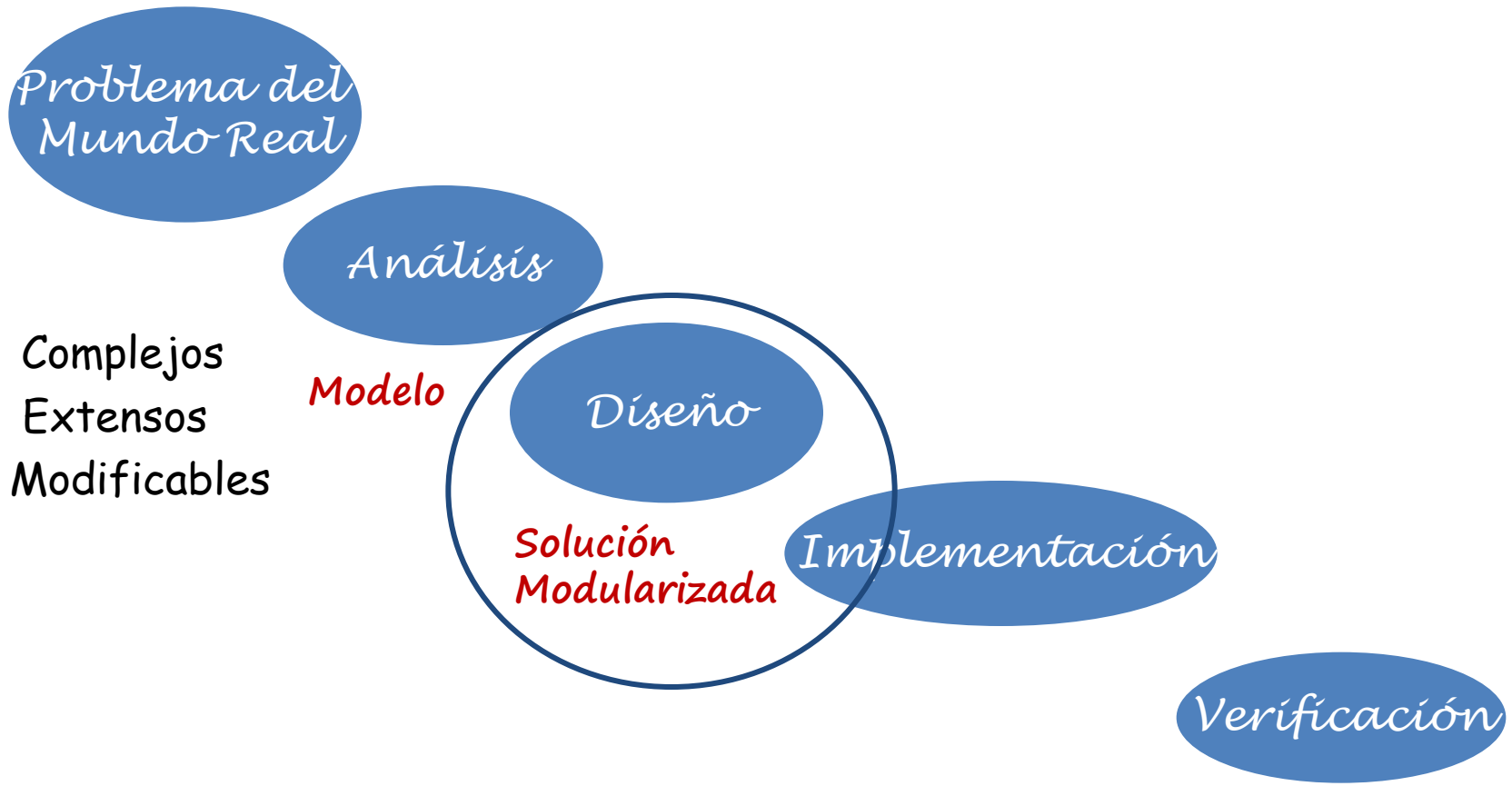
Modelo

Diseño

*Solución
Modularizada*

Implementación

Verificación



Modularización

Los problemas del mundo real implican:

- Complejidad
- Extensión
- Modificaciones/Situaciones de cambio

Los tratamos de resolver con:

- Abstracción.
- Descomposición funcional.

Modularización

Modularizar significa dividir un problema en partes funcionalmente independientes, que encapsulen operaciones y datos.



No se trata simplemente de subdividir el código de un sistema de software en bloques con un número de instrucciones dado.

Separar en funciones lógicas con datos propios y datos de comunicación perfectamente especificados.

Modularización - Abstracción

La descomposición tiene siempre un objetivo.

Se busca obtener:

Alta Cohesión: medida del grado de identificación de un módulo con una función concreta.

Bajo Acoplamiento: medida de la interacción de los módulos que constituyen un programa.



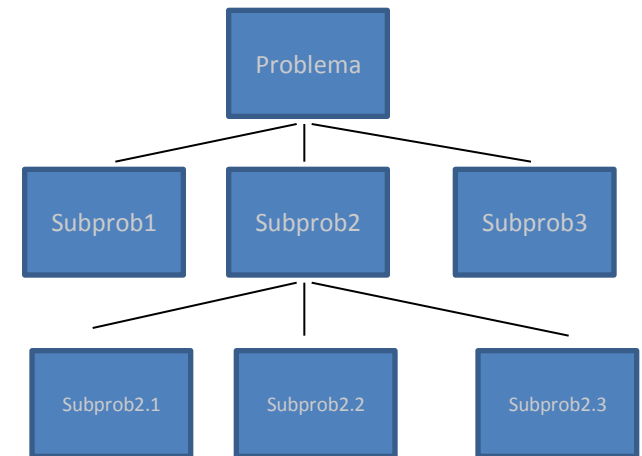
Modularización - Abstracción

Principio de "Divide y vencerás"

Descomponer el problema en partes (subproblemas) mas simples

Al descomponer un problema se debe tener en cuenta:

- Que cada subproblema resuelva una parte "bien" simple.
- Que cada subproblema pueda resolverse independientemente
- Que las soluciones a los subproblemas deben combinarse para resolver el problema original



Modularización - Descomposición

¿Qué son los Módulos?

Es un conjunto de instrucciones que cumplen una tarea específica bien definida, se comunican entre sí adecuadamente y cooperan para conseguir un objetivo común.



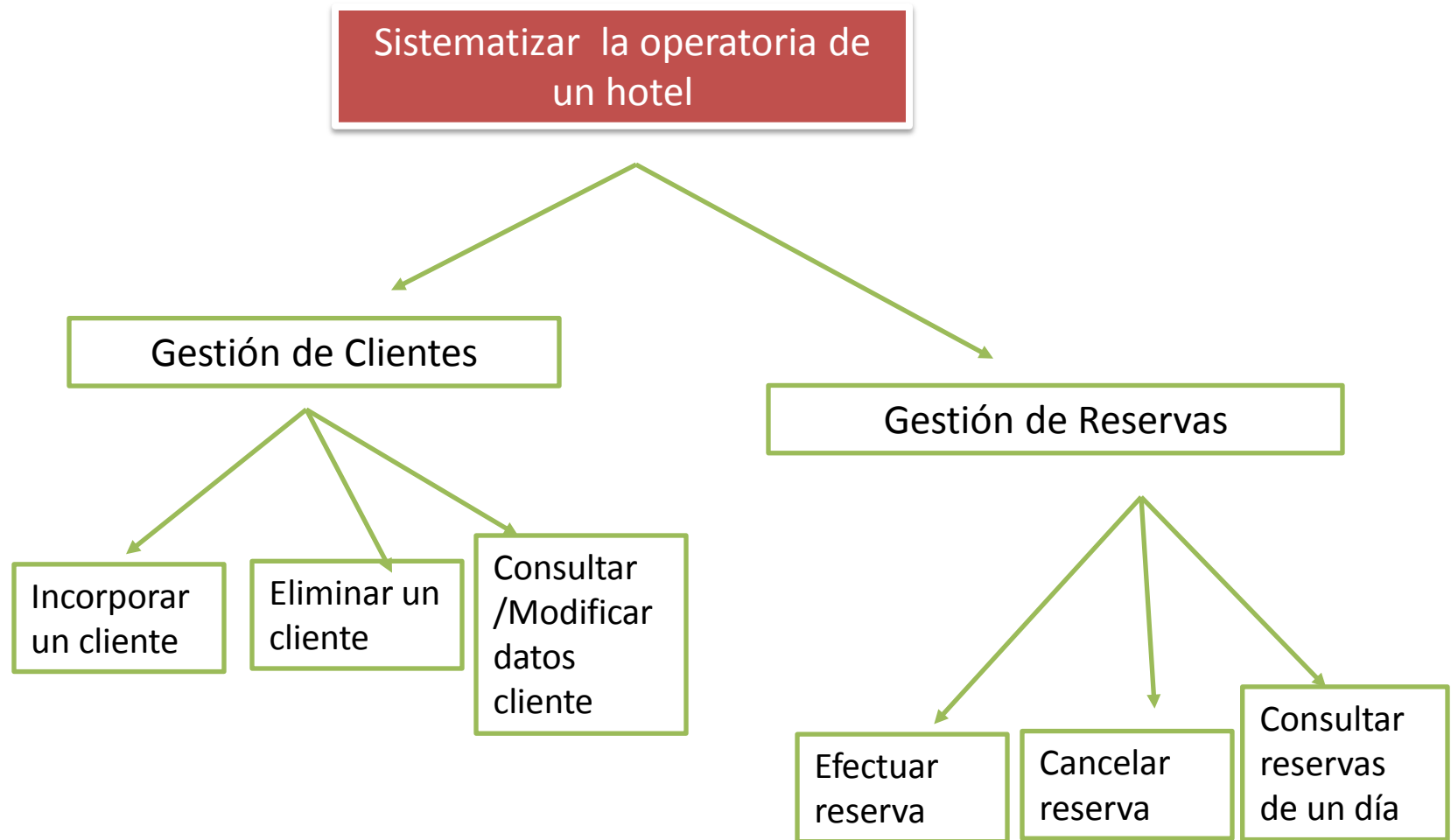
Cada módulo encapsula, acciones tareas o funciones



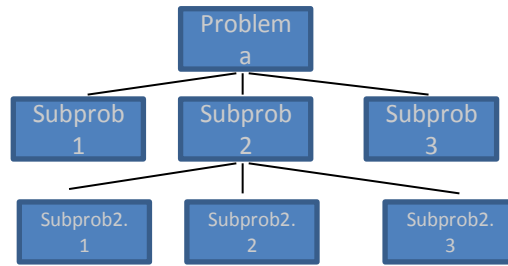
Hay que representar los objetos relevantes del problema a resolver.



Modularización - Ejemplo



VENTAJAS DE LA MODULARIZACIÓN



Permite distribuir el trabajo

Favorece el mantenimiento correctivo

Ventajas de la descomposición del problema

Facilita la reutilización dentro del mismo problema o en otro similar

Facilita el crecimiento de los sistemas

Aumenta la legibilidad

Modularización - Ventajas

Mayor productividad

Al dividir un sistema de software en módulos funcionalmente independientes, un equipo de desarrollo puede trabajar simultáneamente en varios módulos, incrementando la productividad (es decir reduciendo el tiempo de desarrollo global del sistema). Ejemplo.

Modularización - Ventajas

Sistematizar la operatoria de un hotel

Gestión de Clientes

Incorporar un cliente

Eliminar un cliente

Consultar /Modificar datos cliente

Gestión de Reservas

Efectuar reserva

Cancelar reserva

Consultar reservas de un día



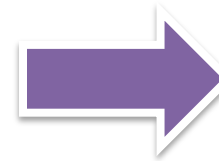
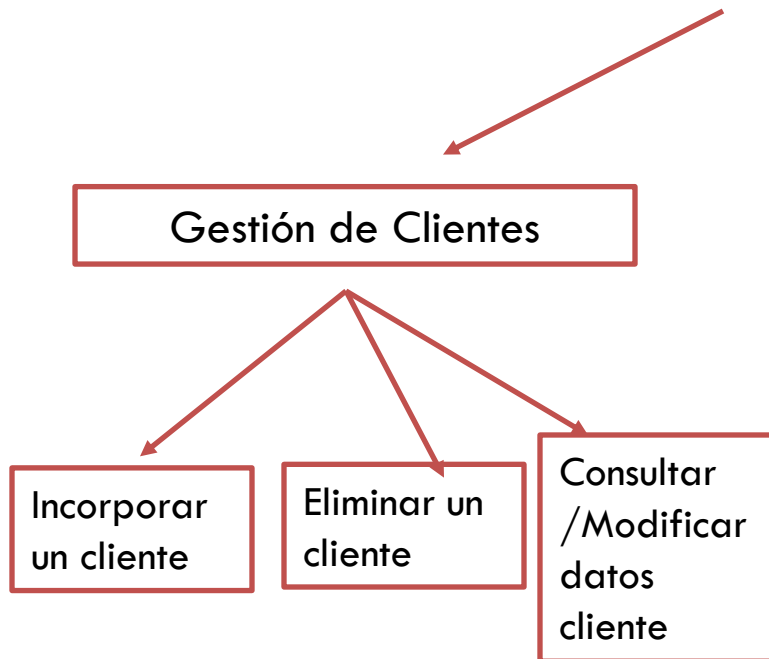
Modularización - Ventajas

Reusabilidad

Un objetivo fundamental de la Ingeniería de Software es la *reusabilidad*, es decir la posibilidad de utilizar repetidamente el producto de software desarrollado.

Naturalmente la **descomposición funcional** que ofrece la **modularización favorece el reuso**. Ejemplo.

Modularización - Ventajas



Se puede utilizar
para cualquier otro
sistema



Modularización - Ventajas

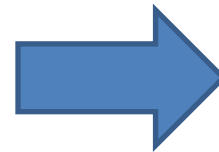
Facilidad de Mantenimiento Correctivo

La división lógica de un sistema en módulos permite **aislar los errores** que se producen con mayor facilidad. Esto significa poder **corregir los errores** en menor tiempo y disminuye los costos de mantenimiento de los sistemas.
Ejemplo

Modularización – Ejemplo



No puedo eliminar
un usuario



**Eliminar un
usuario**

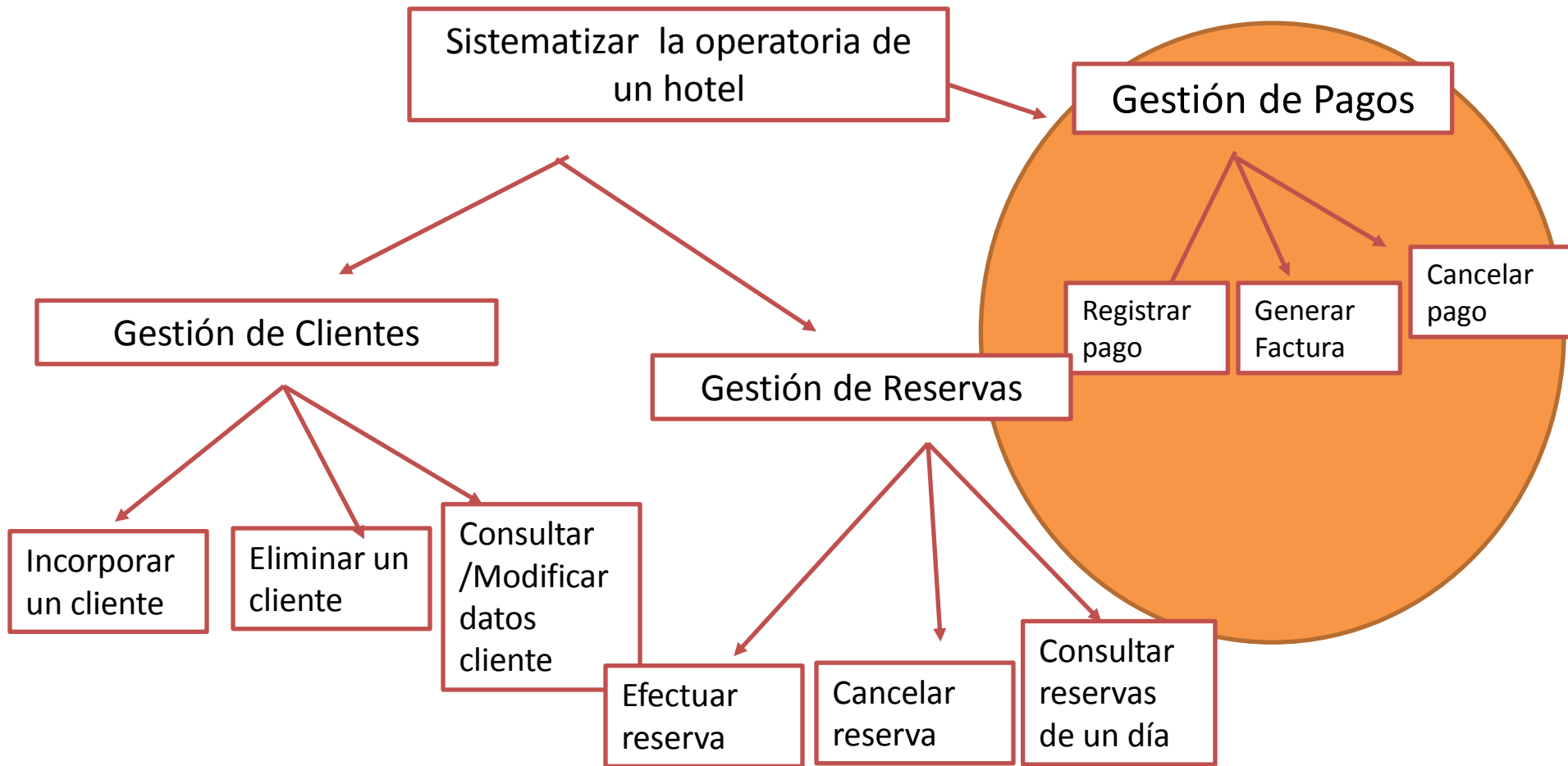


Modularización - Ventajas

Facilidad para el crecimiento del sistema

Los sistemas de software reales crecen (es decir aparecen con el tiempo nuevos requerimientos del usuario). La modularización permite **disminuir los riesgos y costos de incorporar nuevas prestaciones** a un sistema en funcionamiento. Ejemplo.

Modularización - Ventajas



Modularización - Ventajas

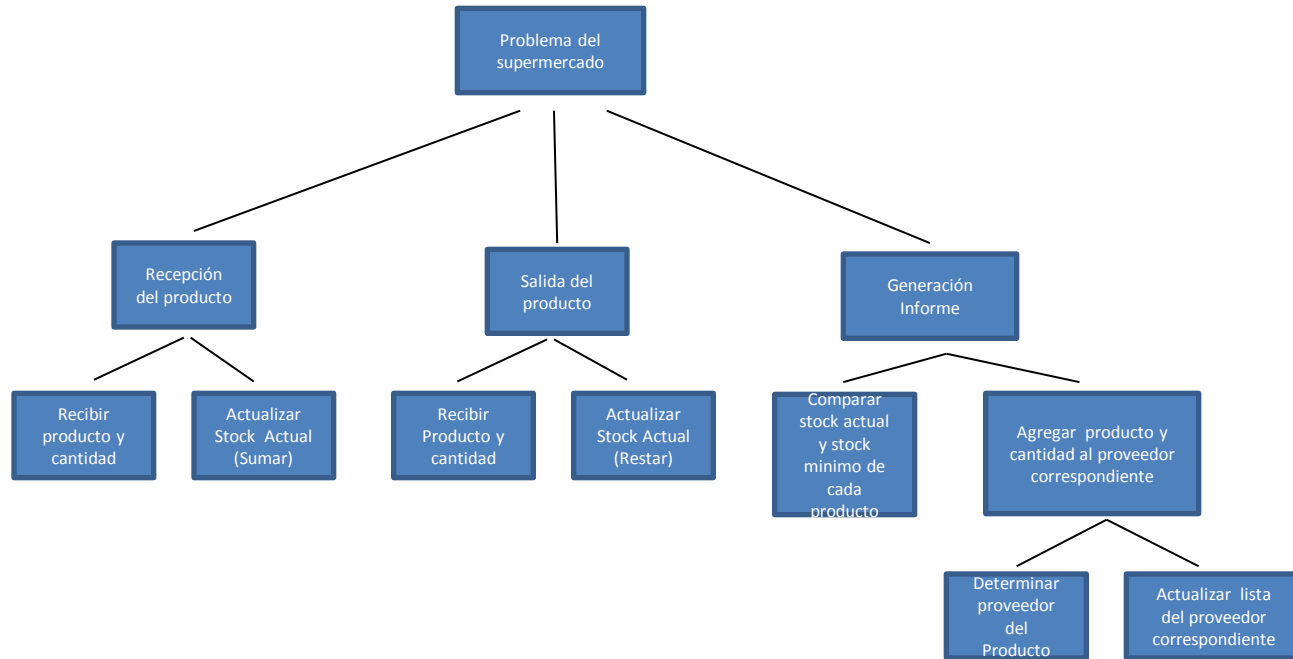
Mayor Legibilidad

Un efecto de la modularización es una **mayor claridad para leer y comprender el código fuente.**

El ser humano maneja y comprende con mayor facilidad un número limitado de instrucciones directamente relacionadas.
Ejemplo.

Retomando el concepto de Modularización

La tarea de **Modularizar** implica dividir un problema en partes. Se busca que cada parte realice una tarea simple y pueda resolverse de manera independiente a las otras tareas.



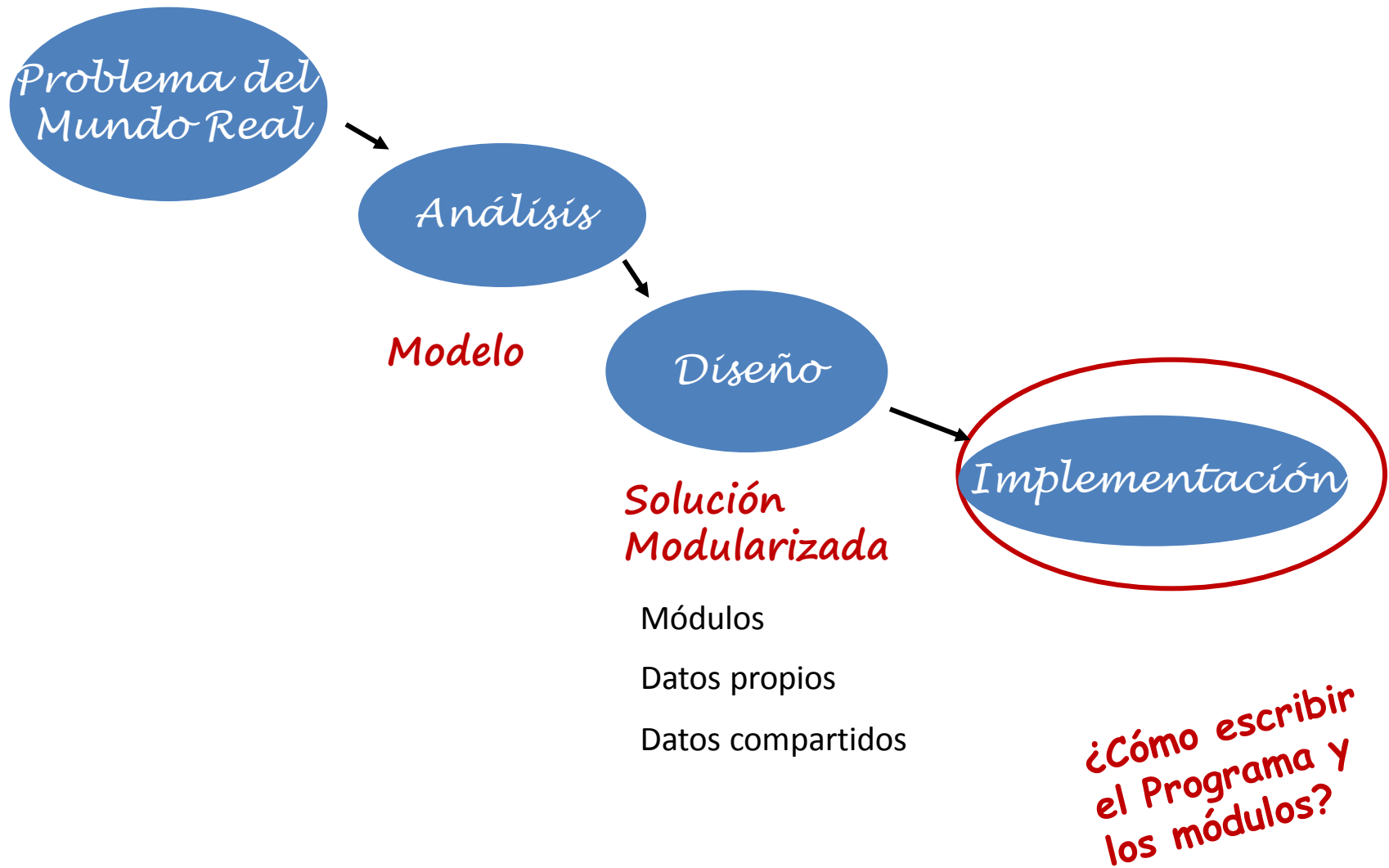
Cada uno de los módulos

Resuelve un subproblema particular

Define el conjunto de acciones

Define los datos necesarios

Avanzamos a la etapa de Implementación



2 Procedimientos y Funciones

- Se debe elegir el lenguaje de programación para escribir los algoritmos de cada módulo y la declaración de sus datos
- Los lenguajes de programación ofrecen diversas opciones para implementar la modularización.

Definición del módulo

¿Qué hace el módulo cuando se ejecuta?



- Encabezamiento (Interface)
 - Tipo de módulo
 - Identificación
 - Datos de comunicación
- Declaración de tipos
- Declaración de variables
- Sección de instrucciones ejecutables

Invocación del módulo

¿Qué se hace cuando se quiere usar el módulo?



- Se debe conocer de qué manera se invoca al módulo para que ejecute sus acciones

La invocación
puede hacerse
mas de una
vez

¿Qué ocurre
con el flujo de
control del
programa?

¿Qué tipos de módulos ofrece Pascal?

PROCEDIMIENTOS
(PROCEDURE)

FUNCIONES
(FUNCTION)

Tienen características comunes, pero ciertas particularidades determinan cual es el mas adecuado para implementar un módulo particular

- ¿El módulo devuelve datos?
 - ¿Cuántos datos devuelve?
 - ¿De qué tipo son los datos que devuelve?
 - ¿Qué tipo de acciones ejecuta el módulo?

¿PROCEDURE?

¿FUNCTION?

¿Cuáles son los aspectos que los diferencian?

- Encabezamiento del módulo
- Invocación
- Lugar donde retorna el flujo de control una vez ejecutado el módulo

PROCEDURE

Conjunto de instrucciones que realiza una tarea específica y como resultado puede retornar 0, 1 o más valores.

¿Cómo se define el módulo?

```
Procedure nombre (lista de parámetros formales);
```

```
Type
```

```
.....
```

```
Var
```

```
.....
```

```
begin
```

```
.....
```

```
.....
```

```
end;
```

Encabezamiento

Declaración de tipos internos
del módulo (opcional)

Declaración de variables
internas del módulo (opcional)

Sección de instrucciones

PROCEDURE

Conjunto de instrucciones que realiza una tarea específica y como resultado puede retornar 0, 1 o más valores.

¿Cómo se invoca el módulo?

Program uno;

.....

procedure Calculo (Parámetros Formales);

Type

... *

Var

.....

Begin

.....

... *

End;

Begin

.....

Calculo (parámetros actuales);

.....

End.

Conjunto de instrucciones que realiza una tarea específica y como resultado puede retornar 0, 1 o más valores.

Luego de ejecutado el módulo ¿Qué instrucción se ejecuta?

¿Qué ocurre
con el flujo de
control del
programa?

Luego de ejecutado el módulo, el flujo de control retorna a la instrucción siguiente a la invocación del módulo

Program uno;

.....

procedure Calculo (Parámetros Formales);

Type

... *

Var

.....

Begin

.....

... *

End;

Begin

.....

Calculo (parámetros actuales);

.....

End.



PROCEDURE

Puede ocurrir que un módulo Procedure contenga además, de la declaración de tipos y variables propias, la definición de otros módulos. Se dice que el procedimiento contiene módulos anidados. Por ejemplo:

```
Procedure principal (lista de parametros formales);
```

```
Type
```

```
.....
```

```
Var
```

```
.....
```

```
Procedure uno (lista de parametros formales);
```

```
  begin
```

```
  end;
```

```
Procedure dos (lista de parametros formales);
```

```
  begin
```

```
  end;
```

```
Var .....
```

```
Begin {instrucciones ejecutables del procedure principal}
```

```
  ...
```

```
    uno (parametros actuales);
```

```
    dos (parametros actuales);
```

```
    .....
```

```
end;
```

FUNCTION

Conjunto de instrucciones que realiza una tarea específica y como resultado retorna un único valor de tipo simple.

¿Cómo se define el módulo?

```
Function nombre (lista de parámetros formales): tipo;
```

Encabezamiento

```
Type   .....
```

Declaración de tipos
(opcional)

```
Var    .....
```

Declaración de variables
(opcional)

```
begin
```

```
.....
```

```
nombre := ...;
```

```
end;
```

— **Asignación
(obligatoria)**

Sección de instrucciones

FUNCTION

Conjunto de instrucciones que realiza una tarea específica y como resultado retorna un único valor de tipo simple.

¿Cómo se invoca el módulo?

```
Program otro;
```

```
Function cubo(Parámetros Formales):integer;  
Type  
    ...  
Var  
    .....  
Begin  
    .....  
    ...  
End;
```

```
Begin  
    ...  
    ...  
    write (cubo (parametro actual));  
    ...  
End.
```

```
Program otro;
```

```
Function cubo(Parámetros Formales):integer;  
Type  
    ...  
Var  
    .....  
Begin  
    .....  
    ...  
End;
```

```
Var a: integer;  
begin  
    ...  
    ...  
    a := cubo (parametro actual);  
    ...  
End.
```

¿Qué ocurre con
el flujo de control
del programa?

Luego de ejecutado el módulo, el flujo de control retorna a la misma instrucción de invocación del módulo

FUNCTION

Conjunto de instrucciones que realiza una tarea específica y como resultado retorna un único valor de tipo simple.

¿Cómo se invoca el módulo?

```
Program otro;
```

```
Function cubo(Parámetros Formales):integer;  
Type  
    ... *  
Var  
    .....  
Begin  
    .....  
    ... *  
End;
```

```
begin  
    ... *  
    ... *  
    if (cubo (parametro actual) > 100) then ....;  
End.
```

```
Program otro;
```

```
Function cubo(Parámetros Formales):integer;  
Type  
    ... *  
Var  
    .....  
Begin  
    .....  
    ... *  
End;
```

```
Var a: integer;
```

```
begin....
```

```
    ... *  
    ... *  
    while (cubo (parametro actual) > 50) do
```

```
        ..... *  
End.
```

También puede ocurrir que un módulo function contenga además, de la declaración de tipos y variables propias, la definición de otros módulos procedimientos y/o funciones.

Esquema general de un programa que utiliza módulos

```
program uno;
```

```
Const
```

```
{Declaración de constantes del programa}
```

```
Type
```

```
{Declaración de tipos definidos}
```

```
Var
```

```
{variables}
```

```
Procedure Calculo (parámetros formales);
```

```
.....
```

```
Function EsPar (parámetros formales): Boolean;
```

```
...
```

```
Var
```

```
{variables}
```

```
Begin {Sección del programa principal}
```

```
.....
```

```
End.
```

Zona de Declaración de constantes

Zona de Declaración de tipos

Zona de Declaración de variables accedidas desde la sección del programa y los módulos

Zona de Declaración de los módulos

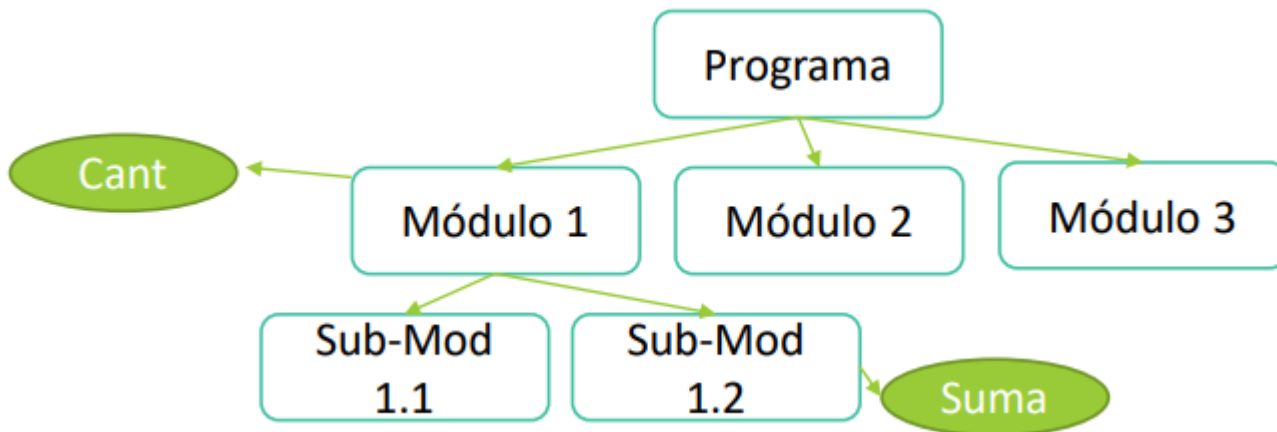
Zona de Declaración de variables accedidas solo desde la sección del programa

Zona de Instrucciones ejecutables

3 Alcance de variables

Alcance de variables

Alcance de una variable: Es el contexto donde una variable pueden ser referenciada o nombrada y ésta es reconocida.



¿Cuál es el alcance de la variable **Cant** y cuál es el de la variable **Suma**?

Analicemos el alcance de Variables...

```
Program Uno;  
Var  
  y, j: integer;
```

```
  procedure prueba;  
  var  
    x: integer;  
  Begin  
    x:= 9;  
    write (x);  
  End;
```

```
  Begin  
    j:= 80;  
    y:= j * 2;  
    prueba;  
  End.
```

Variables
globales

Variables
locales

▪ ¿Dónde se pueden utilizar j e y?

▪ ¿Dónde se puede utilizar x?

▪ ¿Qué pasa si dentro de prueba se declara y: integer?

▪ ¿Qué pasa si dentro de prueba se declara y: char?

Variables globales y

- ➔ **Variable global:** su declaración se hace en la sección de declaración del programa principal, es decir fuera de todos los módulos del programa y podrá ser usada en el programa y en todos los módulos del mismo.
- ➔ **Variable local a un módulo:** su declaración se hace en un módulo particular y sólo podrá ser usada por ese módulo. Si este módulo contiene a su vez otros módulos, entonces esa variable puede ser también usada por todos los módulos interiores.
- ➔ **Variable local al programa:** su declaración se hace antes de la sección de instrucciones ejecutables del programa y después de la declaración de los módulos del programa. Su uso se limita a la sección de instrucciones ejecutables.

¿Cuáles son variables locales y cuáles globales?

```
Program dos;  
Var  
y, j: integer;
```

```
procedure prueba;  
var x: integer;  
  procedure otro;  
  var z: integer;  
  begin  
    z:= 50;  
    z := z div 10;  
    write (z, x);  
  end;
```

```
Begin  
x:= 9;  
write (x);  
x := x * j;  
otro;  
End;
```

```
Var z: integer;  
Begin  
j:= 90; z:= 10  
y:= j*z;  
prueba;  
End.
```

Ejemplo

¿Prueba tiene
Variables locales?

¿Con que datos
Trabaja el módulo?

¿Qué imprime?

```
Program tres;  
  Var dato: Integer;
```

```
  Procedure prueba;  
    begin  
      dato := 25;  
      write ( dato );  
      dato := 30;  
    end;
```

```
  begin      {del programa principal}  
    dato := 0;  
    write ( dato );  
    prueba;  
    write ( dato );  
  end.
```

Ejemplo

¿Proceso tiene
Variables locales?

¿Con que datos
Trabaja el módulo?

¿Qué imprime?

```
Program cuatro;  
  Var dato: Integer;
```

```
  Procedure prueba;  
    Var dato: integer;  
    begin  
      write ( dato );  
      dato: = 30;  
      write ( dato );  
    end;
```

```
  begin    {del programa principal}  
    dato:= 0;  
    write ( dato);  
    prueba;  
    write ( dato);  
  end.
```

Program cinco;

Var num: Integer;

Procedure prueba;

Var dato: integer;

begin

dato: = 30;

num := num +1;

write (dato, num);

end;

Procedure otro;

Var dato: integer;

begin

write (dato);

dato: = 15;

write (num, dato);

end;

Var dato: integer;

begin *{del programa principal}*

num:= 0;

write (num);

prueba;

write (num);

otro;

write (dato);

end.

Ejemplo

¿Qué imprime
en cada write?

Programación I

Temas

- ✓ Repaso
- ✓ Comunicación entre módulos
- ✓ Ejemplos

Repaso

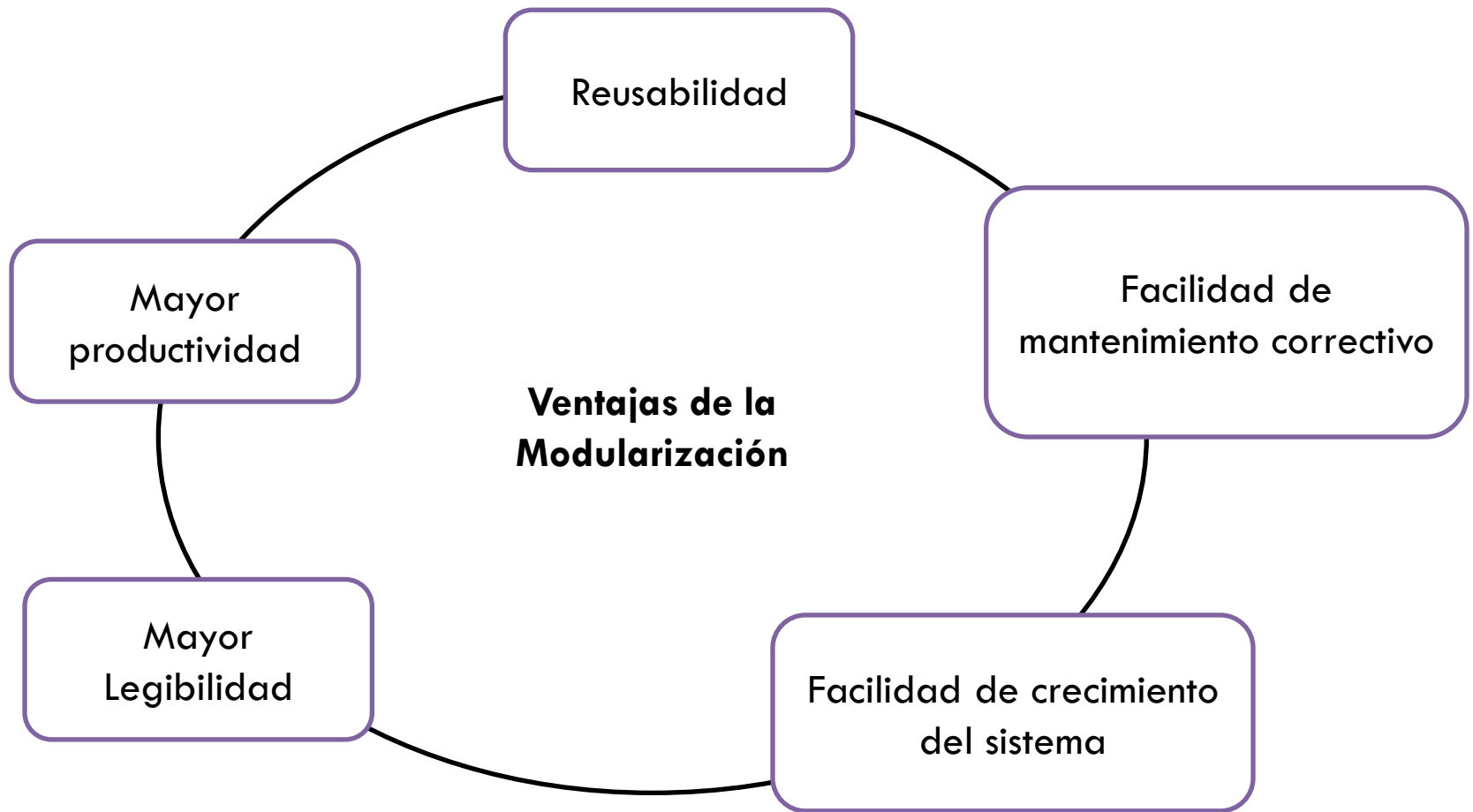
✓ ¿Qué es la modularización?

Modularizar es una estrategia que implica dividir un problema en partes funcionalmente independientes, que encapsulen operaciones y datos

✓ ¿A qué llamamos módulo?

Es un conjunto de instrucciones que cumplen una tarea específica bien definida, se comunican entre sí adecuadamente y cooperan para conseguir un objetivo común

Repaso



Alcance de una variable

- ✓ Establece el contexto donde la variable es conocida o puede ser referenciada en el marco de un programa.
- ✓ La variable puede ser de alcance: **GLOBAL** O **LOCAL**

```
program otroejemplo;
var a, b: integer;
```

```
procedure cuatro (parámetros formales);
var c: char;
procedure cinco (parámetros formales);
var d: real;
begin
  b := a * 2 + 4; d:= 2,5;
  writeln(a, b, c, d);
end;
```

```
var h: integer;
begin
  b := a * 2;
  cinco (parámetros actuales);
  c:= 'A';
  writeln(a, b, c, h);
  h:= seis (parámetros actuales);
end;
```

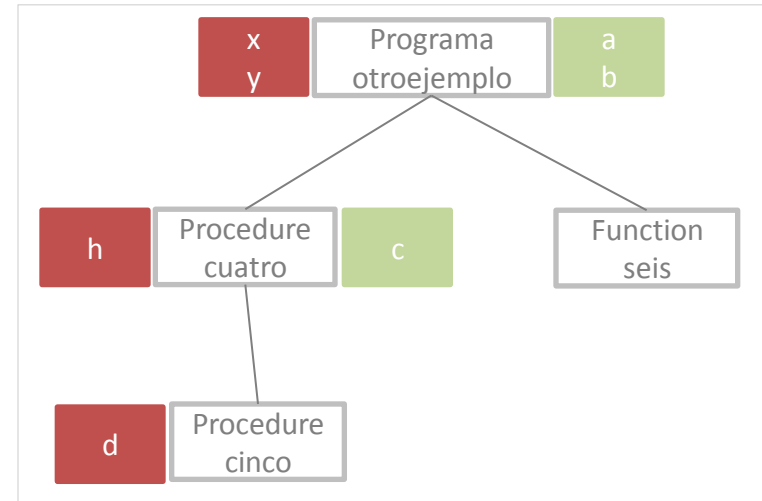
```
Function seis (parámetros formales): tipo;
begin
  a:= a+b;
  seis:= (a+c)/100;
end;
```

```
Var x, y: integer;
begin
  x:= 5; y:= 20 mod 10;
  cuatro (parámetros actuales);
  cinco (parámetros actuales);
  a:= seis (parámetros actuales);
  writeln(x, y);
  writeln(a, b);
end.
```

Supongamos que identificamos:

variables locales

variables “globales”



Teniendo en cuenta el alcance de las variables y la visibilidad de los módulos, analizar:

¿Qué invocaciones son válidas?

¿Es válida la utilización de las variables en el programa principal?

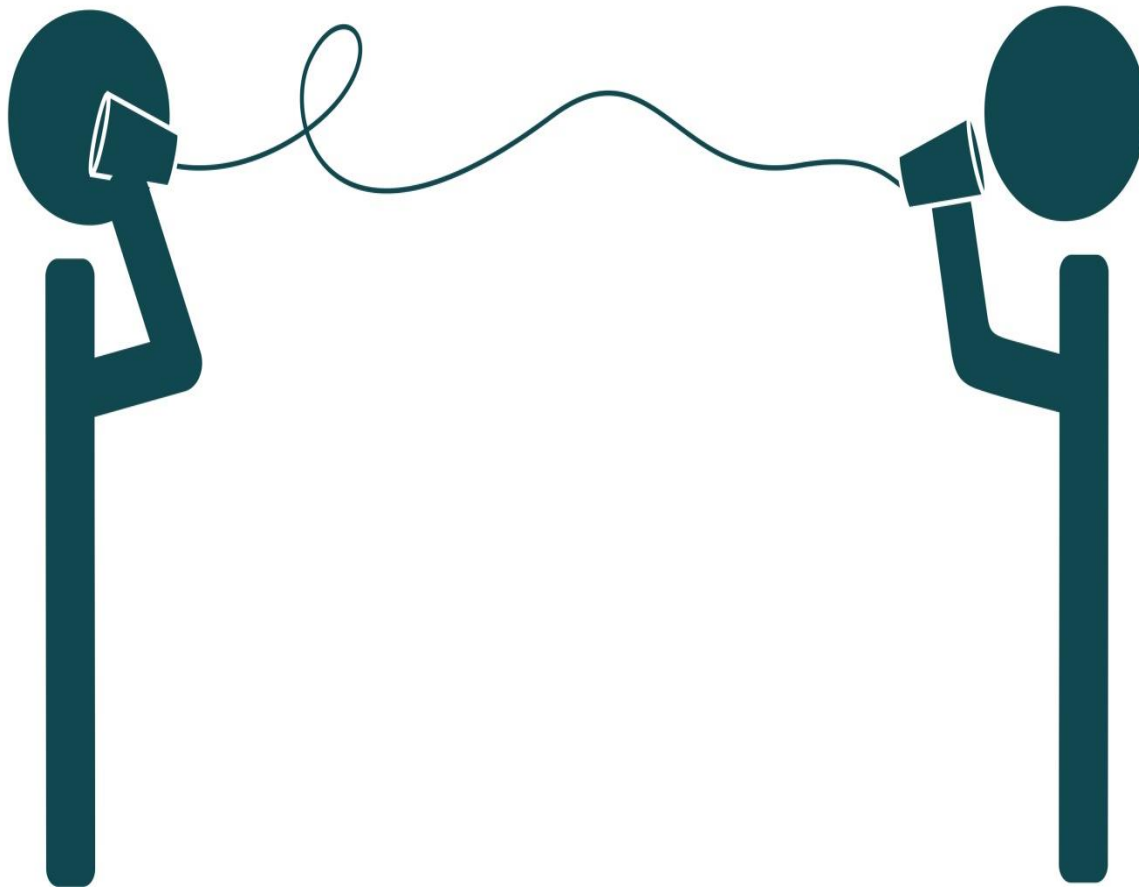
¿Y en el procedure cuatro?

¿Y en el procedure cinco?

¿Y en la function seis?

Alcance
de los
datos

COMUNICACIÓN ENTRE MÓDULOS



Modularización - Comunicación

¿CÓMO SE COMUNICAN LOS DATOS ENTRE MÓDULOS Y PROGRAMA?

VARIABLES GLOBALES

NO ACONSEJABLE. PUEDE
TRAER EFECTOS COLATERALES.
PERJUDICA LA LEGIBILIDAD

PARÁMETROS

INDICAN DE MANERA EXPLÍCITA QUÉ
DATOS UTILIZARÁ EL MÓDULO

Desventajas de la comunicación a través de variables globales

- Posibilidad de perder integridad de los datos, al modificar involuntariamente en un módulo alguna variable que luego deberá utilizar otro módulo. ¿Independencia? ¿Reusabilidad?
- Dificultad durante la etapa de la verificación
- Demasiadas variables en la sección de declaración
- Posibilidad de conflicto entre los nombres de las variables utilizadas por diferentes programadores
- Falta de especificación del tipo de comunicación entre los módulos
- Uso de memoria

No se recomienda el uso de variables globales

Se recomienda una solución que combina:

OCULTAMIENTO DE DATOS Y PARAMETROS

- El **ocultamiento de datos** significa que los datos exclusivos de un módulo o programa **NO** deben ser "visibles" o utilizables por los demás módulos.

- El **uso de parámetros** significa que los datos compartidos se deben especificar como parámetros que se transmiten entre módulos.

Variables locales del módulo



Los datos propios del módulo se declararan locales al módulo

Variables locales del programa



Los datos propios del programa se declararan locales al programa



Los datos compartidos se declararán como parámetros.

No nos sirven para la comunicación!!

El parámetro se define como una variable que representa un dato compartido entre módulos o entre un módulo y el programa principal.

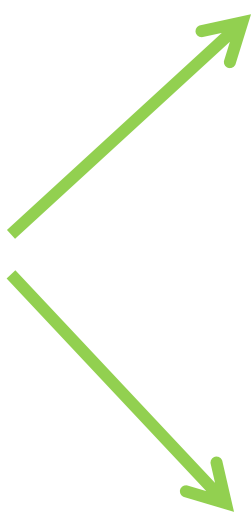
Por lo tanto, el dato compartido se especificará como un parámetro que se trasmite entre los módulos.

Parámetros

¿Cómo se implementan los parámetros en Pascal?

Modularización - Comunicación

La comunicación
mediante parámetros
puede ser **en Pascal**



Parámetros por valor

Parámetros por referencia

Parámetros por valor



Comunicación – Parámetros por valor

- Un dato de entrada por valor es llamado parámetro **IN** y significa que el módulo recibe (sobre una variable local) un valor proveniente de otro módulo (o del programa principal).
- Con él se pueden realizar operaciones y/o cálculos, pero **no producirá ningún cambio ni tampoco tendrá incidencia fuera del módulo.**

Un parámetro por valor es un dato de entrada que significa que el módulo recibe una copia de un valor proveniente de otro módulo o del programa principal.

Con este dato el módulo puede realizar operaciones y/o cálculos, pero fuera del módulo ese dato NO reflejará cambios.

Parámetro
por valor

```
Program ejemplo1;  
  
  Procedure uno (x:integer);  
  Begin  
    x:= x+1;  
    write (x);  
  End;  
  
  var num: integer;  
  
  Begin  
    num:=9;  
    uno (num);  
    write (num);  
  End.
```

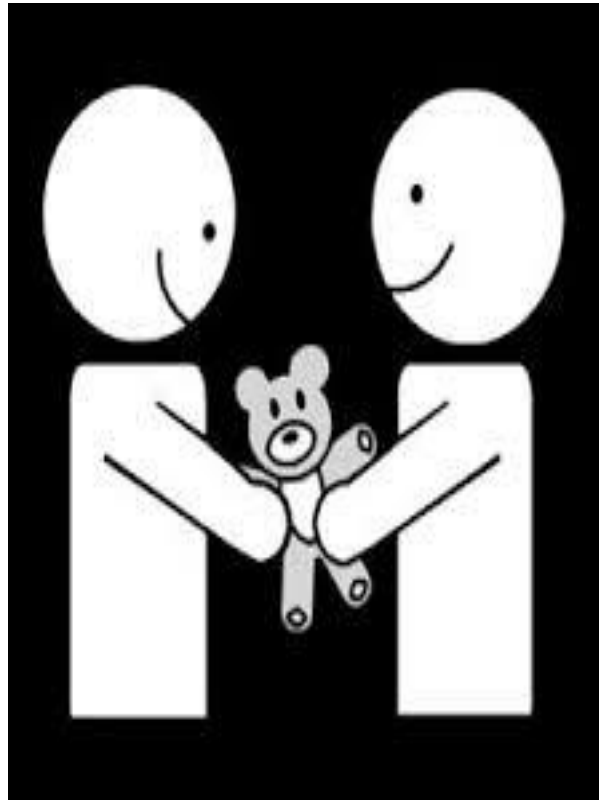
¿Qué valores
imprime?

ejemplo1

num=9

Memoria

Parámetros por referencia



Comunicación – Parámetros por referencia

La comunicación por referencia significa que **el módulo recibe la dirección de memoria** de una variable conocida en el punto de invocación.

Dentro del módulo se puede operar con el valor original contenido en esa dirección de memoria, y las modificaciones que se produzcan se reflejan en los demás módulos que conocen la variable.

En el encabezado del módulo se distinguen por tener la palabra clave VAR.

El módulo que recibe este parámetro puede operar con la información que se encuentra en la dirección de memoria compartida y las modificaciones que se produzcan se reflejarán en los demás módulos que conocen esa dirección de memoria compartida.

Parámetro
por
referencia

```
Program ejemplo2;  
  Procedure dos (var x:integer);  
  Begin  
    x:= x+1;  
    write (x);  
  End;  
var num: integer;  
Begin  
  num:=9;  
  dos (num);  
  write (num);  
End.
```

¿Qué valores
imprime?

Ejemplo2

num=90

Memoria

```
Program ejemplo1;
```

```
Procedure uno (x:integer);
```

```
Begin
```

```
  x:= x+1;
```

```
  write (x); {1}
```

```
End;
```

```
var num: integer;
```

```
Begin
```

```
  num:=9;
```

```
  uno (num);
```

```
  write (num); {2}
```

```
End.
```

En {1} se muestra 10 como valor de x

En {2} se muestra 9 como valor de x

```
Program ejemplo2;
```

```
Procedure dos (var x:integer);
```

```
Begin
```

```
  x:= x+1;
```

```
  write (x); {3}
```

```
End;
```

```
var num: integer;
```

```
Begin
```

```
  num:=9;
```

```
  dos (num);
```

```
  write (num); {4}
```

```
End.
```

En {3} se muestra 10 como valor de x

En {4} se muestra 10 como valor de x

¿Cómo se explican los valores que se imprimen?

Consideraciones...

Un **parámetro por valor** debe ser tratado como una variable local del módulo.

La utilización de este tipo de parámetros puede significar una utilización importante de memoria.

Los **parámetros por referencia** en cambio operan directamente sobre la dirección de la variable original, en el contexto del módulo que llama.

Esto significa que no requiere memoria local.

Variables globales VS Parámetros

Variables globales

- Posibilidad de perder integridad de los datos, al modificar involuntariamente en un módulo alguna variable que luego deberá utilizar otro módulo
- Posibilidad de conflicto entre los nombres de las variables utilizadas por diferentes programadores
- Dificultad durante la etapa de la verificación

Parámetros



Los módulos no pueden afectar involuntariamente los datos de otros módulos. ¡Independencia! ¡Reusabilidad!



Se reduce significativamente la cantidad de variables globales



Permite distinguir el tipo de comunicación



Los parámetros por referencia no utilizan memoria local



El uso de parámetros por valor puede significar una importante utilización de memoria local

COMUNICACIÓN DE DATOS ENTRE MODULOS Y PROGRAMA

```
graph TD; A[COMUNICACIÓN DE DATOS ENTRE MODULOS Y PROGRAMA] --> B[VARIABLES GLOBALES]; A --> C[PARAMETROS]; C --> D[Por valor]; C --> E[Por referencia];
```

VARIABLES
GLOBALES

PARAMETROS

Por valor

Por referencia

*No recomendables
para la
comunicación!!*

En resumen

Cuando se invoca a un módulo se deben tener cuenta las restricciones propias del lenguaje de implementación. En Pascal:

- La invocación al módulo (parámetros actuales)
- La interface del módulo (parámetros formales)
- Los parámetros actuales y formales



Deben coincidir en cantidad y tipo de dato

Se relacionan 1 a 1

Consideraciones
generales

```
Program ejemplo3;  
  Procedure Calcular (x, y: integer;  
                     var suma, prod: integer);  
  begin  
    suma:= x + y;  
    prod:= x * y;  
  end;
```

```
Var  val1, val2, s, p: integer;
```

```
Begin
```

```
  Calcular (6, 17, s, p);
```



```
  val1:= 10; val2:= 5;
```

```
  Calcular (val1, val2, s, p);
```



```
  Calcular (23, val2, 14, p);
```



```
End.
```

Program Ejemplo4;

```
procedure Imprimir (var a:integer; b: integer; c: integer);  
begin  
    writeln ('a= ', a, ' b= ', b, ' c= ', c);  
    writeln;  
    a := 10;  
    c := a + b;  
    b := b * 5;  
    writeln ('a= ', a, ' b= ', b, ' c= ', c);  
    writeln;  
end;
```

var x, y, z: integer;

begin

y := 6;

z := 5;

writeln ('x= ', x, ' y= ', y, ' z= ', z);

writeln;

imprimir (x, y, z);

writeln ('x= ', x, ' y= ', y, ' z= ', z);

readln;

end.

Ejercitación

Ejercitación

```
program ejemplo5;
```

```
  procedure cuenta(var b: integer; var a: integer);
```

```
    procedure calculo (var b: integer; a: integer);
```

```
    begin
```

```
      b := a * 2 + 4;
```

```
      writeln('a= ', a, ' b= ', b);
```

```
      writeln;
```

```
    end;
```

```
  var c: integer;
```

```
  begin
```

```
    b := a * 2;
```

```
    calculo(a,b);
```

```
    c:= 0;
```

```
    writeln('a= ', a, ' b= ', b, ' c= ', c);
```

```
    writeln;
```

```
  end;
```

```
Var a, b,c: integer;
```

```
begin
```

```
  a:= 5;
```

```
  b:= 20 div 10;
```

```
  writeln('a= ', a, ' b= ', b, ' c= ', c);
```

```
  writeln;
```

```
  cuenta(c, a);
```

```
  writeln('a= ', a, ' b= ', b, ' c= ', c);
```

```
  writeln;
```

```
  readln;
```

```
end.
```

*Observar la
declaración de la
variable local c*

*¿Qué imprime
en cada
sentencia
write?*

Ejercitación

```
Program Ejemplo6;
```

```
var a: integer;
```

```
procedure Imprimir (var b: integer; c: integer);
```

```
begin
```

```
  writeln('a= ', a, ' b= ', b, ' c= ', c);
```

```
  writeln;
```

```
  a := a + 5;
```

```
  c := a + b;
```

```
  b := b * 5;
```

```
  writeln('a= ', a, ' b= ', b, ' c= ', c);
```

```
  writeln;
```

```
end;
```

```
var b, c: integer;
```

```
begin
```

```
  a := 20;
```

```
  b := 6;
```

```
  writeln('a= ', a, ' b= ', b, ' c= ', c);
```

```
  writeln;
```

```
  imprimir (b, a);
```

```
  writeln('a= ', a, ' b= ', b, ' c= ', c);
```

```
  writeln;
```

```
  readln;
```

```
end.
```

*¿Qué imprime
en cada
sentencia
write?*

¿Qué imprime en cada sentencia write para la siguiente secuencia?
10 15 3 -5 24

```
Program ejemplo7;
```

```
procedure calcular ( n: integer; var cant, aux: integer);  
Begin  
    cant := cant + 1;  
    if (n mod 2 = 0) then aux := aux+1;  
end;
```

```
Var
```

```
    a,c, num: integer;
```

```
begin
```

```
    writeln ('a= ',a, ' c= ', c);
```

```
    writeln;
```

```
    c:=0;
```

```
    a:=0;
```

```
    write ('Ingrese un numero (para finalizar 24): ');
```

```
    readln (num);
```

```
    while (num <> 24) do begin
```

```
        calcular (num,c, a);
```

```
        write ('Ingrese un numero (para finalizar 24): ');
```

```
        readln (num);
```

```
    end;
```

```
    writeln;
```

```
    writeln ('a= ',a, ' c= ', c);
```

```
    readln;
```

```
end.
```

Ejercitación

Ejercitación

```
Program queimprime;
  Var
    c, n, num1: integer;

  procedure valores (var cant:integer; var aux:integer; num:integer);
    var num1: integer;
  Begin
    num:= 10;
    aux:= num + 15;
    cant:= 0;
    writeln ('num= ', num, ' num1= ', num1, ' aux= ', aux, ' cant= ', cant);
    writeln;
  end;

var
  a: integer;
begin
  num1:= 50;
  writeln ('a= ', a, ' c= ', c, ' n= ', n, ' num1= ', num1);
  writeln;
  valores (c, a, n);
  writeln ('a= ', a, ' c= ', c, ' n= ', n, ' num1= ', num1);
  readln;
end.
```

¿Qué imprime
en cada
sentencia
write?

Escribir un programa que:

- a. Lea un número entero e invoque a un módulo que reciba dicho número y devuelva la cantidad de dígitos pares que contenga. El programa luego debe imprimir el resultado.
- b. Invoque a un módulo que lea una secuencia de caracteres y devuelva la cantidad de letras minúsculas leídas, la cantidad de mayúsculas leídas, la cantidad de dígitos y la cantidad del resto de los caracteres.
- c. Invoque a un módulo que lea 10 números enteros y retorne el número más grande y el más chico.
- d. Lea un número entero e invoque a un módulo que reciba dicho número y devuelva verdadero si contiene exactamente 3 dígitos 5, o falso en caso contrario. El programa luego debe informar si cumple o no.

Modularización

- Al **modularizar** es muy importante **obtener independencia funcional de cada módulo**. Esto disminuye el acoplamiento con el resto del sistema y por lo tanto reduce el impacto de las fallas y modificaciones.
- La **comunicación** entre módulos **debe** acotarse a intercambio de datos por parámetros. Siempre que se pueda deben utilizarse parámetros por valor.
- Las **funciones** pueden pensarse como operadores definidos por el usuario, que **reciben** variables (parámetros por valor) y **producen** un resultado único.
- Los **procedimientos** son subprogramas que interactúan en el espacio de datos del módulo. Puede devolver resultados a través de parámetros por referencia.
- En todos los casos **deben** evitarse las variables globales.

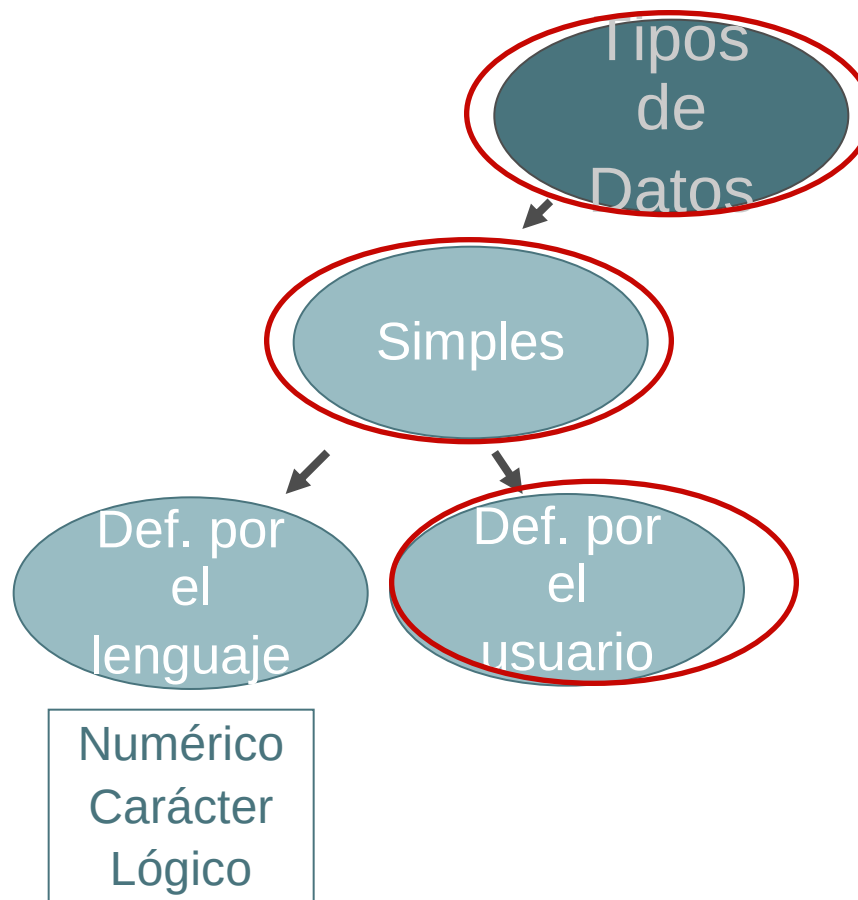
Programación I

TEMAS
de la
CLASE

- 1 Tipos de datos definidos por el usuario
- 2 Tipo de dato estructurado/
Estructura de datos

Clasificación de Tipos De datos

Hasta aquí presentamos los tipos de datos simples que son aquellos que toman un único valor, en un momento determinado, de todos los permitidos para ese tipo.



Tipos de datos definidos por el usuario

Un aspecto **muy importante** en los lenguajes de programación es la capacidad de **especificar y manejar datos no estándar**, indicando valores permitidos, operaciones válidas y su representación interna.

Esto permite:

- Aumento de la riqueza expresiva del lenguaje, con **mejores posibilidades de abstracción de datos**.
- **Mayor seguridad** respecto de las operaciones que se realizan sobre cada clase de datos.
- **Límites preestablecidos** sobre los valores posibles que pueden tomar las variables que corresponden al tipo de dato.

Tipos de datos definidos por el usuario

¿Qué ventajas tiene DECLARAR tipos?

- **Flexibilidad:** en el caso de ser necesario modificar la forma en que se representa el dato, sólo se debe modificar una declaración en lugar de un conjunto de declaraciones de variables.
- **Documentación:** se pueden usar como identificador de los tipos, nombres autoexplicativos, facilitando de esta manera el entendimiento y lectura del programa.
- **Seguridad:** se reducen los errores por uso de operaciones inadecuadas del dato a manejar, y se pueden obtener programas más confiables.

Tipos de datos definidos por el usuario

Un *tipo de dato definido por el usuario* es aquel que no existe en la definición del lenguaje, y el programador es el encargado de su especificación.

Sintéticamente entonces un Tipo significa una clase de datos que tiene asociado:

- Un rango de valores posibles.
- Una forma de representación.
- Un conjunto de operaciones permitidas.
- Un conjunto de condiciones de valores permitidos que se pueden verificar.

Tipos de datos definidos por el usuario

En Pascal, los tipos deben ser declarados antes de ser usados.

La declaración de tipos se hace a través de la palabra clave TYPE de la siguiente forma:

```
TYPE identificador = tipo;
```

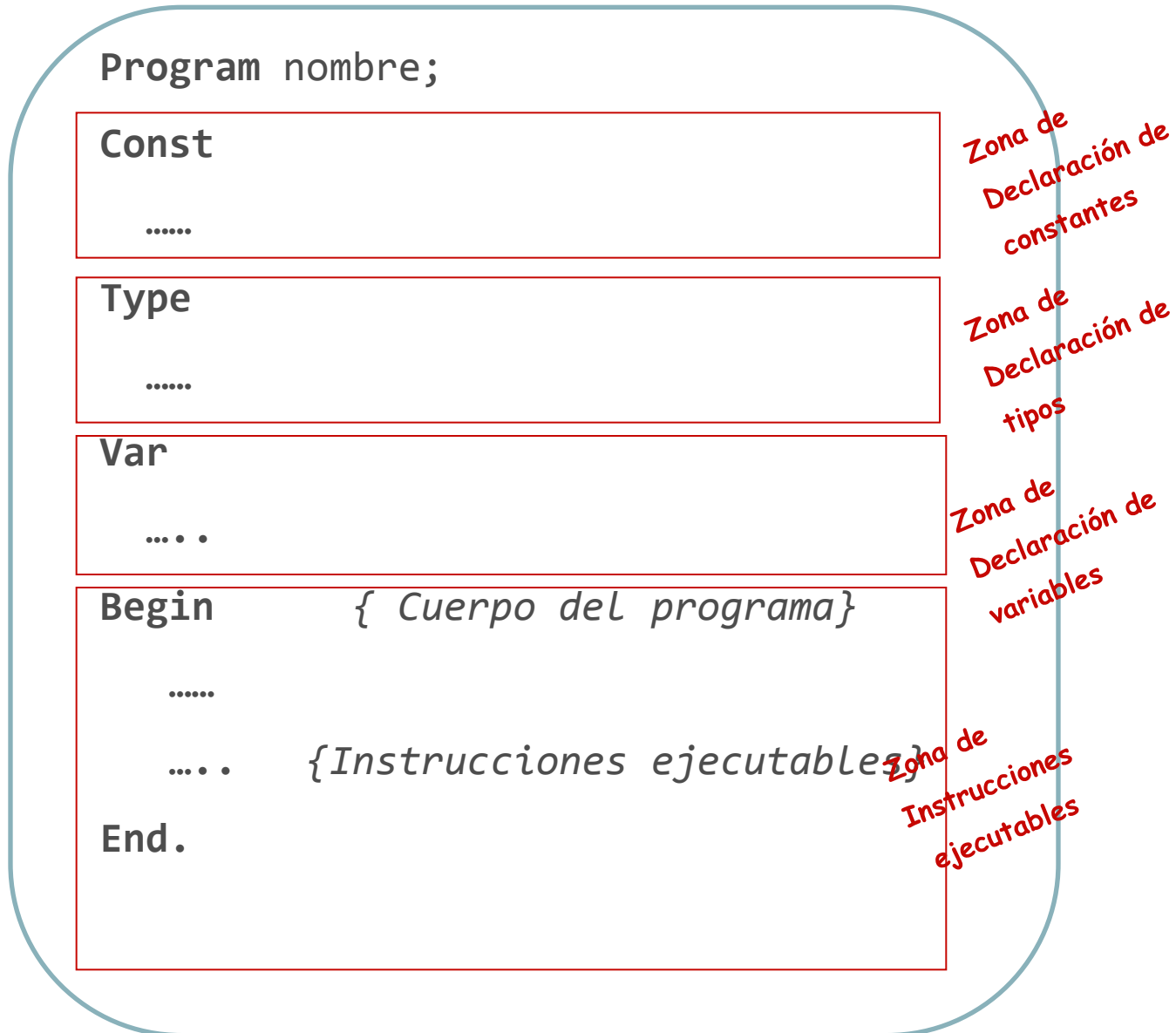


Nombre con que se conocerá al tipo de dato en el programa.



Puede ser un tipo estándar o alguno de los tipos de datos definidos por el usuario.

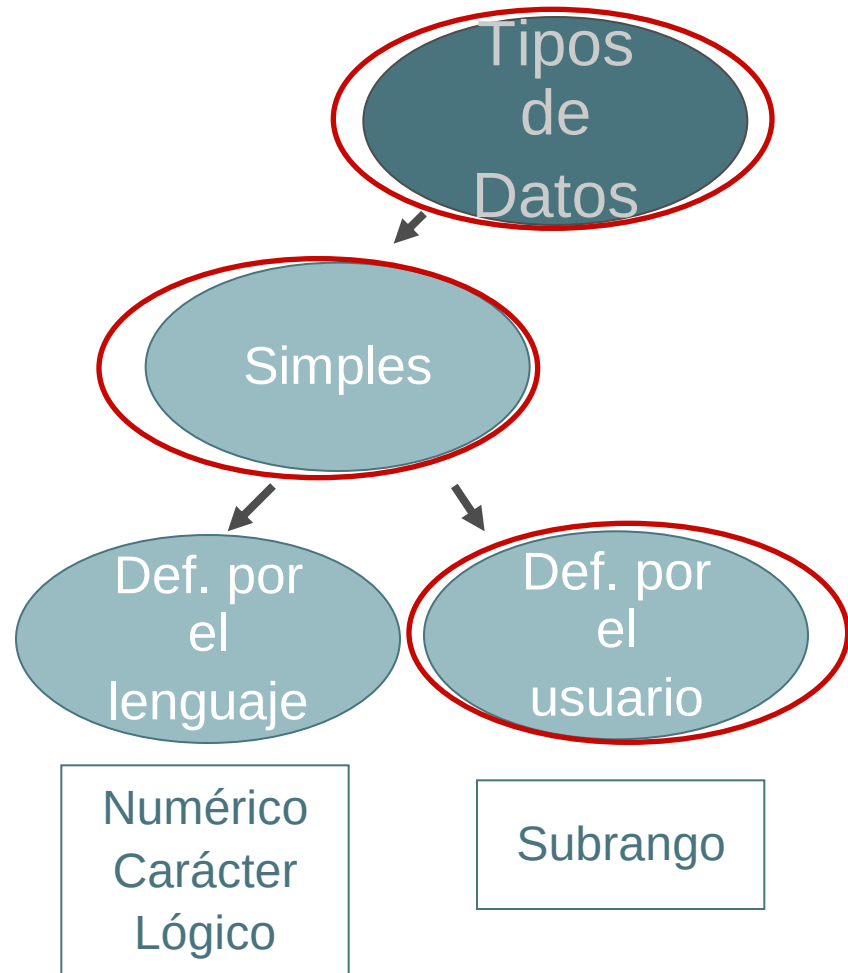
Esquema general de un programa que usa tipos definidos por el usuario



Clasificación de los Tipos de Datos

Recordemos la clasificación de los tipos de datos ya vista...

Ahora vamos a comenzar a trabajar con un tipo de dato **simple y definidos por el usuario**.



Tipo de dato definido por el usuario: SUBRANGO

Un tipo de dato **subrango** es un tipo simple y ordinal que consiste de una sucesión de valores extraídos de un tipo ordinal base.

Para declarar un tipo **SUBRANGO** se deben especificar los valores inicial y final de la sucesión, separados por dos puntos seguidos:

```
Type  identificador = valor inicial .. valor final;
```


Tipo de dato definido por el usuario: SUBRANGO

Program nombre;

Const

fin = 1999;

Type

siglo_pasado= 1900 .. Fin;

mayusculas = 'A' .. 'Z';

Var

letra: mayusculas;

año: siglo_pasado;

Begin { *Cuerpo del programa*}

...

... {*Instrucciones ejecutables*}

End.

Tipo SUBRANGO - Operaciones

- Las **operaciones** de un tipo de dato subrango se heredan del tipo base.

Los tipos base de los que se pueden definir subrangos son:

- ✓ Enteros
- ✓ Caracteres

- La ocupación en memoria estará condicionada por el tipo base

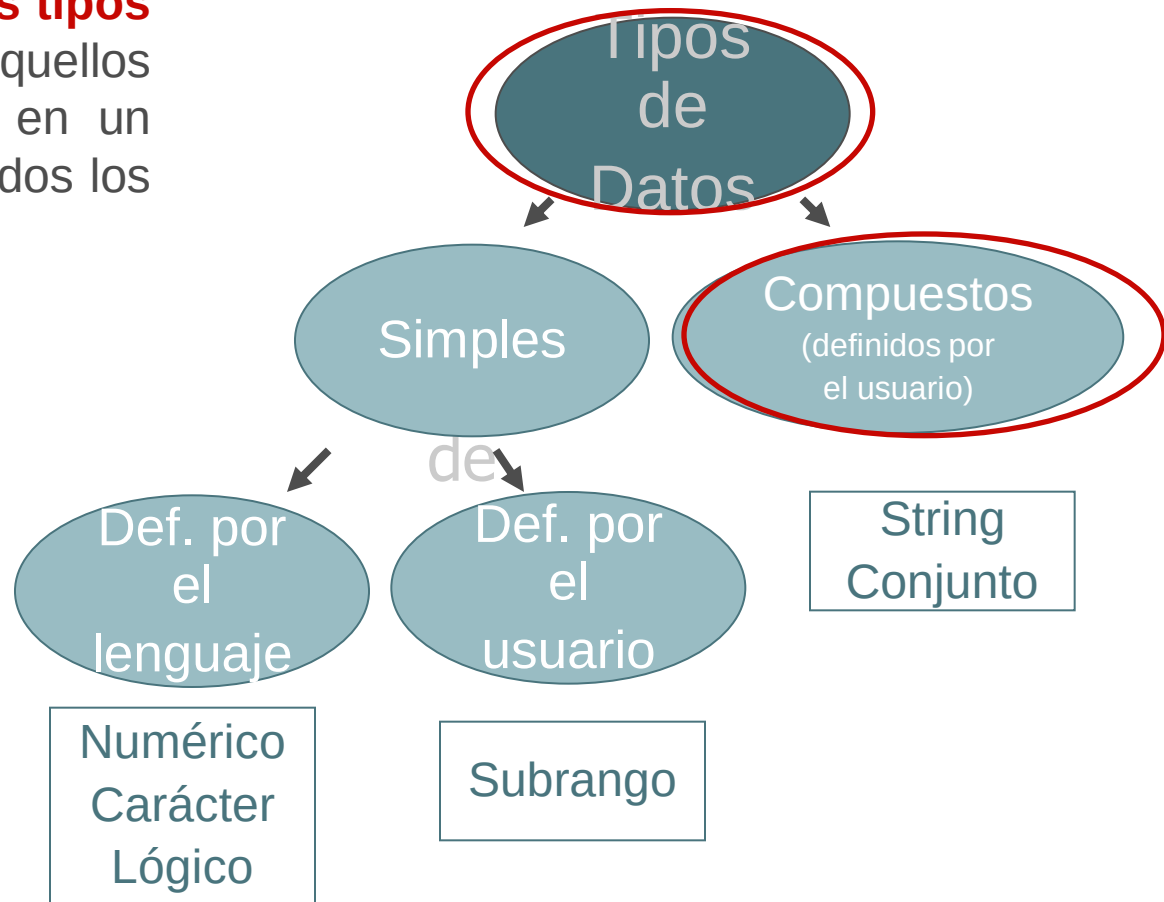
```
Program ejemplo;  
Const  minimo = 1;  
        maximo = 500;  
  
Type  
    rango = minimo .. maximo;  
    meses = 1 .. 12;  
    letras = 'A' .. 'z';  
  
Var  
    dato1, dato2 : rango;  
    descanso : meses;  
    letra: letras;  
  
Begin  
    dato1 := dato1 Div dato2;  
    descanso:= 1;  
    Read (letra);  
    ...  
End.
```



Ejercicio 1: Realice un programa que lea edades de 20 alumnos de la facultad e informe el promedio de edades y la edad más grande.

Clasificación de Tipos De datos

- **Hasta aquí presentamos los tipos de datos simples** que son aquellos que toman un único valor, en un momento determinado, de todos los permitidos para ese tipo.



- **Ahora comenzaremos a trabajar con los tipos de datos compuestos** que son aquellos que pueden tomar varios valores a la vez que guardan alguna relación lógica entre ellos.

Tipo de dato definido por el usuario: STRING

Un tipo de dato string es una sucesión de caracteres de longitud determinada.

```
TYPE identificador = string [ longitud ];
```

- Longitud es el número máximo de caracteres que puede contener el dato.
- En PASCAL cuando no se especifica la longitud ese identificador podrá contener como máximo 255 caracteres.
- La cantidad de memoria que utiliza una variable está determinada por su longitud. Recordar que cada carácter ocupa 1 byte.

Tipo de dato definido por el usuario: STRING

Program nombre;

Type

cadena10 = string [10];

cadena25 = string [25];

fecha = string [8];

día = string [2];

Var

nom1, nom2, nom3 : cadena10;

apellido : cadena25;

fecha1, fecha2 : fecha;

Begin { *Cuerpo del programa*

.....

.... {*Instrucciones ejecutables*}

End.

Tipo de dato STRING: Operaciones

Las operaciones permitidas son:

- **Asignación ($:=$)**
- **Entrada/Salida (Read / write)**
- **De relación ($>, <, =, \dots$)**



Ejercicio 2: Realice un programa que lea nombres y DNI de personas hasta leer el nombre 'Ana', que debe procesarse. Se debe informar la cantidad personas cuyo DNI es par e informar el nombre de las personas cuyo DNI tiene exactamente 3 dígitos con valor 5.

Tipo CONJUNTO - Características

- Desde el punto de vista informático un tipo conjunto representará una colección de datos simples (además los datos que estarán guardados en el conjunto deben ser de tipo ordinal), sin repetición y limitada por la implementación en cada lenguaje o sistema operativo.
- No necesariamente existe en la mayoría de los lenguajes.
- Es un tipo de datos compuesto.
- No es un tipo de datos ordinal.

Tipo CONJUNTO - Características

- Se pueden tener conjuntos de valores cuyos tipos serán integer, boolean o char.
- En la implementación de Pascal el conjunto no puede tener más de 255 elementos (en la práctica esto no se tendrá en cuenta).
- No permite operaciones de lectura - escritura.
- Permite la asignación, unión, intersección, pertenencia, diferencia

Tipo CONJUNTO – Definición en Pascal

Type

identificador = **set of** *tipo ordinal*;

letras = set of char;

Var

conjLetras: letras;

Tipo CONJUNTO – Operaciones

■ Asignación

```
Program conj1;  
Type  
  letras = set of char;  
Var  
  letras1, letras2, letras3: letras;  
  
Begin  
  letras1:= [];  
  letras2:= ['a'..'f'];  
  letras3:= letras2;  
  ...  
End.
```

Tipo CONJUNTO – Operaciones

■ Unión

- ❑ Se representa con el signo + y da como resultado otro conjunto.
- ❑ En este conjunto resultado aparecen los elementos de los dos conjuntos y aquellos elementos repetidos aparecen una vez.

```
Program conj2;  
Type  
    conjcar = set of char;  
Var  
    carac1,carac2: conjcar;  
  
Begin  
    carac1:= ['E', '9'];  
    carac2:= ['a'] + carac1;  
    ...  
End.
```

Tipo CONJUNTO – Operación Intersección

■ Intersección

□ Se representa con el signo * y da como resultado otro conjunto. En el conjunto resultado aparecen solamente los elementos comunes a los dos conjuntos.

```
Program conj3;
```

```
Type
```

```
  conjcar = set of char;
```

```
Var
```

```
  carac1, carac2: conjcar;
```

```
Begin
```

```
  carac1:= ['E', '9'];
```

```
  carac2:= ['a'] * carac1;
```

```
  ...
```

```
End.
```

Tipo CONJUNTO – Operación Diferencia

■ Diferencia

- Se representa con el signo - y da como resultado otro conjunto.
- Este conjunto resultado contiene los elementos que están en el primer conjunto y no están en el segundo.

Program conj4;

Type

conjcar = set of char;

Var

carac1,carac2: conjcar;

Begin

carac1:= ['E', '9'];

carac2:= ['a','E'] - carac1;

...

End.

Tipo CONJUNTO – Operación Pertenencia

■ Pertenencia

- ❑ Se representa con el operador `in` y da como resultado un valor lógico.
- ❑ Esta operación devuelve verdadero si el elemento está en el conjunto y falso en caso contrario.

```
Program conj5;
```

```
Type
```

```
    conjcar = set of char;
```

```
Var
```

```
    carac1,carac2: conjcar;
```

```
Begin
```

```
    carac1:= ['E', '9'];
```

```
    if ('a' IN carac1) then
```

```
        ...
```

```
End.
```


Tipo CONJUNTO – Operaciones

■ Comparación

□ Se pueden usar los operadores relacionales para determinar si un conjunto está incluido en otro ($\leq$), si son distintos ($\neq$) ó iguales ($=$).

Program conj6;

Type

conjcar = set of char;

Var

carac1, carac2: conjcar;

Begin

carac1:= ['E', '9'];

carac2:= ['E', '9', 'F'];

if (carac1 $\leq$ carac2)

then ...

End.



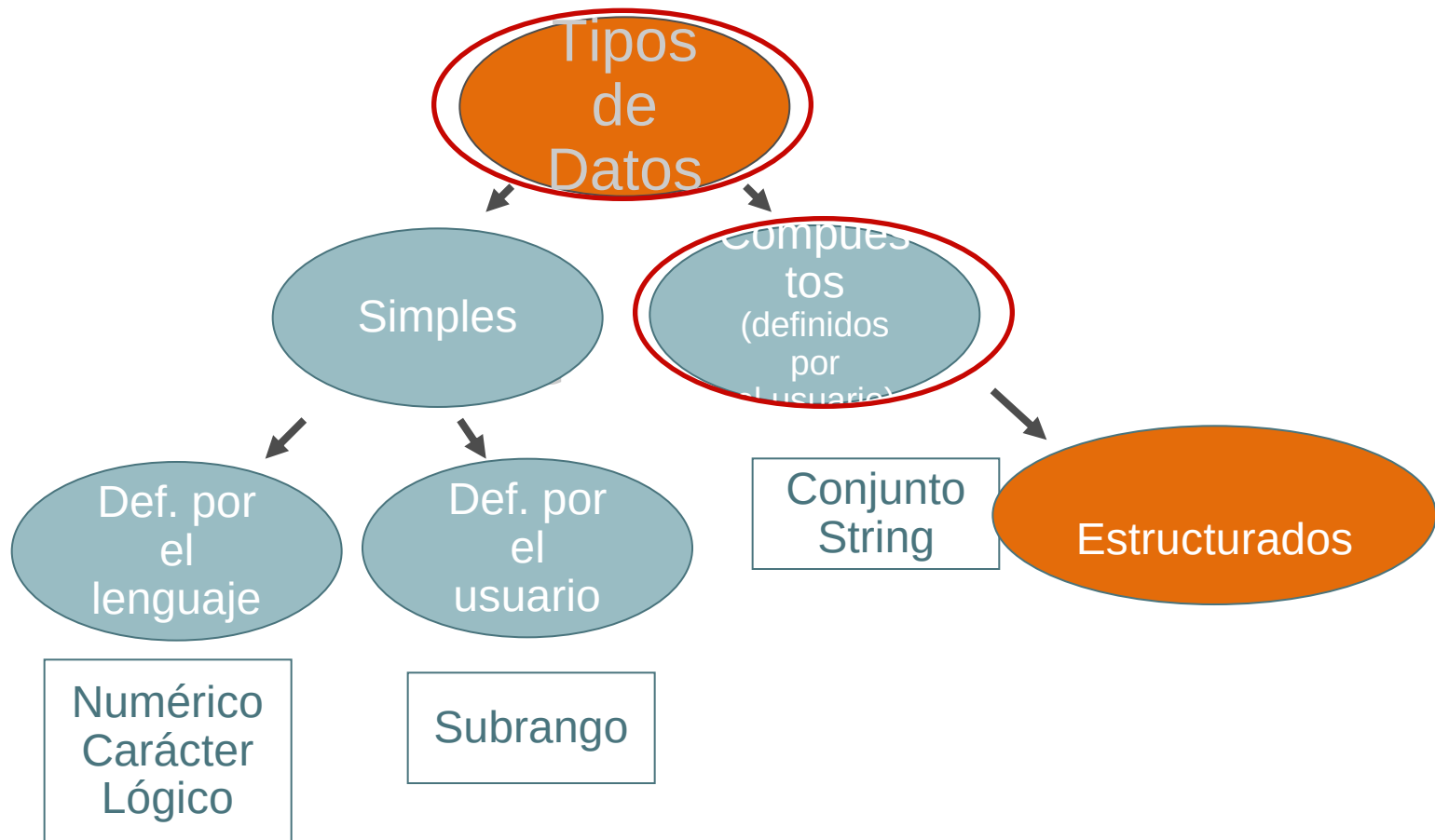
Ejercicio 3: Realice un programa que lea caracteres hasta leer el carácter '@', al finalizar informe la cantidad de consonantes minúsculas y la cantidad de vocales minúsculas leídas.

1

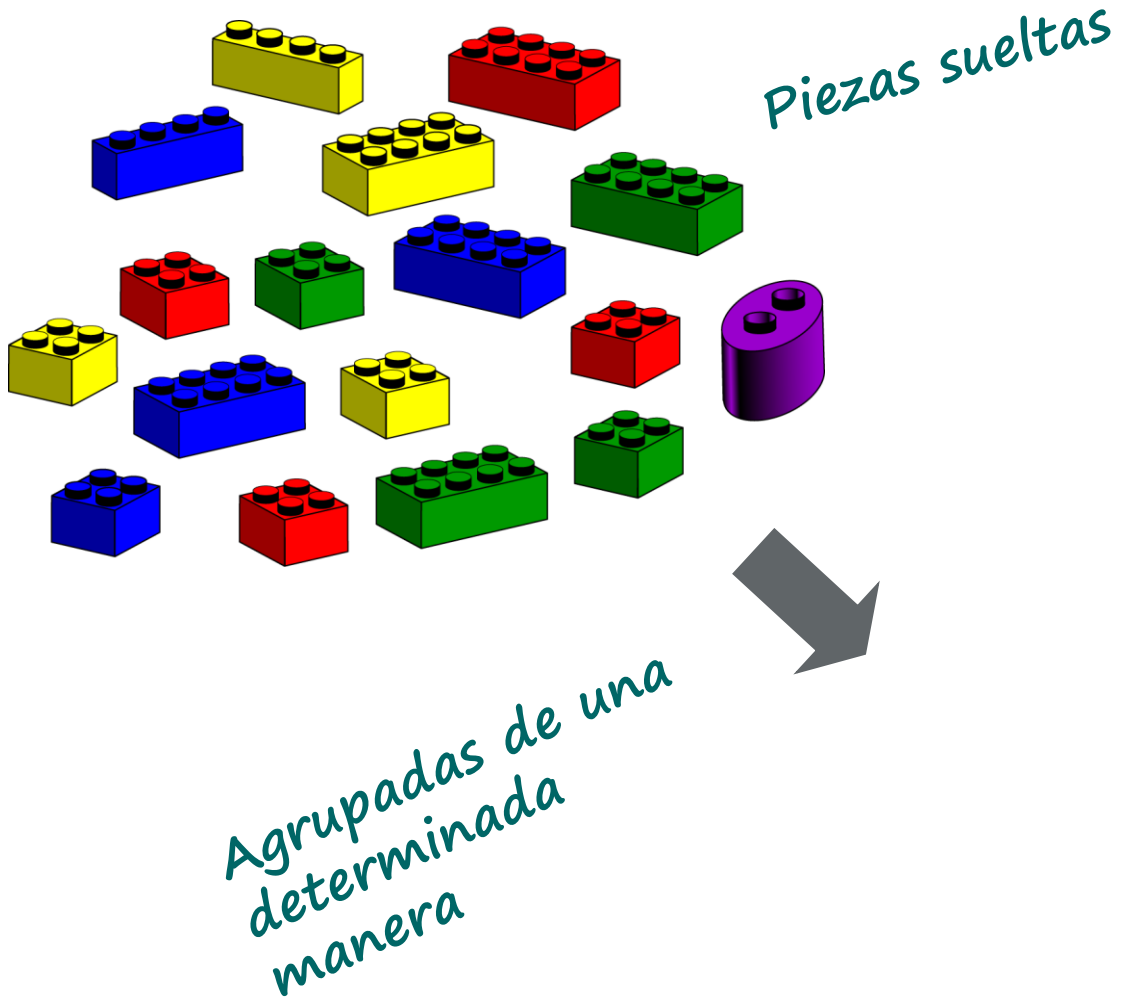
Tipo de dato estructurado/ Estructura de datos

Concepto

Clasificación



Estructuras de datos – Motivación



Mafalda

Estructuras de datos

Una estructura de datos es un conjunto de variables (que podrían ser de distinto tipo) relacionadas entre sí y que se puede operar como un todo, bajo un nombre único.

Esto nos va permitir representar los elementos del mundo real, que generalmente son más complejos que un número entero o una palabra.

Estructuras de datos

Representar los datos de **empleados** de una empresa. Se identifica a través del nombre, el número de documento, la fecha de nacimiento, el número de legajo, el sexo, el sueldo, la antigüedad, etc.



Representar los datos de un **producto** del **supermercado**. Se debería identificar el código, la marca, la identificación, el precio, la fecha de vencimiento, etc.

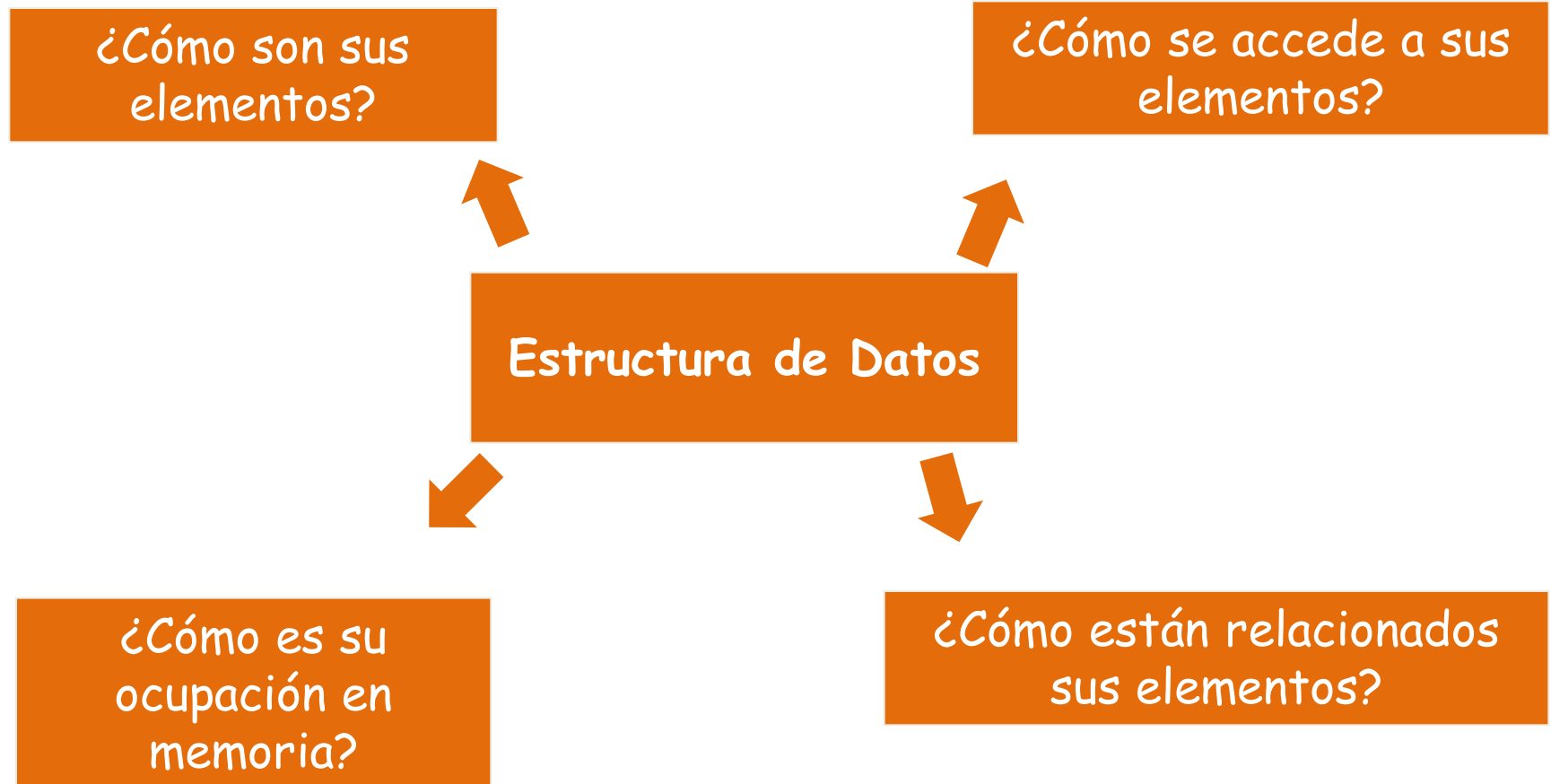


Representar la **lista de clubes de un torneo de fútbol**. Se deberían considerar los nombres de los equipos de fútbol.

Huracán
Independiente
Lanús
San Lorenzo
Arsenal
Banfield
Colón
Newells
Gimnasia (LP)
Estudiantes
Vélez
Olimpo
Quilmes
San Martín (SJ)

Estructuras de datos - Clasificación

Las estructuras de datos se pueden clasificar desde distintos puntos de vista:



Estructuras de datos - Clasificación

De acuerdo a los tipos de datos que se pueden almacenar en la estructura:



Cola de personas

Estructura de Datos

Homogénea

Heterogénea



Los objetos del escritorio



Los Envases Plásticos



Los Animales domésticos



Las partes de mi computadoras

Estructuras de datos - Clasificación

Las estructuras de datos pueden clasificarse, de acuerdo al tipo de datos que la componen en homogéneas y heterogéneas.

Una estructura de datos se dice **homogénea** si los datos que la componen son todos del mismo tipo.

Una estructura de datos se dice **heterogénea** si los datos que la componen son de distinto tipo.



Cola de
personas



Los Envases Plásticos



Los Animales domésticos



Las partes de mi computadoras



Los objetos del escritorio

Estructuras de datos - Clasificación

De acuerdo a la ocupación de memoria las estructuras pueden ser:



Las butacas del teatro

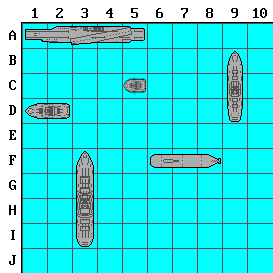
Estructura de Datos

Estática

Dinámica



Una pila de canastos



El cuadro para el juego de la Batalla Naval



La fila de personas frente a la ventanilla de pagos

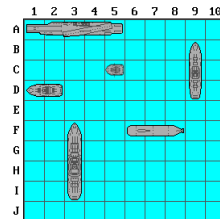
Estructuras de datos - Clasificación

Las estructuras de datos pueden clasificarse, de acuerdo a la ocupación de memoria en estáticas y dinámicas.

Una estructura de datos se dice **estática** si la cantidad de elementos que contiene es fija, es decir que la cantidad de memoria que ocupa no varía durante la ejecución del programa.



Las butacas del teatro



El cuadro para el juego de la Batalla Naval

Una estructura de datos se dice **dinámica** si la cantidad de elementos que contiene es variable, y por lo tanto la cantidad de memoria ocupada puede cambiar durante la ejecución de un programa.



La pila de canastos



La fila de personas frente a la ventanilla de pagos

Estructuras de datos - Clasificación

De acuerdo al acceso a sus elementos, las estructuras pueden ser:

Microsoft Excel - Libro1

	A	B	C	D
1	ANDRADE GONZALEZ ANA ALICIA			
2	ALMENDRA MENDEZ OSCAR DAVID			
3	CASTILLO VEGA JULIO CESAR			
4	BANDA GARCIA BRENDA			
5	CORZO MORENO LUIS ALBERTO			
6	DIAZ NUÑEZ ISRAEL			
7	CRUZ SANTOS CLAUDIA YOMELI			
8	FLORES ESCOBAR NAHUM			
9	GARCIA COLORADO MARTHA IZTAYANA			
10	HERNANDEZ REYES LUCERO MONTSERRAT			
11	HERNANDEZ ESPEJO KAREN			
12	HERNANDEZ DAVILA RAFAEL			
13	LAGUNES MENDOZA DIANA LAURA			
14	LARA RODRIGUEZ TANIA GUADALUPE			
15	LAGUNES CRUZ XOCHITL JEREMY			
16	LARA SALAS CARLOS ALBERTO			
17	MORALES SILVA RUTH EUNICE			
18	SAGUAYA REYES ANA BARBARA			
19	SOLIS PEREZ ZURISADAI			
20	SALDIVAR ARIAS SANDRA IVONNE			
21	SOTO PORTILLO DENISSE			
22	USCANGA CRUZ IVAN			
23	VELAZQUEZ ZARATE CLAUDIA GIOVANNA			
24	CABRERA MARTINEZ JOSE LUIS			
25	COLIN ROMAN ADILENE VIANEY			
26	REYES CAMPOS ELIZABETH			
27	RODRIGUEZ FUENTES ANA BELEM			
28	SILVA ORTEGA ZELTZIN NAYELI			
29	VASQUEZ SANCHEZ MARIANA			
30	WILGGIN JIMENEZ OMAR			
31				



Estructura de Datos

De acceso
secuencial

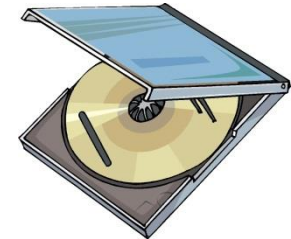
De acceso
directo



Seleccionar una
aplicación determinada



Localizar una imagen en un rollo de película



Seleccionar una canción determinada

Estructuras de datos - Clasificación

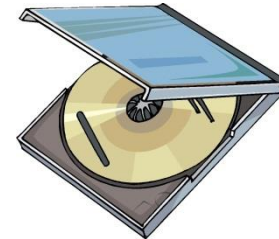
Las estructuras de datos pueden clasificarse, de acuerdo a como se accede a sus elementos, de acceso secuencial o directo.

Una estructura de datos se dice de **acceso secuencial**, si para acceder a un elemento particular se debe respetar un orden predeterminado, por ejemplo, pasando por todos los elementos que le preceden.



Localizar una imagen en un rollo de película

Una estructura de datos se dice de **acceso directo**, si se puede acceder a un elemento particular, directamente, sin necesidad de pasar por los anteriores a él, por ejemplo, indicando una posición.



Seleccionar una canción determinada

Estructuras de datos - Clasificación

De acuerdo a la relación entre sus elementos, las estructuras pueden ser:



Productos a cobrar en la caja del supermercado

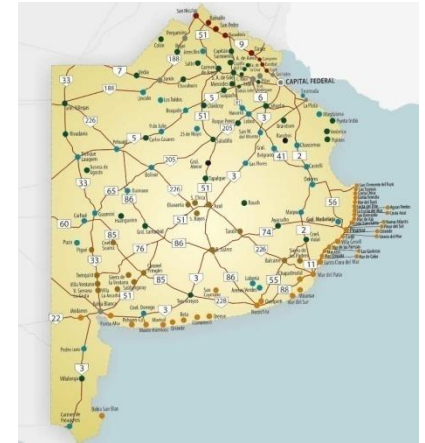
Estructura de Datos

Lineales

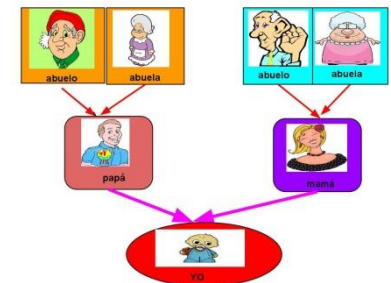
No lineales



Los vehículos que pasan por la cabina de peaje (cada uno tiene uno antes y uno después)



Las rutas que unen localidades de la Pcia de Bs. As.



Nuestro árbol genealógico

Estructuras de datos - Clasificación

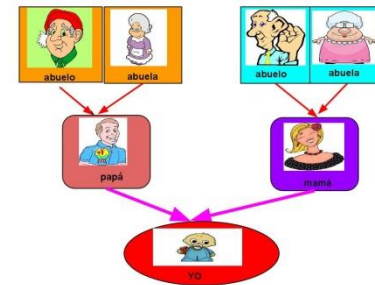
Las estructuras de datos pueden clasificarse, de acuerdo a su linealidad en Lineales y No Lineales.

Una estructura de datos se dice **lineal** cuando está formada por ninguno, uno o varios elementos que guardan una relación de adyacencia ordenada donde a cada elemento le sigue uno y le precede uno, **solamente**.



Productos a cobrar en la
caja del supermercado

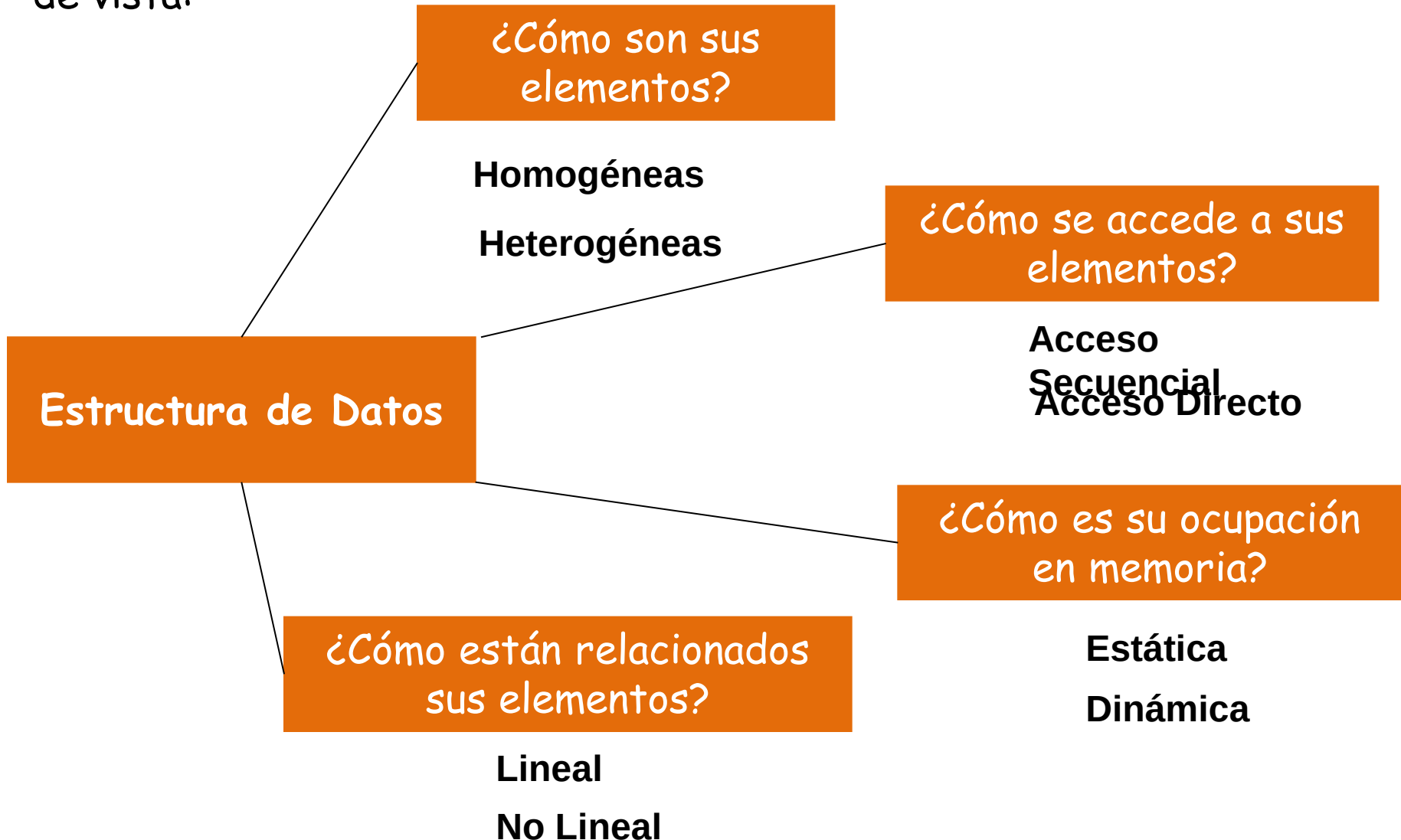
Una estructura de datos se **dice No lineal** si para un elemento dado pueden existir 0, 1 ó mas elementos que le suceden y 0, 1 ó mas elementos que le preceden.



Nuestro arbol genealógico

Estructuras de datos - Resumen

Las estructuras de datos se pueden clasificar desde distintos puntos de vista:



Programación I

1

Tipo de dato REGISTRO

Concepto

Declaración en Pascal

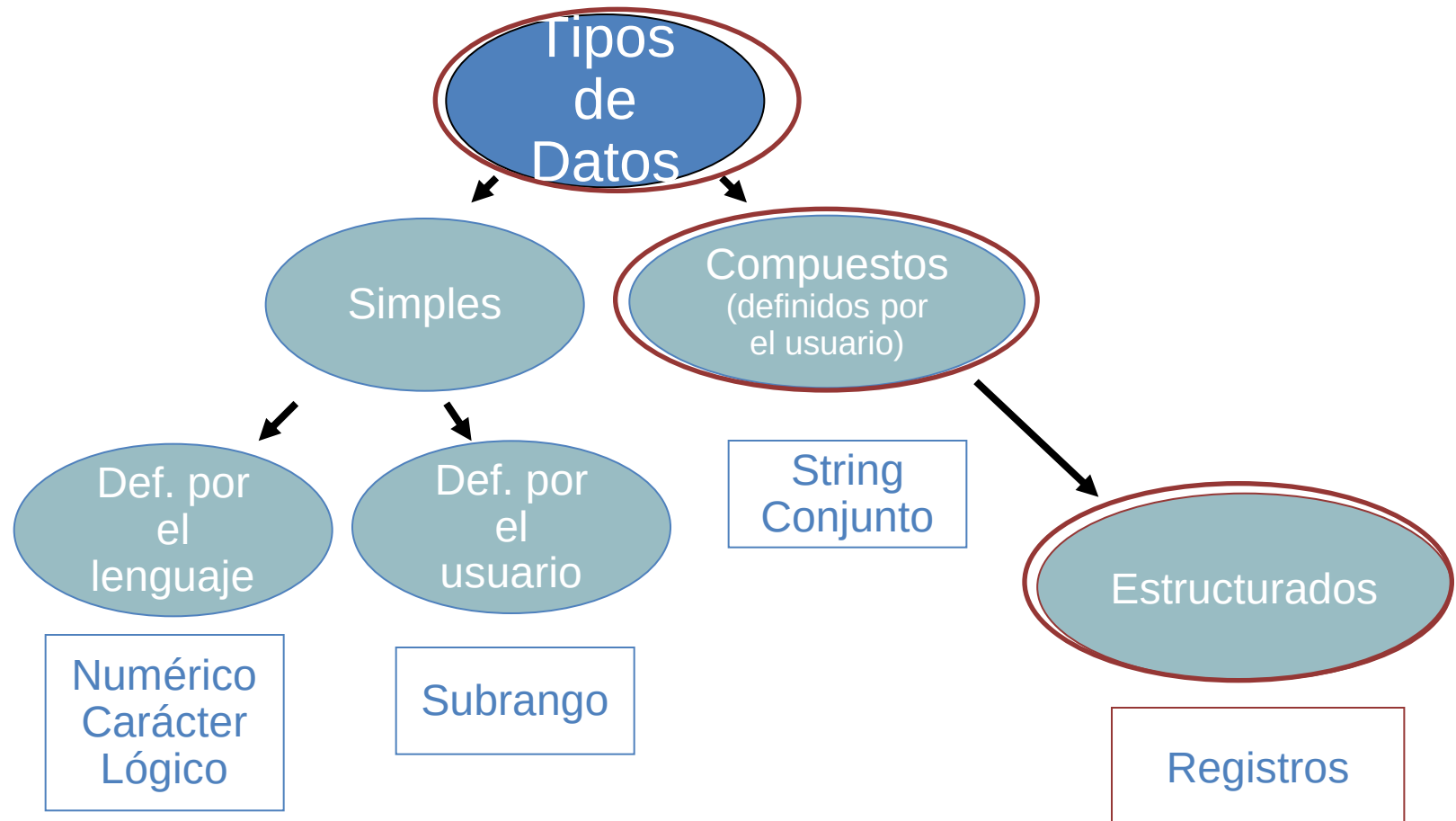
Ejercitación

2

Corte de control

Ejercitación con registros

Trabajaremos con el tipo de dato estructurado REGISTRO



Tipo de dato Registro – Características

Un registro es una estructura de datos que cumple con:

- Los valores pueden ser de diferentes tipos, esto convierte a un registro en una estructura de datos **heterogénea**.
- El almacenamiento ocupado por un registro es fijo, por esto, un registro es una estructura **estática**.
- El **acceso** a sus componentes (campos) es **directo**. Debe referenciarse a través de su nombre.



Nombre
DNI
Fecha Nac.
NroLegajo
Sexo
Sueldo
Antigüedad



Nombre
Nro. Alumno
Datos Personales
Materias que cursa
Materias aprobadas



Origen
Destino
Fecha envío
Mensaje



Código
Marca
Nombre
Precio
Fecha de vencimiento

Tipo de dato Registro – Declaración en Pascal

- Se identifica el nombre del tipo como registro (RECORD).
- Se especifica el **nombre** y **tipo** de los campos individuales que componen el tipo. La lista de campos se encierra entre las palabras claves record y end.
- Cada uno de los campos tiene un identificador. Los campos pueden ser nombrados individualmente, como variables ordinarias.
- Los campos pueden ser de cualquier tipo predefinido o definidos por el usuario (estáticos).

Type

```
nombre = record  
    Campo1: tipoA;  
    Campo2: tipoB;  
    .....  
end;
```

Var

```
nombre-variable:nombre;
```

Tipo de dato Registro – Declaración en Pascal



Código
Marca
Nombre
Precio
Fecha de vencimiento

Type

```
cadena15 = string [15];
```

```
producto= Record
```

```
    codigo: integer;
```

```
    Nombre: cadena15;
```

```
    Marca: cadena15;
```

```
    FechaVenc?????
```

```
    Precio: real;
```

```
End;
```

Var

```
    prod: producto;
```

Observemos el campo fecha

Tipo de dato Registro – Declaración en Pascal



Código
Marca
Nombre
Precio
Fecha de

El tipo fecha puede ser definido como un registro

Type

```
días= 1..31;  
meses = 1..12;  
anios = 1900..2100  
fecha = record  
    día: dias;  
    mes: meses;  
    año: anios;  
end;
```

Type

```
cadena15 = string [15];  
días= 1..31;  
meses = 1..12;  
anios = 1900..2100  
fecha = record  
    día: dias;  
    mes: meses;  
    año: anios;  
end;  
producto= Record  
    codigo: integer;  
    nombre: cadena15;  
    marca: cadena15;  
    fechaVenc: fecha;  
    Precio: real;  
End;
```

Var

```
prod: producto;
```

¿Cuál es la ocupación en memoria para prod?

Tipo de dato Registro – Acceso a los campos

- Para acceder a los campos de un registro, se necesita especificar tanto el nombre del registro como el del campo que interesa.
- Esto se denomina calificar al campo.
- En Pascal se hace de la siguiente forma:

Nombre-Registro.nombre-campo

Prod.nombre

Prod.precio

Prod.fechaVenc.dia

prod.marca

Type

```
cadena15 = string [15];
días= 1..31;
meses = 1..12;
anios = 1900..2100
fecha = record
    día: dias;
    mes: meses;
    año: anios;
end;
producto= Record
    codigo: integer;
    nombre: cadena15;
    marca: cadena15;
    fechaVenc: fecha;
    Precio: real;
End;
```

Var

```
prod: producto;
```

Operaciones aplicadas a los campos de un registro

- Dado que el Registro es un tipo de dato estructurado, las operaciones deberán aplicarse a cada uno de los campos que lo componen.
- Como los campos de un registro son variables de algún tipo, las operaciones posibles sobre un campo son las permitidas para el tipo de dato correspondiente.

- **Asignación**
- **Comparación**
- **Lectura/escritura**

Type

```
cadena15 = string [15];
días= 1..31;
meses = 1..12;
anios = 1900..2100
fecha = record
    día: dias;
    mes: meses;
    año: anios;
end;
producto= Record
    codigo: integer;
    nombre: cadena15;
    marca: cadena15;
    fechaVenc: fecha;
    Precio: real;
End;
Var prod: producto;
```

Operaciones aplicadas a los campos de un registro

Asignación

prod.precio := 20

prod.nombre := 'Yerba'

prod.marca := 'SanCor'

Prod.fechaVenc.año:=2019

Type

```
cadena15 = string [15];  
días= 1..31;  
meses = 1..12;  
anios = 1900..2100  
fecha = record  
    día: dias;  
    mes: meses;  
    año: anios;  
end;  
producto= Record  
    codigo: integer;  
    nombre: cadena15;  
    marca: cadena15;  
    fechaVenc: fecha;  
    Precio: real;  
End;
```

```
Var prod: producto;
```

Operaciones aplicadas a los campos de un registro

Comparación

if prod.precio=20 then ...

**if (prod.precio > 30) and
(prod.precio < 50) then ...**

while (prod.marca= 'SanCor') do begin

if (prod.fechaVenc.año) =2018 then ...

Type

cadena15 = string [15];

días= 1..31;

meses = 1..12;

anios = 1900..2100

fecha = **record**

día: dias;

mes: meses;

año: anios;

end;

producto= **Record**

codigo: integer;

nombre: cadena15;

marca: cadena15;

fechaVenc: fecha;

Precio: real;

End;

Var prod: producto;

Operaciones aplicadas a los campos de un registro



Lectura

```
Read (prod.marca);  
Read (prod.codigo);  
Read (prod.fechaVenc.mes);
```



Escritura

```
write (prod.marca);  
write (prod.nombre);  
write (prod.fechaVenc.año);
```

Type

```
cadena15 = string [15];  
días= 1..31;  
meses = 1..12;  
anios = 1900..2100  
fecha = record  
    día: dias;  
    mes: meses;  
    año: anios;  
end;  
producto= Record  
    codigo: integer;  
    nombre: cadena15;  
    marca: cadena15;  
    fechaVenc: fecha;  
    Precio: real;  
End;  
Var prod: producto;
```

Observar que el orden en que se leen los campos del registro podría no coincidir con el orden especificado en la declaración del tipo empleado ¿Por qué?

Operaciones permitidas para el tipo Registro

→ La única operación permitida entre registros es la de Asignación, **siempre y cuando**, los dos registros estén definidos del mismo tipo.

Type

```
cadena15 = string [15];  
días= 1..31;  
meses = 1..12;  
anios = 1900..2100  
fecha = record  
    día: dias;  
    mes: meses;  
    año: anios;  
end;  
producto= Record  
    codigo: integer;  
    nombre: cadena15;  
    marca: cadena15;  
    fechaVenc: fecha;  
    Precio: real;  
End;
```

```
Var prod1, prod2: producto;  
Begin
```

```
.....
```

```
    prod2 := prod1;
```

```
.....
```

¿Cómo se comparan variables de tipo registro?

No pueden realizarse comparaciones entre registros completos, es decir para saber si dos registros son iguales, se debe evaluar cada uno de los campos.

Type

```
días= 1..31;  
meses = 1..12;  
anios = 1900..2100  
fecha = record  
    día: dias;  
    mes: meses;  
    año: anios;  
end;
```

Para comparar dos registros puede realizarse una función de la forma:

```
function iguales (f1, f2 : fecha) : boolean;  
    begin  
        iguales:= (f1.dia=f2.dia) and (f1.mes=f2.mes) and (f1.año = f2.año);  
    end;
```

Ejemplo para la lectura del registro Producto

Type

```
cadena15 = string [15];
días= 1..31;
meses = 1..12;
anios = 1900..2100
fecha = record
    día: días;
    mes: meses;
    año: anios;
end;
producto= Record
    codigo: integer;
    nombre: cadena15;
    marca: cadena15;
    fechaVenc: fecha;
    Precio: real;
End;
Var prod: producto;
```

```
Procedure LeerProducto (Var prod: producto);
begin
    Readln (prod.codigo);
    Readln (prod.nombre);
    Readln (prod.marca);
    Readln (prod.fechaVenc.día);
    Readln (prod.fechaVenc.mes);
    Readln (prod.fechaVenc.año);
    Readln (prod.precio);
end;
```

¿Podríamos pensar en un
procedimiento de lectura para el
registro fecha?

```
Procedure LeerFecha (Var f: fecha);
begin
    Readln (f.día);
    Readln (f.mes);
    Readln (f.año);
end;
```


Ejercitación



Se leen 100 datos correspondientes a los productos de un supermercado. Obtener un listado con los nombres de los productos con precio entre 25 y 50 pesos e informar la cantidad de productos de marca 'Ala'.

*Se puede utilizar la declaración del tipo
producto vista anteriormente...*

Type

```
cadena15 = string [15];
días= 1..31;
meses = 1..12;
anios = 1900..2100
fecha = record
    día: dias;
    mes: meses;
    año: anios;
end;
producto= Record
    codigo: integer;
    nombre: cadena15;
    marca: cadena15;
    fechaVenc: fecha;
    Precio: real;
End;
```



Se leen 100 datos correspondientes a los productos de un supermercado. Obtener un listado con los nombres de los productos con precio entre 25 y 50 pesos e informar la cantidad de productos de marca 'Ala'.

```
Inicializar cantidad
Repetir 100
  Leer producto
  si precio entre 25 y 50 entonces
    muestro nombre
  si marca = Ala then incremento cantidad
Fin
Mostrar cantidad
```

```
Procedure LeerProducto (Var prod: producto);
  Procedure LeerFecha (Var f: fecha);
    begin
      Readln (f.dia);
      Readln (f.mes);
      Readln (f.año);
    end
  begin
    Readln (prod.codigo);
    Readln (prod.nombre);
    Readln (prod.marca);
    LeerFecha (prod.fechaVenc);
    Readln (prod.precio);
  end;
```

Program ejemplo1;

Type

```
cadena15 = string [15];
días= 1..31;
meses = 1..12;
años = 1900..2100
fecha = record
  día: dias;
  mes: meses;
  año: años;
end
producto = Record
  codigo: integer;
  nombre: cadena15;
  marca: cadena15;
  fechaVenc: fecha;
  Precio: real;
End;
```

{implementación módulo LeerProducto}

```
var prod: producto; cant, i: integer;
```

```
begin {Programa principal}
```

```
  cant:= 0;
```

```
  for i:= 1 to 100 do begin
```

```
    LeerProducto (prod);
```

```
    if (prod.precio >=25 and prod.precio <=50)
      then Writeln (prod.nombre);
```

```
    if (prod.marca = 'Ala') then cant := cant +1;
```

```
  end;
```

```
  write (cant)
```

```
end.
```

Ejercitación



Se leen datos correspondientes a los productos de un supermercado. La lectura finaliza con código igual a -1. Obtener un listado con los nombres de los productos con precio entre 25 y 50 pesos e informar la cantidad de productos de marca 'Ala'.

*Se puede utilizar la declaración del tipo
producto vista anteriormente...*

Type

```
cadena15 = string [15];
días= 1..31;
meses = 1..12;
anios = 1900..2100
fecha = record
    día: dias;
    mes: meses;
    año: anios;
end;
producto= Record
    codigo: integer;
    nombre: cadena15;
    marca: cadena15;
    fechaVenc: fecha;
    Precio: real;
End;
```

Se leen datos correspondientes a los productos de un supermercado. La lectura finaliza con código igual a -1. Obtener un listado con los nombres de los productos con precio entre 25 y 50 y la marca 'Ala'.

```
Inicializar cantidad
Leer producto
Mientras producto <> -1
    si precio entre 25 y 50 entonces
        mostrar nombre
    si marca = Ala then incremento cantidad
    leer otro producto
Fin
Mostrar cantidad
```

```
Procedure LeerProducto2 (Var prod: producto);
Procedure LeerFecha (Var f: fecha);
begin
    Readln (f.dia);
    Readln (f.mes);
    Readln (f.año);
end
begin
    Readln (prod.codigo);
    if (prod.código <> -1)
    then begin
        Readln (prod.nombre);
        Readln (prod.marca);
        LeerFecha (prod.fechaVenc);
        Readln (prod.precio);
    end;
end;
```

Program ejemplo2;

Type

```
cadena15 = string [15];
días= 1..31;
meses = 1..12;
años = 1900..2100
fecha = record
    día: días;
    mes: meses;
    año: años;
end
producto = Record
    código: integer;
    nombre: cadena15;
    marca: cadena15;
    fechaVenc: fecha;
    Precio: real;
End;
```

{implementación módulo LeerProducto2}

var prod: producto; cant: integer;

begin {Programa principal}

cant:= 0;

LeerProducto2 (prod);

While (prod.codigo <> -1) **do begin**

if (prod.precio >=25 and prod.precio <=50)

then **Writeln** (prod.nombre);

if (prod.marca = 'Ala') then cant := cant +1;

LeerProducto2 (prod);

end;

write (cant)

end.

2 Corte de Control

Ejercitación con Registros

Ejercitación



Un supermercado requiere el procesamiento de los productos que dispone. De cada producto se conoce su código, nombre, marca, stock y precio unitario. El procesamiento finaliza con el código -1 y los productos de igual marca se leen consecutivamente.

Se requiere informar:

- La cantidad en stock de productos de cada marca

Codigo	Nombre	Marca	Stock	Precio
1000	Leche	SanCor	100	20
1100	Yoghurt	SanCor	200	25
5500	Manteca	SanCor	50	30
5055	Detergente	Ala	0	55
4500	Jabón en Polvo	Ala	20	200
2400	Fideos	Matarazzo	35	30
3000	Ravioles	Matarazzo	35	80
5250	Cerveza	Quilmes	100	50
-1				

¿Qué significa que los productos de igual marca se leen consecutivamente?

Resultados:

- SanCor 350
- Ala 20
- Matarazzo 70
- Quilmes 100



Un supermercado requiere el procesamiento de los productos que dispone. De cada producto se conoce su código, nombre, marca, stock y precio unitario. El procesamiento finaliza con el código -1 y los productos de igual marca se leen consecutivamente.

Se requiere informar:

- La cantidad en stock de productos de cada marca.

Codigo	Nombre	Marca	Stock	Precio
1000	Leche	SanCor	100	20
1100	Yoghurt	SanCor	200	25
5500	Manteca	SanCor	50	30
5055	Detergente	Ala	0	55
4500	Jabón en Polvo	Ala	20	200
2400	Fideos	Matarazzo	35	30
3000	Ravioles	Matarazzo	35	80
5250	Cerveza	Quilmes	100	50
-1				

```
Leer Datos del producto
mientras haya productos en el super
    inicializar total por marca
    mientras sea la misma marca
        actualizar total por marca
    leer otro producto
fin mientras
Mostrar total por marca
fin mientras
```

¿Analizamos los datos?

Codigo	Nombre	Marca	Stock	Precio
1000	Leche	SanCor	100	20
1100	Yoghurt	SanCor	200	25
5500	Manteca	SanCor	50	30
5055	Detergente	Ala	0	55
4500	Jabón en Polvo	Ala	20	200
2400	Fideos	Matarazzo	35	30
3000	Ravioles	Matarazzo	35	80
5250	Cerveza	Quilmes	100	50
-1				

Leer Datos producto

mientras haya productos en el super

inicializar total por marca

mientras sea la misma marca

actualizar total por marca

leer otro producto

fin mientras

Mostrar total por marca

fin mientras

Type

```
cadena15 = string [15];
```

```
producto= Record
```

```
    codigo: integer;
```

```
    nombre: cadena15;
```

```
    marca: cadena15;
```

```
    stock: integer;
```

```
    precio: real
```

```
End;
```


Ejercitación

Si ahora el problema nos pide, además, que informe:

- Los nombres de los productos con stock en cero.
- Los códigos de los productos que tienen exactamente dos dígitos iguales a 5.
- La cantidad de productos cuyo precio es mayor a \$40.



Codigo	Nombre	Marca	Stock	Precio
1000	Leche	SanCor	100	20
1100	Yoghurt	SanCor	200	25
5500	Manteca	SanCor	50	30
5055	Detergente	Ala	0	55
4500	Jabón en Polvo	Ala	20	200
2400	Fideos	Matarazzo	35	30
3000	Ravioles	Matarazzo	35	80
5250	Cerveza	Quilmes	100	50
-1				

Resultados:

- Nombres productos con stock en cero: Detergente
- Códigos con dos dígitos iguales a 5: 5500, 5250
- Cantidad de productos con precio mayor a \$40: 4



Si ahora el problema nos pide además que informe:

- Los nombres de los productos con stock en cero.
- Los códigos de los productos que tienen exactamente dos dígitos iguales a 5.
- La cantidad de productos cuyo precio es mayor a \$40.

inicializar contador > \$40

Leer Datos producto

mientras haya productos en el super

inicializar total por marca

mientras sea la misma marca

actualizar total por marca

Si stock=0 entonces mostrar nombre

Si código tiene 2 dígitos 5 entonces mostrar código producto

Si precio > 40 entonces aumentar contador > \$40

leer otro producto

fin mientras

Mostrar total por marca

fin mientras

Mostrar contador > \$ 40

¿Qué módulos podemos implementar?

¿Escribimos el programa en Pascal?

inicializar contador > \$40

Leer Datos producto

mientras haya productos en el super

 inicializar total por marca

 mientras sea la misma marca

 actualizar total por marca

Si stock=0 entonces mostrar nombre

Si código tiene 2 dígitos 5 entonces mostrar código producto

Si precio > 40 entonces aumentar contador > \$40

 leer otro producto

fin mientras

 Mostrar total por marca

fin mientras

Mostrar contador > \$ 40

```
{programa principal}
var
  prod:producto;
  totalst, c40:integer;
  aux:cadena10;

begin
  c40:=0;
  leerProducto(prod);
  while (prod.codigo<>-1) do begin
    totalst:=0;
    aux:=prod.marca;
    while (prod.codigo<>-1) and (aux=prod.marca) do begin
      totalst:= totalst+prod.stock;
      if (prod.stock=0) then writeln (prod.nombre);
      if (digitos5(prod.codigo)) then writeln (prod.nombre);
      if (prod.precio>40) then c40:= c40+1;
      leerProducto(prod)
    end;
    writeln ('El total stock marca ', aux, ' es: ', totalst);
  end;
end.
```

```

program CortedeControl;
Type cadena10=string[10];
      producto=record
          codigo:integer;
          nombre:cadena10;
          marca:cadena10;
          stock:integer;
          precio:real;
      end;
procedure leerProducto(var p:producto);
begin
  readln(p.codigo);
  if (p.codigo<>-1) then begin
    readln(p.nombre);
    readln(p.marca);
    readln(p.stock);
    readln(p.precio);
  end;
end;
function digitos5 (num:integer): boolean;
var cant:integer;
begin
  cant:=0;
  while (num<>0) and (cant<3) do begin
    if (num mod 10 = 5) then cant:= cant+1;
    num:= num div 10;
  end;
  digitos5:= cant=2;
end;

```

```

{programa principal}
var
  prod:producto;
  totalst, c40:integer;
  aux:cadena10;

begin
  c40:=0;
  leerProducto(prod);
  while (prod.codigo<>-1) do begin
    totalst:=0;
    aux:=prod.marca;
    while (prod.codigo<>-1) and (aux=prod.marca)
    do begin
      totalst:= totalst+prod.stock;
      if (prod.stock=0)
      then writeln (prod.nombre);
      if (digitos5(prod.codigo))
      then writeln (prod.nombre);
      if (prod.precio>40)
      then c40:= c40+1;
      leerProducto(prod)
    end;
    writeln ('Total stock marca ', aux, ' es: ', totalst);
  end;
end.

```

Programación I

Estructura de datos ARREGLO

- 1 Motivación
- 2 Definición de tipo de dato Arreglo
- 3 Declaración del tipo Vector
- 4 Operaciones frecuentes en el tipo Vector
- 5 Ejercitación

Arreglos: Motivación



Se leen 100 productos de un supermercado. Cada producto está caracterizado por código, nombre, marca, stock y precio. Informar los nombres y precios de los productos cuyo precio supera el promedio de precios del supermercado.



1000
Leche
SanCor
100
20



1100
Yoghurt
Sancor
200
23



5055
Detergente
Ala
0
55



2400
Fideos
Matarazzo
35
30



5250
Cerveza
Quilmes
100
50

¿Cómo podemos procesar los datos para obtener el promedio de los precios y compararlo con el precio de cada producto?

Pueden existir diferentes soluciones con los tipos de datos que hemos visto hasta ahora en el curso:

Código
Nombre
Marca
Stock
Precio

Arreglos: Motivación



1000
Leche
SanCor
100
20



1100
Yoghurt
Sancor
200
23



5055
Detergente
Ala
0
55



2400
Fideos
Matarazzo
35
30



5250
Cerveza
Quilmes
100
50

a) Ingresar 2 veces el conjunto de datos.

Se ingresan los datos para calcular el promedio de precios y luego volvemos a ingresar los mismos datos, comparando el precio promedio con el precio de cada producto.

ATENCIÓN!!! Para los 100 productos, donde cada uno tiene 5 datos nos obliga a leer una vez $100 \times 5 = 500$ datos. Y luego otra vez $100 \times 5 = 500$ datos. En total se leen 1000 datos.

Código
Nombre
Marca
Stock
Precio

Arreglos: Motivación



1000
Leche
SanCor
100
20



1100
Yoghurt
Sancor
200
23



5055
Detergente
Ala
0
55



2400
Fideos
Matarazzo
35
30



5250
Cerveza
Quilmes
100
50

b) Usar tantas variables diferentes como productos existen.

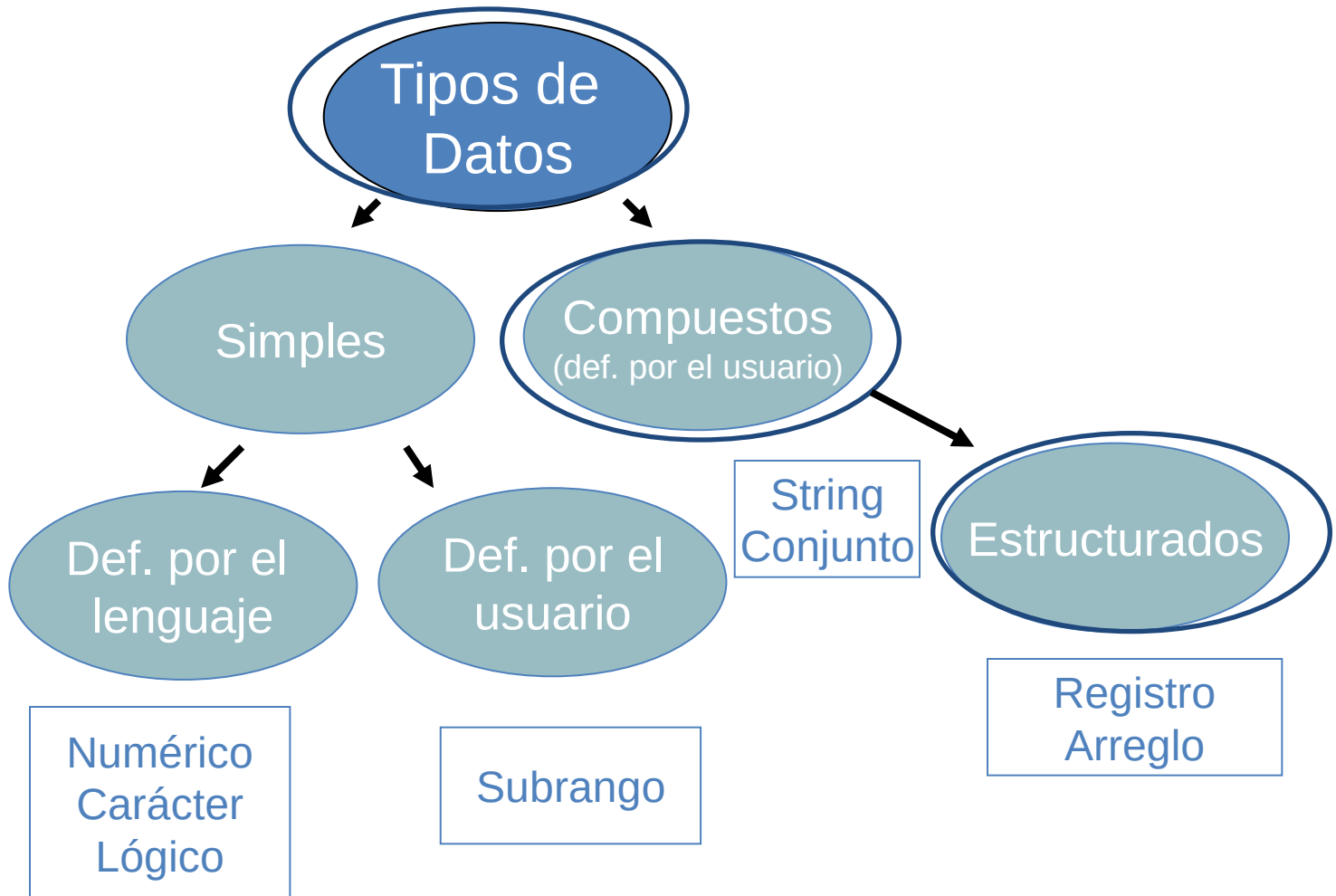
Es decir en cada variable guardamos un producto distinto a medida que se lee (en este caso necesitamos 100 variables diferentes). Luego calculamos el promedio y comparamos cada precio con el mismo.

ATENCIÓN!!! Esta solución resulta más compleja a medida que aumenta el número de productos ¿Por qué?

Arreglos: Motivación

- A partir de la solución (b) resulta claro que sería conveniente tener una estructura que reúna a todos los productos bajo un único nombre y que a la vez permita diferenciar (y acceder) a los datos de cada uno.
- Para resolver estos problemas podemos usar una estructura de datos tipo ARREGLO.
- Un arreglo es una estructura de datos que permite acceder a cada componente a través de índices, que indican la posición de cada componente dentro de la estructura de datos.

Arreglos: Recordemos clasificación...



Arreglos: Definición

- Un tipo de dato **Arreglo** es una colección de elementos que se guardan consecutivamente en la memoria y se pueden referenciar a través de índices.

Esta estructura de datos reúne las siguientes características:

- ✓ Todos los elementos son del mismo tipo de datos, por eso es una estructura de datos **homogénea**.
- ✓ Los elementos o componentes pueden recuperarse en cualquier orden, indicando simplemente su posición, por eso es una estructura de datos de **acceso directo**. Como el acceso se hace a través del índice se la denomina también indexada.
- ✓ La memoria ocupada durante la ejecución del programa es fija, por eso se dice que es una estructura de datos **estática**.
- ✓ Dado que cada elemento tiene un elemento que le precede y uno que le sigue, esta estructura se denomina **lineal**.

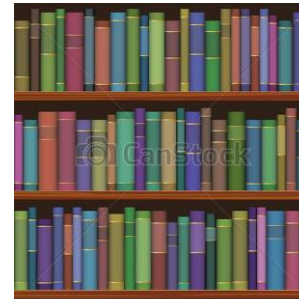
Tipos de arreglos

Existen distintos tipos de arreglos, alguno de ellos son:

- Arreglos unidimensionales: vectores
- Arreglos bidimensionales: matrices



Un Indice: vector



Dos Indices: matriz

Horarios de Clase

Hora	Lunes	Martes	Miércoles	Jueves	Viernes



Tipo Vector

Volviendo al problema inicial, si tenemos los 100 productos del supermercado

Productos



1000
Leche
SanCor
100
20



1100
Yoghurt
Sancor
200
23



5055
Detergente
Ala
0
55



2400
Fideos
Matarazzo
35
30



5250
Cerveza
Quilmes
100
50

los podemos almacenar en un vector llamado PRODUCTOS de esta forma:

1000 Leche SanCor 100 20	1100 Yoghurt Sancor 200 23	5055 Detergente Ala 0 55	2400 Fideos Matarazzo 35 30						5250 Cerveza Quilmes 100 50
1	2	3	4		...				100

Productos

Tipo Vector: Posición y contenido

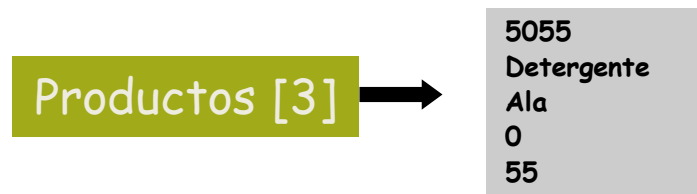
Cuando se trabaja con vectores hay que tener en cuenta que:

Productos

1000 Leche SanCor 100 20	1100 Yoghurt Sancor 200 23	5055 Detergente Ala 0 55	2400 Fideos Matarazzo 35 30						5250 Cerveza Quilmes 100 50
1	2	3							100

Contenido (circled around the third row)

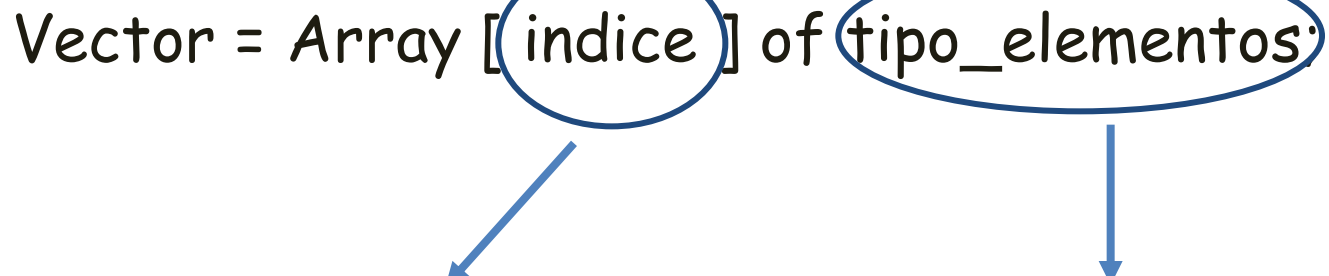
Posición (circled around the number 3 in the second row)



- ➡ El valor 3 es la posición ó ubicación dentro de la estructura de datos (índice).
- ➡ Los datos del registro son el contenido de esa posición o ubicación.

Tipo Vector: Declaración en Pascal

Vector = Array [**índice**] of **tipo_elementos**



Es posible indexar los elementos por un **índice** que corresponde a cualquier tipo ordinal:

- Entero
- Carácter
- Subrango

Los **elementos** de un arreglo pueden pertenecer a cualquier tipo de datos de asignación estática:

- Entero, Real, Lógico, Carácter
- String
- Registros
- Otro arreglo

Tipo Vector: Declaración en Pascal

Type

```
cadena15 = string [15];
```

```
producto= Record
```

```
    codigo: integer;
```

```
    nombre: cadena15;
```

```
    marca: cadena15;
```

```
    stock: integer;
```

```
    precio: real;
```

```
End;
```

```
vectorProductos=array [1..100] of producto;
```

Var

```
Productos : vectorProductos;
```

Se puede decir que:

✓ La variable **Productos** tiene asociada un área de memoria fija consecutiva que es el lugar donde se almacenará la información de los productos.

✓ La variable **Productos** ocupa 100 posiciones de memoria como lo indica su declaración.

Tipo Vector: Declaración en Pascal

```
Const  limite=1000;
```

```
Type
```

```
    periodo = 2000..2015;
```

```
    AñosAutos = array [periodo]  of integer;
```

```
    Cajas = array [ 'A' .. 'D' ] of real;
```

```
    numeros = array [1..limite] of integer;
```

```
    cadena15 = string [15];
```

```
    cadena7 = string [7];
```

```
    auto = Record
```

```
        patente: cadena7;
```

```
        marca: cadena15;
```

```
        modelo: cadena15;
```

```
        precio: real;
```

```
    End;
```

```
    vectorAutos= array [1..50] of auto;
```

```
Var
```

```
    N: numeros;      {1000 elementos enteros}
```

```
    Autos: añosAutos; {16 elementos enteros}
```

```
    concesionaria: vectorAutos; {50 elementos tipo auto}
```

```
    TotalporCaja: cajas; {4 elementos reales}
```

Analicemos para cada variable:

- ✓ ¿Memoria ocupada?
- ✓ ¿Tipo de índice de cada vector?
- ✓ ¿Tipo de los elementos de cada vector?

Tipo vector: operaciones

- Asignación de contenido a un elemento
- Lectura / Escritura
- Recorridos
- Cargar datos en un vector
- Agregar elementos al final
- Insertar elementos
- Borrar elementos
- Buscar un elemento

Operación: Asignación de contenido a un elemento del vector

Const

```
    limite=1000;
```

Type

```
    numeros = array [1..limite] of integer;
```

```
    cadena15 = string [15];
```

```
    producto= Record
```

```
        codigo: integer;
```

```
        nombre: cadena15;
```

```
        marca: cadena15;
```

```
        stock: integer;
```

```
        precio: real;
```

```
    End;
```

```
    vectorProductos = array [1..100] of producto;
```

```
    cajas = array [ 'A' .. 'D' ] of real;
```

Var

```
    productos : vectorProductos; N: números;
```

```
    i : integer; totalPorCaja: cajas;
```

Begin

```
    totalPorCaja['C'] := 2500,50;
```

```
    N [5] := 500;
```

```
    N [1] := N [5] * 2;
```

```
    productos[1].precio:= 100;
```

```
    productos[3].precio:= productos[1].precio + 10;
```

```
    productos[5].precio:= productos[1].precio + productos[3].precio;
```

```
    i:= 10;
```

```
    productos[i].stock:= 50;
```

```
    . . .
```

End.

Operación: Asignación de contenido a un elemento del vector

Const

```
    limite=1000;
```

Type

```
    numeros = array [1..limite] of integer;
```

```
    cadena15 = string [15];
```

```
    producto= Record
```

```
        codigo: integer;
```

```
        nombre: cadena15;
```

```
        marca: cadena15;
```

```
        stock: integer;
```

```
        precio: real;
```

```
    End;
```

```
    vectorProductos = array [1..100] of producto;
```

Var

```
    N: números; i:integer;
```

```
    Productos : vectorProductos;
```

Begin

```
    N [6]:= 'a';
```

```
    N ['b'] := 12;
```

```
    i:= 1;
```

```
    productos[120].precio := 25.5;
```

```
    productos[i].stock := 23.5;
```

```
    . . .
```

```
End.
```

N [6] := 'a'
¿Es válida? ¿Por qué?

N ['b'] := 12
¿Es válida? ¿Por qué?

productos[120].precio := 25.5
¿Es válida? ¿Por qué?

productos[i].stock := 23.5
¿Es válida? ¿Por qué?

Const

```
    limite=1000;
```

Type

```
    índice = 1..limite;
    numeros = array [índice] of integer;
    cadena15 = string [15];
    producto= Record
```

```
        código: integer;
        nombre: cadena15;
        marca: cadena15;
        stock: integer;
        precio: real;
```

End;

```
    vectorProductos=array [1..100] of producto;
```

```
Procedure LeerProducto (Var prod: producto);
begin
```

```
    Readln (prod.codigo);
    Readln (prod.nombre);
    Readln (prod.marca);
    Readln (prod.stock);
    Readln (prod.precio);
```

```
end;
```

```
Procedure MostrarProducto (prod: producto);
begin
```

```
    Writeln (prod.codigo);
    Writeln (prod.nombre);
    Writeln (prod.marca);
    Writeln (prod.stock);
    Writeln (prod.precio);
```

```
end;
```

Cada componente del vector se trabaja individualmente.

Var

```
    N:números;
```

```
    Productos : vectorProductos;
```

```
    i : integer; k:índice;
```

Begin

```
    readln (N[1]);
```

```
    writeln (N[1]);
```

```
    For i:= 1 to 100 do
```

```
        LeerProducto (Productos[i]);
```

```
    For i:= 1 to 100 do
```

```
        MostrarProducto (Productos[i]);
```

```
    . . .
```

```
End.
```

Tipo Vector : Operación de Recorrido

La operación de **Recorrido** en un vector consiste en recorrer el vector de manera **total** o **parcial**, para realizar algún proceso sobre sus elementos.

La operación de **Recorrido Total**, implica analizar **todos** los elementos del vector, lo que lleva a recorrer completamente la estructura.

La operación de **Recorrido Parcial**, implica analizar los elementos del vector, **hasta** encontrar aquel que cumple con lo pedido. Puede ocurrir que se recorra todo el vector.

¿Qué estructuras de control conviene utilizar en cada caso?

Tipo Vector : Ejemplo de Recorrido Total



Por ejemplo, si se necesita conocer la cantidad total del stock de productos del supermercado, habrá que realizar un recorrido total que acumule los stocks de cada producto.

Estructura de control

Type

```
cadena15 = string [15];
```

```
producto= Record
```

```
    codigo: integer;
```

```
    nombre: cadena15;
```

```
    marca: cadena15;
```

```
    stock: integer;
```

```
    precio: real;
```

```
End;
```

```
vectorProductos=array [1..100] of producto;
```

Var

```
productos: vectorProductos;
```

```
st_total:integer;
```

```
i: integer;
```

begin

```
{el vector contiene datos de 100 productos}
```

```
st_total:= 0;
```

```
{recorrido total del vector}
```

```
For i := 1 to 100 do
```

```
    st_total := st_total + productos[i].stock;
```

```
writeln ( 'El stok total es: ', st_total);
```

```
end.
```


Tipo Vector : Ejemplo de Recorrido parcial



Por ejemplo, se quiere conocer el nombre del primer producto con stock en 0, (seguro existe). Tendremos que recorrer el vector de productos hasta encontrar el primer producto con stock en 0.

*Estructura
de control*

```
Program    stock0;
Type
  cadena15 = string [15];
  producto= Record
    codigo: integer;
    nombre: cadena15;
    marca: cadena15;
    stock: integer;
    precio: real;
  End;
  vectorProductos = array [1..100] of producto;
Var
  productos: vectorProductos; i:integer;
begin
  {el vector contiene datos de 100 productos}
  i := 1;
  while (productos[i].stock <> 0) do
    i:= i+1;
  writeln('Producto: ', productos[i].nombre);
end.
```

Vector : Ejemplo de Recorrido parcial



¿Qué ocurre si en el ejemplo anterior se cambia la precondition y el producto con stock en 0 podría no existir?

Se debe reemplazar por:

Analizar condición al salir:

```
Program    stock0;
```

```
Type
```

```
cadena15 = string [15];
```

```
producto= Record
```

```
    codigo: integer;
```

```
    nombre: cadena15;
```

```
    marca: cadena15;
```

```
    stock: integer;
```

```
    precio: real;
```

```
End;
```

```
vectorProductos = array [1..100] of producto;
```

```
Var
```

```
    productos: vectorProductos; i:integer;
```

```
begin
```

```
    {el vector contiene datos de 100 productos}
```

```
    i := 1;
```

```
    While ( i <= 100) and ( productos[i].stock<>0) do
```

```
        i:= i+1;
```

```
    If i<=100 then writeln ('Producto:',  productos[i].nombre)
```

```
        else writeln ('No existe');
```

```
End:
```

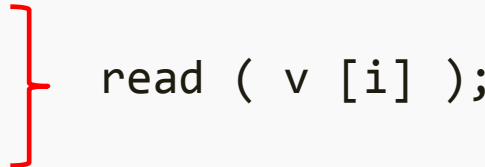
Analicemos la condición del While...

Tipo Vector : Operación de Carga Completa

La operación de **Carga Completa** en un vector consiste en guardar un elemento en cada posición del vector.

Por ejemplo, implementar un programa que cargue un vector con 50 números enteros que se leen.

```
Program cargarVector;  
Const  
    fin = 50;  
Type  
    vectorNúmeros = array [1..fin] of integer;  
Var  
    i, num: integer;  
    v: vectorNúmeros;  
begin  
    For i:= 1 to fin do  
        begin  
            read (num);  
            v [i] := num;  
        end;  
    end.  
end.
```



Tipo Vector: Ejercitación



1. Se leen 100 productos de un supermercado. Cada producto está caracterizado por código, nombre, marca, stock y precio. Informar los nombres y precios de los productos cuyo precio supera el promedio de precios del supermercado.

- Leer, Guardar y Sumar todos los precios
- Calcular promedio
- Recorrer y Comparar cada precio con el promedio

Leer, Guardar y Sumar todos los precios

Inicializar suma de precios

Repetir 100

leer datos del producto

guardar datos del producto

actualizar suma de precios

Recorrer y Comparar cada nota con el promedio

Repetir 100

acceder a los datos del producto

si $\text{precio} > \text{promedio}$ entonces

informar nombre y precio



Se leen 100 productos de un supermercado. Cada producto está caracterizado por código, nombre, marca, stock y precio. Informar los nombres y precios de los productos cuyo precio supera el promedio de precios del supermercado.

Program ejercitación;

const total= 100;

type

cadena15 = string [15];

producto= Record

codigo: integer;

nombre: cadena15;

marca: cadena15;

stock: integer;

precio: real;

End;

vectorProductos = array [1..100] of producto;

{implementación LeerGuardarSumar}

{implementación RecorreryComparar}

var

super: vectorProductos;

suma, promedio : Real;

Begin

LeerGuardarSumar (super, suma);

promedio := suma/total;

RecorreryComparar (super, promedio);

End.

Leer, Guardar y Sumar todos los precios

Inicializar suma de precios

Repetir 100

leer datos de producto

guardar datos del producto

actualizar suma de precios

```
const
    total= 100;
type
    cadena15 = string [15];
    producto= Record
        codigo: integer;
        nombre: cadena15;
        marca: cadena15;
        stock: integer;
        precio: real;
    End;
vectorProductos=array [1..100]
    of producto;
```

```
Procedure LeerGuardarSumar(var v:vectorProductos;
                           var sum: Real);
Procedure leerProducto (var p:producto);
begin
    read (p.codigo);
    read (p.nombre);
    read (p.marca);
    read (p.stock);
    read (p.precio);
end;
var
    j : integer; prod:producto;
begin
    sum := 0;
    for j := 1 to total do begin
        leerProducto (prod);
        v [j]:= prod;
        sum := sum + v [j].precio;
    end
end;
```

*Recorrer y Comparar cada precio
con el promedio*

Repetir 100

acceder a los datos del producto

si precio > promedio entonces

informar nombre y precio

```
const
    total= 100;
type
    cadena15 = string [15];
    producto= Record
        codigo: integer;
        nombre: cadena15;
        marca: cadena15;
        stock: integer;
        precio: real;
    End;
vectorProductos=array [1..100]of producto;
```

```
Procedure RecorreryComparar (v: vectorProductos; prom: real);
    var i: integer;
begin
    for i := 1 to total do
        if (v [i].precio > prom) then
            Writeln (v [i].nombre, ' ', v [i].precio);
End.
```

ESTRUCTURA DE DATOS VECTOR – Para resolver en clase



2. Implementar un programa que lea una secuencia de caracteres que termina en punto. Se debe informar la cantidad de veces que aparece cada letra minúscula en la secuencia.

3. Implementar un programa que lea 200 artículos de una juguetería. De cada artículo se lee: código, descripción, fecha de fabricación, edad recomendada y precio. Informar la cantidad de artículos que superan el promedio de precios.

Programación I

Estructura de datos VECTOR

- 1 Operaciones frecuentes en el tipo Vector
 - Cargar elementos
 - Agregar un elemento al final
 - Insertar un elemento en una posición determinada
 - Borrar un elemento de una posición determinada
- 2 Ejercitación

Tipo Vector: Dimensiones Física y lógica

Cuando trabajamos con vectores se deben considerar:

➡ **Dimensión Física del vector**

Se especifica en el momento de la declaración y determina su ocupación de memoria. La cantidad de memoria no variará durante la ejecución del programa.

➡ **Dimensión Lógica del vector**

Se determina cuando se cargan elementos al vector. Indica la cantidad de posiciones de memoria ocupadas con contenidos cargados desde la posición inicial en forma consecutiva.

Tipo Vector: Carga de datos

La operación de Cargar datos en un vector consiste en incorporar un elemento a continuación del otro desde la posición inicial en forma consecutiva.

Esta operación debe controlar que la cantidad de elementos que se cargan no supere la dimensión física.

CONSIDERACIONES

- ➡ **dimL** : cantidad de elementos en el vector (dimensión lógica).
- ➡ **dimF** : dimensión física del vector (tamaño especificado en la declaración del vector)
- ➡ **Espacio suficiente**: Debe verificarse que $\text{dimL} < \text{dimF}$ (dimensión lógica < dimensión física)
- ➡ **Condición de corte**: El ingreso de datos debe tener un fin

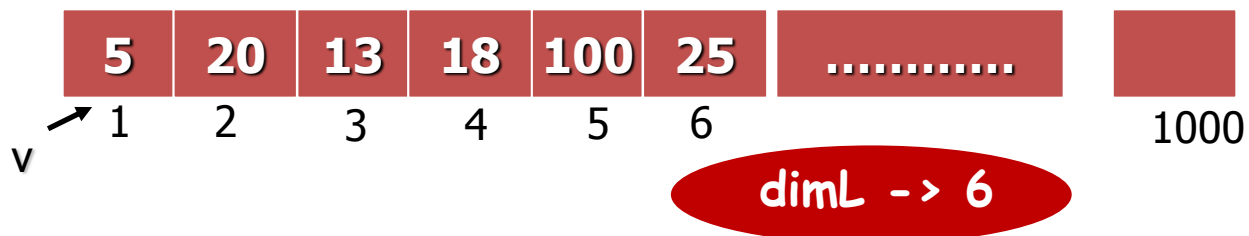
Tipo Vector: Carga de datos

Implementar un módulo que lea números enteros y los cargue en un vector de a lo sumo 1000 elementos. La lectura finaliza con el valor 99:

```
Const dimF = 1000;  
Type vector = Array [ 1..dimF] of integer;  
Procedure CARGAR ( var v: vector; var dimL: integer );  
  var dato: integer;  
  begin  
    dimL := 0;  
    read (dato);  
    while (dato <> 99) and ( dimL < dimF ) do begin  
      dimL := dimL + 1;  
      v [dimL] := dato;  
      read (dato);  
    end;  
  End;
```

¿Qué retorna el procedimiento?

Si se leen los valores: 5, 20, 13, 18, 100, 25 y 99, obtendremos:



Tipo Vector: Agregar un elemento al final

La operación de Agregar un elemento en un vector consiste en incorporar el elemento a continuación del último ingresado, es decir, en la posición siguiente a la indicada en la dimensión lógica.

Esta operación debe verificar que haya lugar en la estructura, es decir que la dimensión lógica sea menor que la dimensión física.

CONSIDERACIONES

- ➡ **dimL** : cantidad de elementos en el vector (dimensión lógica).
- ➡ **dimF** : dimensión física del vector (tamaño especificado en la declaración del vector)
- ➡ **Espacio suficiente**: Debe verificarse que $\text{dimL} < \text{dimF}$ (dimensión lógica < dimensión física)
- ➡ El elemento a **agregar** ocupa la **posición dimL+1**.
- ➡ Luego de la operación la **cantidad** de elementos es **dimL+1**.

Tipo Vector: Agregar un elemento al final

Const

```
dimF = 1000;
```

Type

```
vector = Array [ 1..dimF] of integer;
```

```
Procedure AGREGAR (var v: vector; var dimL: integer;  
                  elemento: integer; var éxito: boolean);
```

Begin

```
éxito:= false;
```

```
{verificar espacio suficiente}
```

```
If (dimL < dimF) then begin
```

```
    éxito:= true;
```

```
    dimL:= dimL+1; {actualizar cantidad de elementos}
```

```
    v [dimL]:= elemento;
```

```
end;
```

```
end;
```

Tipo Vector: Insertar un elemento

La operación de Insertar un elemento en un vector consiste en incorporar el elemento en una posición determinada o de acuerdo a un orden impuesto a sus datos.

Esta operación también tiene que verificar que haya lugar en la estructura, es decir que la dimensión lógica sea menor que la dimensión física.

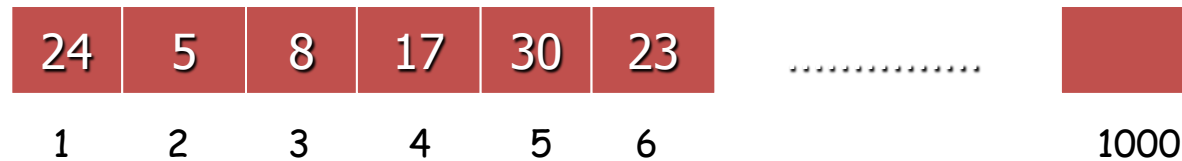
CONSIDERACIONES

- ➡ **dimL** : cantidad de elementos en el vector (dimensión lógica).
- ➡ **dimF** : dimensión física del vector (tamaño especificado en la declaración del vector)
- ➡ **Espacio suficiente**: Debe verificarse que $\text{dimL} < \text{dimF}$ (dimensión lógica < dimensión física)
- ➡ Luego de la operación la **cantidad** de elementos es **dimL+1**.

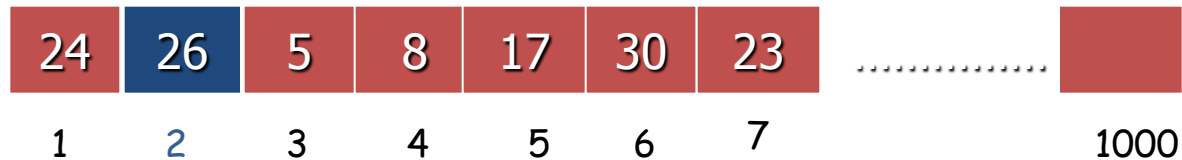
Tipo Vector: Insertar un elemento

Cuando se requiere insertar un elemento en un vector se presentan dos posibilidades distintas:

1 Insertar un elemento en una posición determinada



Posición 2



Elemento 26

2 Insertar un elemento manteniendo un orden predeterminado

Lo veremos más adelante

Tipo Vector: Insertar un elemento en una posición determinada

➡ Esta operación también tiene que verificar que la posición sea válida.

1. Verificar la posición a insertar

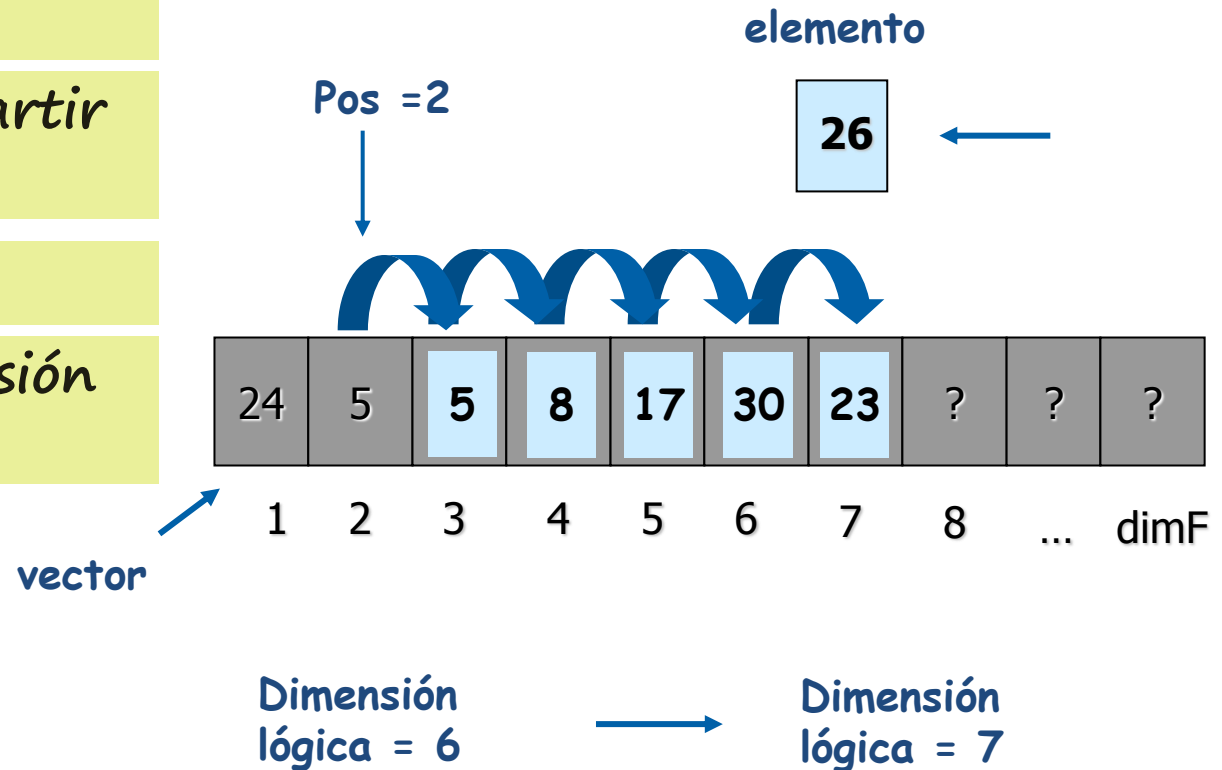
2. Verificar espacio en el vector

3. Abrir el vector (a partir de la dimensión lógica)

4. Asignar el valor

5. Aumentar la dimensión lógica

Supongamos que se quiere insertar el valor 26 en la posición 2 de un vector de valores enteros...



Tipo Vector: Insertar un elemento en una posición determinada

1. Verificar la posición a insertar

2. Verificar espacio en el vector

3. Abrir el vector (a partir de la dimensión lógica)

4. Asignar el valor

5. Aumentar la dimensión lógica

Parámetros de la operación:

- ✓ v: vector a trabajar
- ✓ dimL: cantidad de elementos
- ✓ elemento: dato a insertar
- ✓ pos: posición donde insertar
- ✓ exito: resultado operación

```
Procedure INSERTARPOS (var v:vector; var dimL: integer;  
    elemento: integer; pos: integer; var exito: boolean);
```

```
var i : integer;
```

```
Begin
```

```
    exito:= false; Verificar espacio y posición válida
```

```
    if (dimL < dimF) and ((pos>=1) and (pos<= dimL))
```

```
        then begin
```

```
            exito:= true;
```

```
            for i:= dimL downto pos do
```

```
                v [ i + 1 ] := v [ i ] ;
```

```
                v [pos] := elemento; Asignar el valor
```

```
                dimL := dimL + 1; Aumentar la dimensión
```

```
            end;
```

```
end;
```

Abrir el arreglo

Tipo Vector: Borrar un elemento

La operación de Borrar un elemento en un vector consiste en eliminar un elemento determinado o bien eliminar un elemento que ocupa una posición determinada. En el caso de borrar un elemento en una posición determinada, se debe verificar que la posición sea válida. En el caso de eliminar un elemento determinado, hay que verificar que exista dicho elemento.

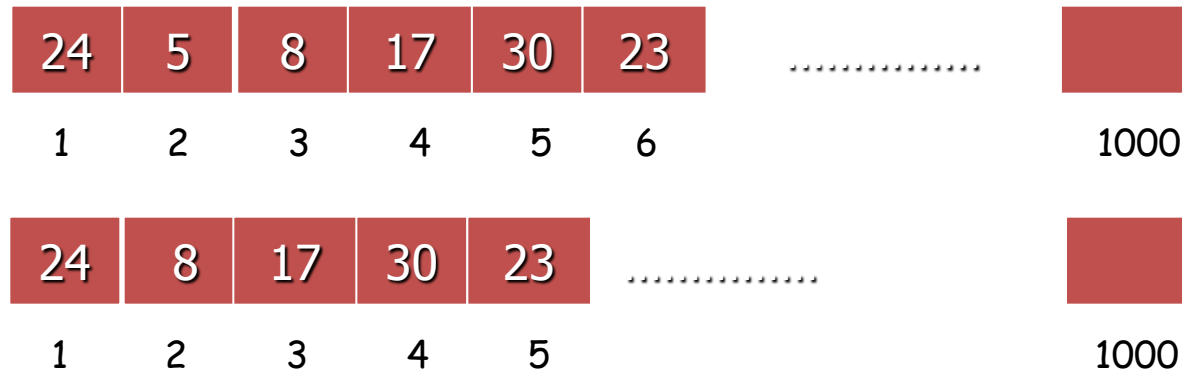
CONSIDERACIONES

- ➡ **dimL** : cantidad de elementos en el vector (dimensión lógica).
- ➡ Luego de la operación la **cantidad** de elementos es **dimL-1**.

Tipo Vector: Borrar un elemento

Cuando se requiere borrar un elemento en un vector se presentan dos posibilidades distintas:

1 Borrar un elemento de una posición determinada



2 Borrar un elemento determinado del vector

*Lo veremos
más adelante*

Tipo Vector: Borrar un elemento de una posición determinada

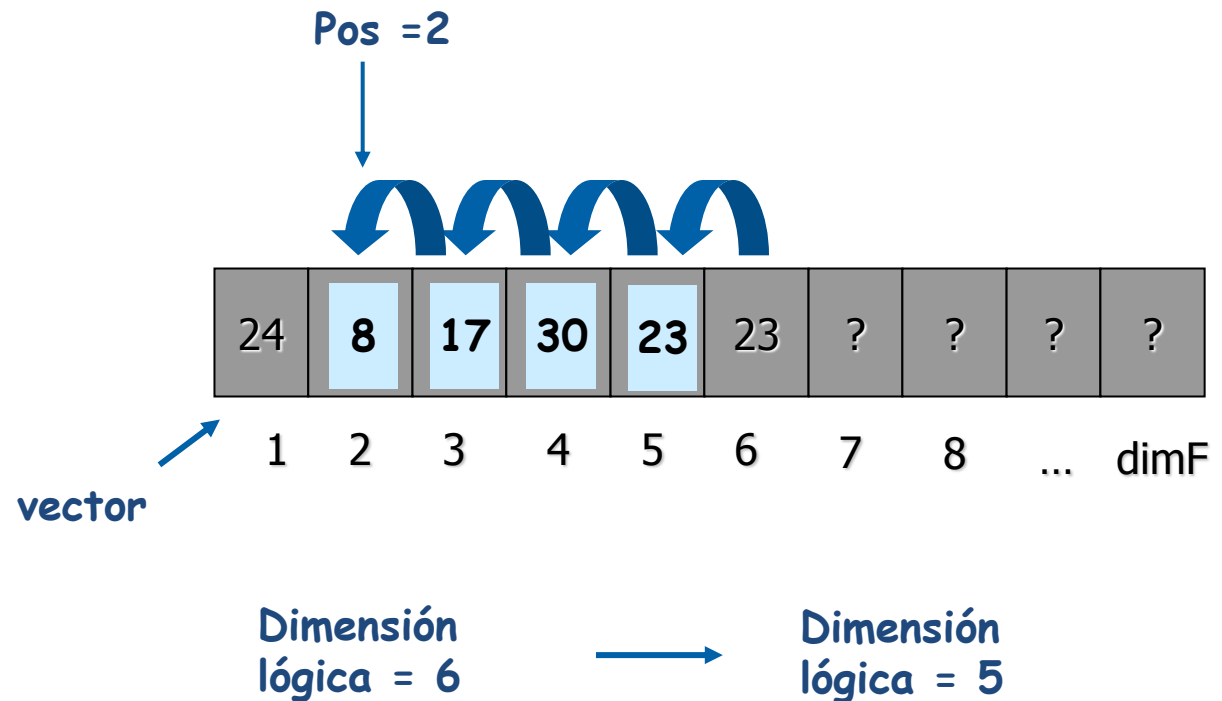
➡ Esta operación también tiene que verificar que la posición sea válida.

1. Validar la posición a borrar

2. Desplazar elementos (a partir de la posición siguiente)

3. Disminuir la dimensión lógica

Supongamos que se quiere borrar el elemento de la posición 2 de un vector de valores enteros...



Tipo Vector: Borrar un elemento de una posición determinada

1. Validar la posición a borrar

2. Desplazar elementos (a partir de la posición siguiente)

3. Disminuir la dimensión lógica

Parámetros de la operación:

- ✓ v: vector a trabajar
- ✓ dimL: cantidad de elementos
- ✓ pos: posición a borrar
- ✓ éxito: resultado operación

```
Procedure BorrarPos (var v: vector;  
                    var dimL: integer; pos: posicion;  
                    var éxito: boolean );  
  
var i: integer;  
begin  
    éxito := false; Validar la posición a borrar  
    if (pos >= 1 and pos <= dimL)  
        then begin  
            éxito := true;  
  
            Desplazar elementos  
            for i:= pos + 1 to dimL do  
                v [ i - 1 ] := v [ i ] ;  
  
            dimL := dimL - 1 ; Disminuir la dimensión lógica  
        end;  
end;
```



Ejercitación

1. Se lee una sucesión nombres y notas de alumnos que finaliza con nombre "Fulano". Informar los nombres y notas de los alumnos que superan el promedio del grupo.

Nota: Como máximo se pueden ingresar 100 alumnos.

- Leer, Guardar y Sumar todas las notas
- Calcular promedio
- Recorrer y Comparar cada nota con el promedio

Leer, Guardar y Sumar todas las notas

Inicializar suma de notas

Leer datos de alumno

Mientras (haya lugar) y (no ingrese "Fulano")

guardar en el vector

actualizar suma de notas

leer datos de alumno

Recorrer y Comparar cada nota con el promedio

Repetir cantidad alumnos guardada

acceder a los datos del alumno

si nota > promedio entonces

informar nombre



3. Se lee una sucesión nombres y notas de alumnos que finaliza con nombre "Fulano". Informar los nombres y notas de los alumnos que superan el promedio del grupo.

Nota: como máximo se pueden ingresar 100 alumnos.

- *Leer, Guardar y Sumar todas las notas*
- *Calcular promedio*
- *Recorrer y Comparar cada nota con el promedio*

Program ejemplo2;

const dimF= 100; {dimensión física}

type

str20 := string [20];

alumno = record

nombre: str20;

nota: real;

end;

rango = 0.. dimF;

vectorDatos = array [1..dimF] of alumno;

{implementación LeerGuardarSumar}

{implementación Recorrer y Comparar}

var

datos: vectorDatos; dimL: rango;

suma, promedio : Real;

Begin

LeerGuardarSumar(datos, dimL, suma);

if dim <>0

then begin

promedio := suma/dimL;

Recorrer y Comparar(datos, dimL, promedio)

end;

End.

Leer, Guardar y Sumar todas las notas

Inicializar suma de notas

Leer datos de alumno

Mientras (haya lugar) y (no ingrese "Fulano")

guardar en el vector

actualizar suma de notas

leer datos de alumno

```
const dimF= 100; {dimensión física}
type
  str20 := string [20];
  alumno = record
      nombre: str20;
      nota: real;
  end;
rango = 0.. dimF;
vectorDatos=array[1.. dimF]of alumno;
```

```
procedure LeerGuardarSumar
  (var dat:vectorDatos; var dimL: rango;
   var sum: Real);

  Procedure leerAlumno (var alu:alumno);
  begin
    read (alu.nombre);
    if (alu.nombre <> 'Fulano')
      then read (alu.nota);
    end;

  var
    a : alumno;

  begin
    sum := 0;
    dimL := 0; {dimensión lógica}
    leerAlumno (a);
    while (dimL < dimF) and (a.nombre <> 'Fulano')
      do begin
        dimL := dimL + 1;
        dat[dimL]:= a;
        sum := sum + dat[dimL].nota;
        leerAlumno (a);
      end
    end;
end;
```

Recorrer y Comparar cada nota con el promedio

*Repetir cantidad alumnos guardada
acceder a los datos del alumno
si nota > promedio entonces
informar nombre*

```
const dimF = 100; {dimensión física}
type
  str20 := string [20];
  alumno = record
    nombre: str20;
    nota: real;
  end;
rango = 0.. dimF;
vectorDatos=array[1.. dimF]of alumno;
```

```
Procedure RecorreryComparar (dat: vectorDatos; dimL:rango; prom: real);
  var i: integer;
begin
  for i := 1 to dimL do
    if (dat[i].nota > prom) then
      Writeln (dat[i].nombre, ' ', dat[i].nota );
    end;
  end;
End.
```

ESTRUCTURA DE DATOS VECTOR



Un centro de deportes nos ha encargado un sistema para el manejo de sus clientes. En el centro se ofrecen 5 actividades distintas. De cada cliente se conoce código de cliente, DNI, apellido, nombre, edad, el número de actividad que realiza, mes y año del último pago. La lectura finaliza cuando llega el DNI 0.

Se pide:

- a) Almacenar el precio mensual de las actividades a través de la lectura de los valores.
- b) Generar la estructura de datos que almacene los clientes del centro de deportes (como máximo 500), leyendo los datos de los clientes hasta ingresar un DNI -1.
- c) Informar la cantidad de clientes que no tienen la cuota al día.
- d) Eliminar de la estructura el cliente de una posición que se lee.

Programación I

Estructura de datos VECTOR

1

Operaciones frecuentes en el tipo Vector

- Búsqueda
- Borrar un elemento determinado
- Insertar un elemento en un vector con orden

2

Ejercitación

Tipo vector: Operación de Búsqueda

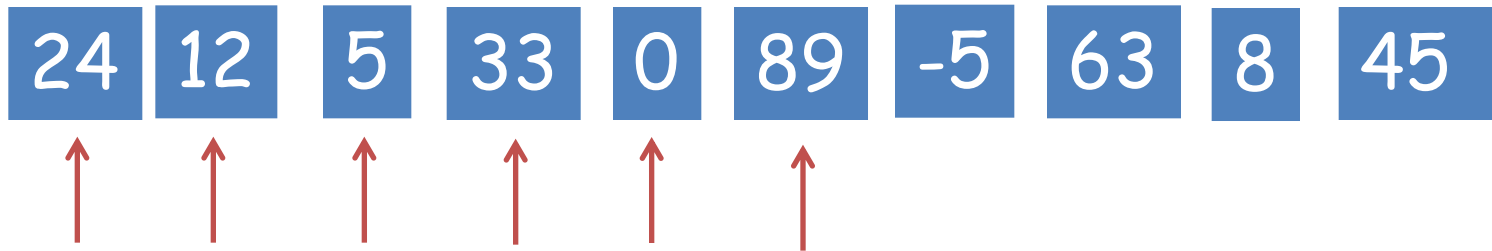
El proceso de ubicar información particular en una colección de datos es conocido como método de búsqueda.

Se deben considerar los siguientes casos:

- ➡ Los datos en el vector están almacenados sin ningún orden.
- ➡ Los datos en el vector están almacenados ordenados por algún criterio

Tipo Vector: Búsqueda Lineal o secuencial (elementos sin orden)

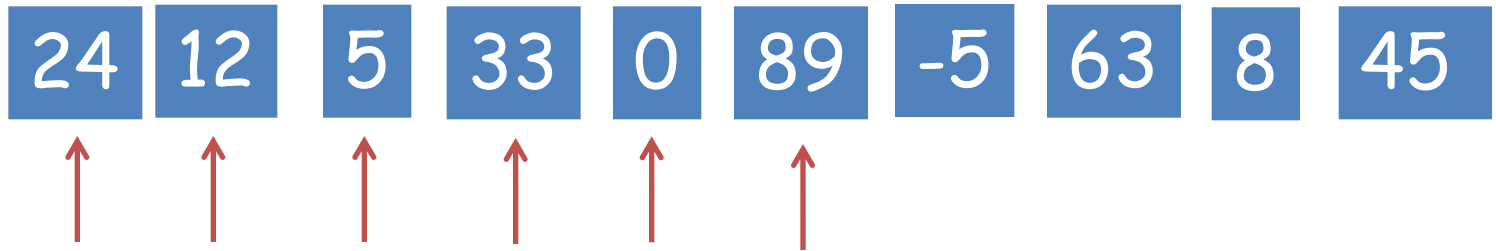
X= 89



- La búsqueda comienza desde el principio y se avanza por la estructura de manera secuencial, uno a uno.
- La solución debería recorrer el vector y detenerse en caso de encontrar el elemento X.

Tipo Vector: Búsqueda Lineal o secuencial (elementos sin orden)

X= 89



Buscar (Recibe el vector donde buscar, el elemento a buscar, la dimensión lógica y devuelve la posición donde se encontró)

Ubicarse al principio del vector

Mientras (no llegue al final del vector) y (no encuentre el elemento)
avanzar una posición en el vector

Al salir del mientras se debe evaluar por cual de las condiciones finalizó

Operación de Búsqueda Lineal o secuencial (elementos sin orden)

Consideremos la siguiente declaración genérica:

Const

DimF = ... *{máxima longitud del vector}*

Type

TipoElem = ... *{tipo de datos del vector }*

Indice = 0.. DimF;

vector = **Array** [1..DimF] **of** TipoElem;

entos sin orden)

Buscar (Recibe el vector donde buscar, el elemento a buscar,
la dimensión lógica y devuelve la posición donde se encontró)
Ubicarse al principio del vector
Mientras (no llegue al final del vector) y (no encuentre el elemento)
avanzar una posición en el vector
Al salir del mientras se debe evaluar por cual de las condiciones finalizó

Const

DimF = ...

Type

Indice = 0..DimF;

vector= **Array** [1..DimF] of integer;

*¿Qué tipo de módulo
Conviene utilizar?*

Function BuscarPosElem (x:integer; v:vector; dimL: Indice): Indice;

var pos:Indice; exito: boolean;

Begin

pos:=1;

exito:= false;

while (pos <= dimL) and (not exito) do

if (x = v[pos]) then exito:= true

else pos:=pos+1;

if (exito = false) then pos:=0;

BuscarPosElem := pos;

end;

*¿Qué valor toma pos cuando
el elemento no está?*

Características de la Búsqueda Lineal o Secuencial

- Se aplica cuando los elementos no tienen orden.
-

- Requiere excesivo consumo de tiempo en la localización del elemento.
-

- Número medio de comparaciones $(\text{dimL} + 1) / 2$
-

- Es ineficiente a medida que el tamaño del arreglo crece.

Tipo Vector: Borrar un elemento

Recordemos que la operación de Borrar un elemento en un vector admite dos posibilidades:

1 Borrar un elemento de una posición determinada

ya lo vimos

2 Borrar un elemento determinado del vector

Tipo Vector: Borrar un elemento determinado

➡ Esta operación requiere primero buscar el elemento y luego borrarlo

Buscar la posición del elemento a borrar

Si el elemento está entonces

Borrar el elemento

```
Const
    DimF = ...
Type
    Indice = 0.. DimF;
    vector = Array [ 1..DimF] of integer;
```

```
Procedure BorrarElem (var v: vector; var dimL: indice;
                      elem : integer; var exito: boolean);
var pos: indice;

begin
    exito:= false;
    pos:= BuscarPosElem (elem, v, dimL);
    if pos <> 0 then begin
        BorrarPosModif (v, dimL, pos);
        exito:= true;
    end;
end;
```

Ya lo vimos!!

*Revisar BorrarElemPos
y adaptarlo!!*

```
Procedure BorrarElem (var v: vector; var dimL: indice;  
                      elem : integer; var exito: boolean);
```

```
Function BuscarPosElem (x:integer;v:vector;dimL: Indice) : Indice;  
var pos:Indice; exito: boolean;  
Begin  
  pos:=1; exito:= false;  
  while (pos <= dimL) and (not exito) do  
    if (x = v[pos]) then exito:= true  
      else pos:=pos+1;  
  if (exitto = false) then pos:=0;  
  BuscarPosElem := pos;  
end;
```

Ya lo vimos!!

```
Procedure BorrarPosModif (var v:vector; var dimL:integer; pos:Indice);  
begin  
  
end;
```

```
var pos: indice;  
Begin  
  exito:= false;  
  pos:= BuscarPosElem (elem, v, dimL);  
  if pos <> 0 then begin  
    BorrarPosModif (v, dimL, pos);  
    exito:= true;  
  end;  
end;
```

Tipo Vector: Borrar un elemento determinado

```
Procedure BorrarPos (var v:vector; var dimL:integer; pos:Indice;  
                    var exito:boolean);  
  
var i: integer;  
begin  
    exito := false;  
    if (pos >=1 and pos <= dimL)  
    then begin  
        exito := true  
        for j:= pos + 1 to dimL do  
            v [ i - 1 ] := v [ i ] ;  
        dimL := dimL - 1 ;  
    end;  
end;
```

Ya lo vimos!!

```
Procedure BorrarPosModif (var v:vector; var dimL:integer; pos:Indice);  
var i: integer;  
begin  
    for i:= pos + 1 to dimL do  
        v [ i - 1 ] := v [ i ] ;  
    dimL := dimL - 1 ;  
end;
```

*BorrarPos
adaptado*


```

Procedure BorrarElem (var v: vector; var dimL: indice;
                      elem : integer; var exito: boolean);

  Function BuscarPosElem (x:integer;v:vector;dimL: Indice) : Indice;
  var pos:Indice; exito: boolean;
  Begin
    pos:=1; exito:= false;
    while (pos <= dimL) and (not exito) do
      if (x = v[pos]) then exito:= true
      else pos:=pos+1;
    if (exito = false) then pos:=0;
    BuscarPosElem := pos;
  end;

  Procedure BorrarPosModif (var v:vector; var dimL:integer; pos:Indice);
  var i: integer;
  begin
    for i:= pos + 1 to dimL do
      v [ i - 1 ] := v [ i ] ;
    dimL := dimL - 1 ;
  end;

  var pos: indice;
  Begin
    exito:= False;
    pos:= BuscarPosElem (elem, v, dimL);
    if pos <> 0 then begin
      BorrarPosModif (v, dimL, pos);
      exito:= true;
    end;

  end;

```

Métodos de Búsqueda



```
graph TD; A[Métodos de Búsqueda] --> B[Método 1  
Secuencial optimizado:]; A --> C[Método 2  
Binaria o Dicotómica:];
```

Método 1

Secuencial optimizado:

Se recorre el vector hasta encontrar el número buscado o hasta encontrar uno mayor que él.

Método 2

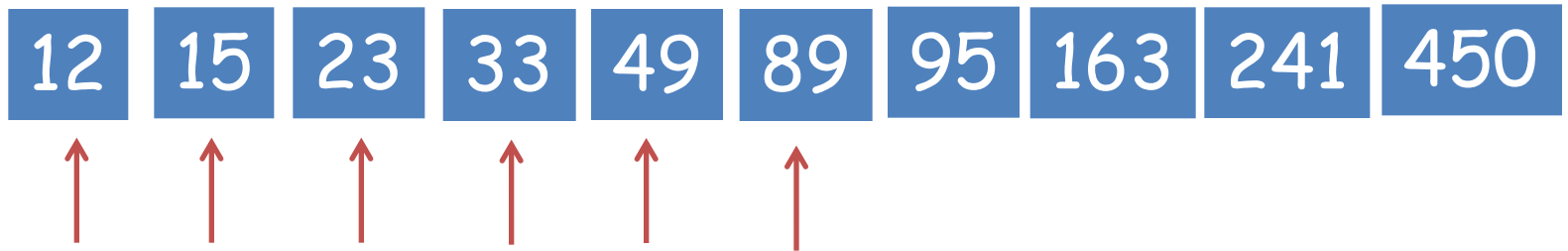
Binaria o Dicotómica:

Acceder a los elementos del vector de una manera mas “eficiente”...

Tipo vector: Búsqueda en Arreglos Ordenados

Método 1: Secuencial Optimizado

X= 89

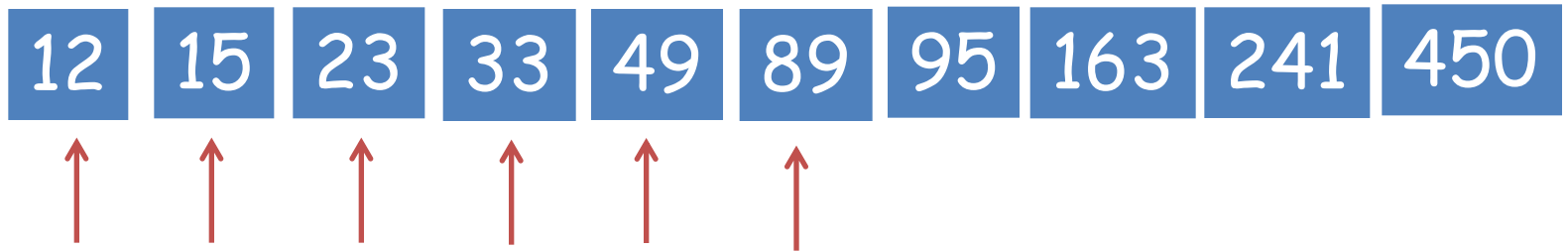


- Se aplica cuando los elementos tienen orden.
- La búsqueda comienza desde el principio y se avanza por la estructura de manera secuencial y de a uno hasta que encuentro el número buscado o hasta que encuentro uno mayor.

Tipo vector: Búsqueda en Arreglos Ordenados

Método 1: Secuencial Optimizado

X= 89



Módulo Buscar (el elemento a buscar, el vector donde buscar,
la dimensión lógica y devuelve la posición donde se encontró)

Ubicarse al principio del vector

Mientras (no llegue al final del vector) y
(el elemento a buscar sea mayor que el elemento observado)
avanzar una posición en el vector

Al salir del mientras se debe evaluar por cual de las condiciones finalizó.

Tipo vector: Búsqueda en Arreglos Ordenados

Módulo Buscar (el elemento a buscar, el vector donde buscar,
la dimensión lógica y devuelve la posición donde se encontró)

Ubicarse al principio del vector

Mientras (no llegue al final del vector) y
(el elemento a buscar sea mayor que el elemento observado)
avanzar una posición en el vector

Al salir del mientras se debe evaluar por cual de las condiciones finalizó

Const

DimF=...

Type

Indice = 0.. DimF;

vector = **Array** [1..DimF] **of** integer;

*Vector
ordenado de
menor a mayor*

Function BuscoPosElemOrd (x: integer; v:Vector; dimL: Indice): Indice;

var pos : Indice;

begin

pos:=1;

while (pos <= dimL) **and** (x > v[pos]) **do**

pos:=pos+1;

if (pos > dimL) **or** (x < v [pos]) **then** pos:=0;

BuscoPosElemOrd:= pos;

end;

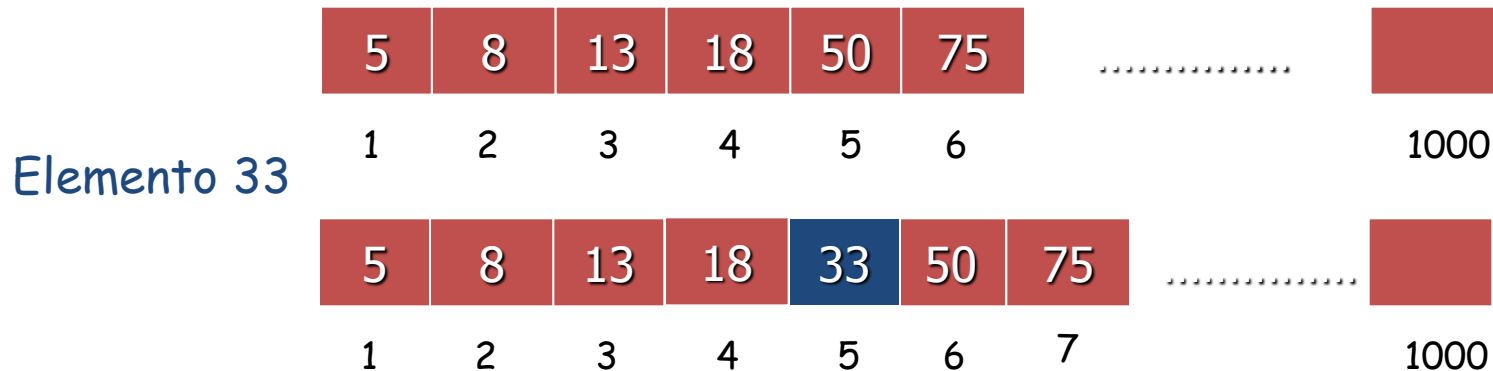
Tipo Vector: Insertar un elemento

Recordemos que la operación de Insertar un elemento en un vector admite dos posibilidades:

1 Insertar un elemento en una posición determinada

ya lo vimos

2 Insertar un elemento manteniendo un orden determinado



Tipo Vector: Insertar un elemento en un vector ordenado

- ➔ Esta operación requiere verificar el espacio disponible, buscar la posición correspondiente manteniendo el orden y luego insertar el elemento en el vector

Verificar espacio en el vector

Determinar posición donde se insertara

Insertar elemento en la posición determinada

```
Const
  DimF = ...
Type
  Indice  = 0.. DimF;
  vector = Array [ 1..DimF] of integer;
```

```
Procedure InsertarElemOrd (var v: vector; var dimL: indice;
                           elem : TipoElem; var exito: boolean);
```

```
var pos: indice;
```

```
Begin
```

```
  exito := false;
```

```
  if (dimL < dimF)
```

```
  then begin
```

```
    pos:= BuscarPosicion (elem, v, dimL);
```

```
    Insertar (v, dimL, pos, elem);
```

```
    exito := true;
```

```
  end;
```

```
end;
```

*Revisar BuscoPosElemOrd
y adaptarlo!!*

*Revisar InsertarPos
y adaptar*

Adaptamos "BuscoPosElemOrd" para escribir "BuscarPosicion"

```
Const
  DimF = ...
Type
  Indice = 0.. DimF;
  vector = Array [1..DimF] of integer;
```

```
Function BuscoPosElemOrd (x: integer; v:Vector; dimL: Indice): Indice;
var pos : Indice;
begin
  pos:=1;
  while (pos <= dimL) and (x > v[pos]) do
    pos:=pos+1;
  if ( pos > dimL ) or (x < v [pos]) then pos:=0;
  BuscoPosElemOrd:= pos;
end;
```

Ya lo vimos!!

```
Function BuscarPosicion ( x: integer; v:Vector; dimL: Indice): Indice;
var pos : Indice;
begin
  pos:=1;
  while (pos<=dimL) and (x > v[pos]) do
    pos:=pos+1;
  BuscarPosicion:= pos;
end;
```

BuscoPosElemOrd adaptado

Adaptamos "InsertarPos" para escribir "Insertar"

```
Const
  DimF = ...
Type
  Indice = 0.. DimF;
  vector = Array [ 1..DimF] of integer;
```

```
Procedure INSERTARPOS(var v: vector; var dimL: integer; elemento: integer;
                      pos: integer; var exito: boolean );

var i : integer;
Begin
  exito := false;
  if (dimF > dimL) and ((pos>=1) and (pos<= dimL))
    then begin
      exito := true;
      for i := dimL downto pos do
        v [ i + 1 ] := v [ i ] ;
      v [pos] := elemento;
      dimL := dimL + 1;
    end;
end;
```

Ya lo vimos!!

```
Procedure Insertar (var v:vector; var dimL:Indice; pos: Indice; elem:integer);
var j: indice;
begin
  for j:= dimL downto pos do
    v [ j +1 ] := v [ j ] ;
  v [ pos ] := elem;
  dimL := dimL + 1;
End;
```

InsertarPos adaptado

Tipo Vector: Insertar un elemento en un vector ordenado

Procedure InsertarElemOrd (**var** v: vector; **var** dimL: indice; elem : TipoElem;
 var exito: boolean);

```
Function DeterminarPosicion ( x: integer; v:Vector; dimL: Indice): Indice;  
  var pos : Indice;  
  begin  
    pos:=1;  
    while (pos<=dimL) and (x > v[pos]) do  
      pos:=pos+1;  
    DeterminarPosicion:= pos;  
  end;
```

Nuevo!!!

```
Procedure Insertar (var v:vector; var dimL:Indice; pos: Indice; elem:integer);  
  var j: indice;  
  begin  
    for j:= dimL downto pos do  
      v [ j +1 ] := v [ j ];  
    v [ pos ] := elem;  
    dimL := dimL + 1;  
  End;
```

Nuevo!!!

var pos: indice;

Begin

 exito := false;

if (dimL < dimF) **then begin**

 pos:= DeterminarPosicion (elem, v, dimL);

 Insertar (v, dimL, pos, elem);

 exito := true;

end;

end;

Tipo vector: Búsqueda en Arreglos Ordenados

Método 2 – Búsqueda Dicotómica

- Se aplica cuando los elementos tienen orden.
- Se compara el valor buscado (x) con el ubicado en el medio del vector (a):
 - Si el elemento ubicado al medio del vector es igual a x , entonces la búsqueda termina.
 - Si no es el valor buscado, debería quedarse con la mitad del vector que conviene, para seguir la búsqueda. Este paso se repite tantas veces hasta que se acaba el vector o encuentro el valor.

Tipo vector: Búsqueda en Arreglos Ordenados

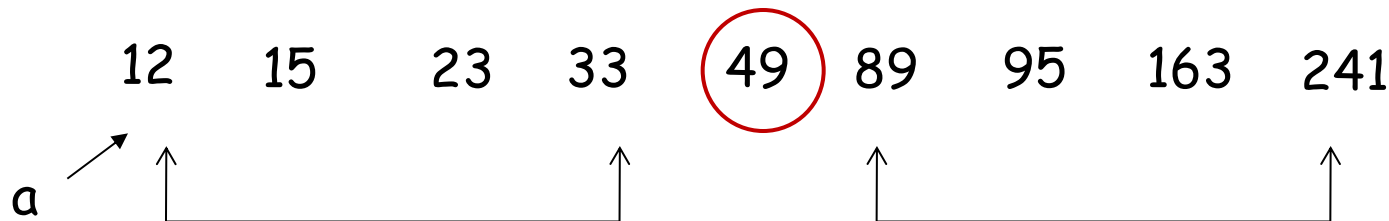
Método 2 – Búsqueda Dicotómica

Primera vez

Elemento buscado $X = 89$

➡ Se calcula la posición del medio del vector original

Si *Pri=1*
Ult=9



$a[\text{medio}] = a[5] = 49$

Dado que $89 > 49$, se trabajará con el “subvector” del medio al final

Tipo vector: Búsqueda en Arreglos Ordenados

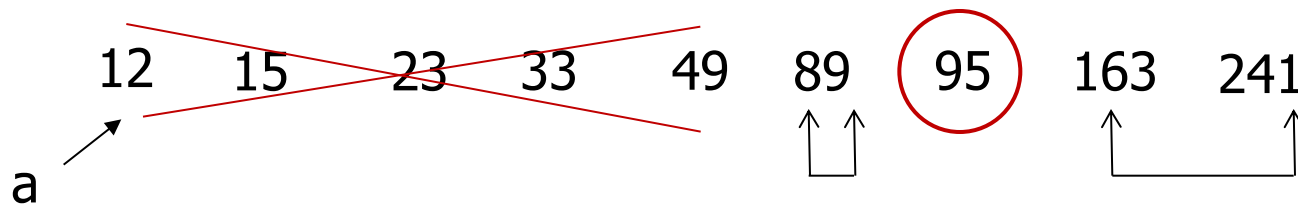
Método 2 – Búsqueda Dicotómica

Segunda vez

Elemento buscado $X = 89$

- ➡ Se descarta la primera parte
- ➡ Se calcula la posición del medio del "subarreglo" delimitado por:

*si Pri=6
Ult=9*



$$a[\text{medio}] = a[7] = 95$$

Dado que $89 < 95$, trabajo con el "subvector" del principio al medio

Tipo vector: Búsqueda en Arreglos Ordenados

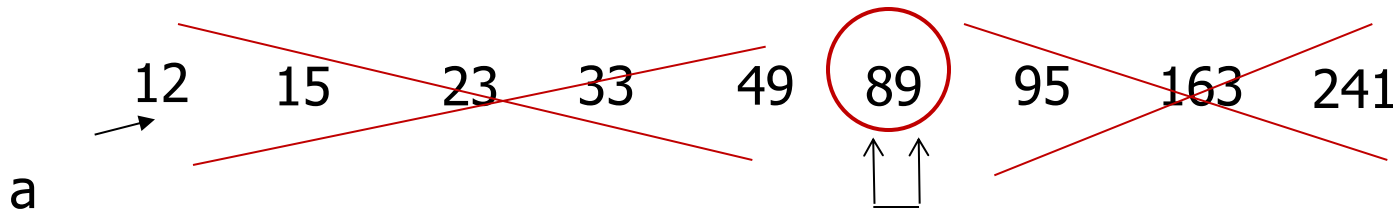
Método 2 – Búsqueda Dicotómica

Tercera vez

Elemento buscado $X = 89$

- ➡ Se descarta la "segunda" parte del "subarreglo" (de 7 a 9)
- ➡ Se calcula la posición del medio del "subarreglo" delimitado por:

Pri=6
si
Ult=6



$$a[\text{medio}] = a[6] = 89$$

89 = 89 se encontró el elemento!!!

Tipo vector: Búsqueda en Arreglos Ordenados

Método 2 – Búsqueda Dicotómica

Observaciones:

- Cada vez que se toma la mitad del arreglo, se va disminuyendo el tamaño del mismo.
- El proceso termina cuando encuentro el elemento, o cuando el vector se hace tan pequeño que no quedan mas elementos, y por lo tanto se puede deducir que el elemento no se encuentra en el vector.

Tipo vector: Búsqueda en Arreglos Ordenados

```
Procedure BusquedaBin ( var v: Vector; var j: Indice;  
                        dimL: Indice, x : TipoElem) ;
```

```
Var pri, ult, medio : Indice ;
```

```
Begin
```

```
  j := 0 ;
```

```
  pri := 1 ;
```

```
  ult := dimL;
```

```
  medio := (pri + ult ) div 2 ;
```

```
While ( pri <= ult ) and ( x <> v [medio]) do begin
```

```
  If ( x < v [ medio ] ) then ult:= medio -1 ;
```

```
    else pri:= medio+1 ;
```

```
    medio := ( pri + ult ) div 2 ;
```

```
end ;
```

```
If pri <= ult then j := medio
```

```
    else j := 0 ;
```

```
End ;
```

Calcula la posición del medio del vector

Se queda con la primera mitad

Se queda con la segunda mitad

Recalcula la posición del medio del "subvector"

¿Qué ocurre cuando el elemento buscado no está en el vector?

Características de la Búsqueda Dicotómica

- Se aplica cuando los elementos tienen orden. Caso contrario debería ordenarse el vector previamente.
-

- Número medio de comparaciones $(1 + \log_2(\text{dimL} + 1)) / 2$.
-

- Cuando dimL crece el número medio de comparaciones es $\log_2(\text{dimL} + 1) / 2$.
-

Eficiencia de la Búsqueda secuencial y dicotómica

	busqueda secuencial		Busqueda dicotómica	
	número medio de comparaciones		Número máximo de comparaciones	
N	localizado	no localizado	Localizado	no localizado
7	4	7	3	3
100	50	100	7	7
1.000	500	1.000	10	10
1.000.000	500.000	1.000.000	20	20

Ejercitación



Un centro de deportes nos ha encargado un sistema para el manejo de sus clientes. En el centro se ofrecen 5 actividades distintas. De cada cliente se conoce código de cliente, DNI, apellido, nombre, edad, el número de actividad que realiza, mes y año del último pago. La lectura finaliza cuando llega el DNI 0.

Se pide realizar un módulo para cada uno de los siguientes incisos:

- a) Almacenar el precio mensual de las actividades a través de la lectura de los valores.
- b) Generar la estructura de datos que almacene los clientes del centro de deportes (como máximo 500), leyendo los datos de los clientes hasta ingresar un DNI -1, **ordenados por código de cliente**.
- c) Informar la cantidad de clientes que realizan cada actividad y la actividad con mayor recaudación mensual.
- d) Eliminar de la estructura el cliente correspondiente a un código que se lee.
- e) Eliminar todos los clientes que realizan la actividad 3.